

Mobil Geliřtiriciler İin Giriř Rehberi

Ömer Faruk Yavuz

November 2025

Introduction

Bu doküman, mobil geliştirme dünyasına React Native ekseninden adım atmak isteyen geliştiriciler için pratik bir giriş rehberi olarak tasarlanmıştır. Amaç, yalnızca teknik detayları art arda sıralamak değil; *modern mobil uygulama geliştirme sürecini* uçtan uca zihinde canlandırabilecek bütüncül bir çerçeve sunmaktır.

Metin boyunca; bileşen temelli düşünme biçimi, navigation yaklaşımları, durum yönetimi, performans optimizasyonu, yerel (native) modüllerle entegrasyon, paket yönetimi, dağıtım (build / release) süreçleri ve platforma özgü (Android / iOS) temel kavramlar sistematik biçimde ele alınmıştır. Böylece okuyucunun yalnızca “nasıl kod yazacağını” değil, aynı zamanda yazdığı kodun platform tarafında *neye dönüştüğünü* ve yaşam döngüsünün hangi aşamalardan geçtiğini görmesi hedeflenmiştir.

Rehber, deneyim seviyesinden bağımsız olarak, iki tür ihtiyaca cevap vermeyi amaçlar:

- Mobil geliştirmeye yeni başlayanlar için: Temel kavramları, en sık kullanılan araçları ve React Native ekosistemini sindirilebilir bir dille tanıtmak.
- Halihazırda geliştirme yapanlar için: Dağınık bilgi kırıntılarını bir araya getiren, kavramsal boşlukları dolduran, “neden böyle tasarlanmış?” sorusuna yanıt veren bir referans çerçevesi oluşturmak.

Bu çerçevede doküman, bir “hızlı başlangıç”tan ziyade, mobil geliştiricinin kafasında sağlam bir zihinsel model inşa etmeyi amaçlayan bir giriş katmanı olarak okunmalıdır. Detaylı dokümantasyonların ve resmi kaynakların öncesinde, nereden başlayacağını bilemeyen geliştiriciye pusula işlevi görmesi hedeflenmiştir.

1. Activity / ViewController Lifecycle

Mobil uygulamalarda ekranlar kullanıcıdan bağımsız olarak **işletim sistemi tarafından yönetilir**. Bu durum, web'deki SPA (Single-Page Application) modelinden tamamen farklıdır.

Android Activity Lifecycle

```
onCreate() → UI oluşturulur, bağımlılıklar yüklenir.  
onStart() → Ekran görünür hale gelir.  
onResume() → Kullanıcı ekranı aktif kullanır.  
---  
onPause() → Kısmen görünür (popup, başka uygulama).  
onStop() → Tamamen görünmez, background'a alınır.  
onDestroy() → OS tarafından tamamen öldürülür.
```

iOS ViewController Lifecycle

```
viewDidLoad()  
viewWillAppear()  
viewDidAppear()  
viewWillDisappear()  
viewDidDisappear()  
deinit()
```

Bu metodlar, ekranın ne zaman oluşturulacağı, görünür olacağı veya yok edileceği konusunda kritik kontrol noktaları sağlar.

2. Background → Foreground Geçişleri

Mobil işletim sistemleri agresif davranır:

- RAM sıkıştığında arka plandaki Activity'leri öldürür.
- CPU yükünde animasyonları durdurur.
- Uzun süre background'da kalan uygulamaları askıya alır (suspend).

Bu süreçte oluşan tipik geçişler:

```
onPause → Kullanıcı görmüyor.  
onStop → Uygulama tamamen arka planda.  
OS Kill → Gerekirse uygulama tamamen yok edilir.
```

Uygulama foreground'a döndüğünde tekrar `onCreate` veya `onResume` çalışabilir.

3. Memory Reclaim Davranışları

Android, arka plandaki Activity'leri RAM kazanmak için *çok agresif* şekilde sonlandırır.

Bu nedenle:

- UI state yalnızca RAM'de tutulmamalıdır.
- Listener / subscription'lar **onDestroy**'da temizlenmelidir.
- Sensor, kamera, mikrofon gibi ağır kaynaklar **onPause**'da durdurulmalıdır.
- Form verileri ViewModel, MMKV, Redux vb. persistent yapılarda saklanmalıdır.

UI State Yalnızca RAM’de Tutulmamalıdır

Modern mobil ve web uygulamalarında **UI state** yalnızca RAM üzerinde (component içi değişkenlerde) tutulursa, uygulama kapandığında veya işletim sistemi süreci sonlandırdığında tüm state kaybolur. Bu nedenle kritik uygulama durumlarının mutlaka kalıcı bir depolama alanına yazılması gerekir.

1. RAM Neden Güvenilir Değildir?

- **Uygulama arka plana alınıp öldürülebilir.** Android ve iOS düşük bellek durumunda uygulama süreçlerini kapatır.
- **Refresh / reload / navigation durumlarında state sıfırlanır.** Web’de F5, mobilde cold start, ekran döndürme gibi durumlarda component life-cycle yeniden oluşur.
- **OS memory reclaim yapabilir.** İşletim sistemi RAM baskısı olduğunda uygulamayı kapatabilir.

Bu nedenle component içinde tutulan state (React `useState`, Angular component properties, context, MobX, Zustand, NgRx store vb.) kalıcı değildir.

2. Hangi State’ler Kalıcı Olmalıdır?

Aşağıdaki state türleri mutlaka persistent storage’a yazılmalıdır:

- **Authentication bilgileri:** access token, refresh token, session metadata.
- **Kullanıcı tercihleri:** dil, tema, font boyutu, izinler.
- **Offline-first veriler:** önbelleğe alınmış API sonuçları, gönderilmemiş formlar, lokal mesajlar.
- **Kullanıcı ilerleme verileri:** onboarding tamamlandı mı, chat geçmiş, notlar, tamamlanmış adımlar.

3. UI State vs Persistent App State

UI State (RAM - Geçici)

- dropdown açık mı?
- modal görünür mü?
- aktif tab hangisi?
- loading state

Persistent App State (Kalıcı)

- kullanıcı giriş durumu
- token bilgileri
- uygulama dili
- offline cache
- onboarding tamamlanma durumu

4. Persistent Storage Seçenekleri

React Native

- MMKV
- AsyncStorage
- SQLite / WatermelonDB
- Realm

Web / Angular

- LocalStorage
- IndexedDB
- Service Worker cache
- NgRx store + localStorage sync

5. Sonuç

UI state geçicidir; uygulamanın kritik durumları RAM’de değil, kalıcı depolamada saklanmalıdır. Bu, veri kaybını önler, performansı artırır, kullanıcı deneyimini güçlendirir.

React Native'de Lifecycle Nasıl Çalışır?

React Native, üç katmanlı bir mimaridir:

Android (Java/Kotlin)
iOS (Objective-C/Swift)
JavaScript (React)

Native taraf Activity/ViewController yaşam döngüsünü yönetirken, JS tarafı **SPA gibi davranır**. Bu iki dünya arasındaki iletişim **Bridge** üzerinden sağlanır. React Native, native yaşam döngüsü değişikliklerini JavaScript tarafına şu API ile iletir:

AppState

active → Uygulama ekranda.
background → Arka planda.
inactive → Geçiş aşaması.

Örnek Kullanım

```
import { AppState } from "react-native";

useEffect(() => {
  const sub = AppState.addEventListener("change", state => {
    if (state === "active") console.log("Foreground");
    if (state === "background") console.log("Background");
  });

  return () => sub.remove();
}, []);
```

6. Native → JS Davranışı

Native Tarafta:

- `onPause` → JS tarafına background sinyali gönderir.
- `onResume` → JS tarafına active sinyali gönderir.
- `onDestroy` → JS engine tamamen kapanır.

JS Tarafta:

- Tüm `useEffect` cleanup'ları tetiklenir.
- Timer'lar durur.
- WebSocket, kamera, sensor gibi ağır kaynaklar durdurulmalıdır.

Mobil ve Web Arasındaki Kritik Farklar

Konu	Web	Mobil
OS sayfayı öldürebilir	Hayır	Evet
Background/foreground state	Basit	Çok kritik
Memory reclaim	Hafif	Agresif
Lifecycle yönetimi	SPA	OS-managed
App state yönetimi	JS hafızası	Native + JS

React Native’de JS → Native Geçişi

React Native, JavaScript (JS) kodu ile platforma özgü (iOS/Android) *native* API’ler arasında iki farklı mimari kullanır:

1. Eski Mimari: **Bridge** Tabanlı Model
2. Yeni Mimari: **JSI + TurboModules + Fabric**

Aşağıda iki mimari arasındaki fark ve geçiş sürecinin teknik detayları anlatılmaktadır.

1. Eski Mimari: Bridge Tabanlı Yapı

React Native 0.69 ve öncesinde JavaScript kodu ile native taraf arasındaki iletişim **asenkron bir Bridge** üzerinden sağlanır.

1.1. Temel Akış

1. JS kodu bir native fonksiyon çağırır:

```
NativeModules.Location.getCurrentPosition();
```

2. Bu çağrı **JSON formatına dönüştürülür** ve Bridge’e iletilir:

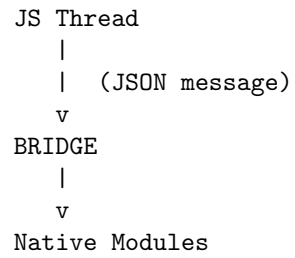
```
{
  "module": "Location",
  "method": "getCurrentPosition",
  "params": []
}
```

3. Bridge bu mesajı **Native Modules Message Queue**’suna aktarır.
4. Native taraf ilgili modülü ve metodu çağırır.
5. Native sonuç döndürmek isterse aynı Bridge üzerinden tekrar JSON mesajı olarak JS’e geri gönderir.

1.2. Eski Mimari’nin Sınırlamaları

- JSON serileştirme/deserileştirme maliyetlidir.
- JS ve Native arasında **thread hop** çok fazladır.
- Büyük veri taşımak (örneğin: video frame’leri) verimsiz ve yavaştır.
- Senkron (synchronous) fonksiyon çağrıları mümkün değildir.

1.3. Zihin Modeli



Yeni Mimari: JSI + TurboModules + Fabric

Modern React Native mimarisinde Bridge ortadan kaldırılmıştır. JS ile native taraf arasında doğrudan **C++ tabanlı bir arayüz (JSI)** kullanılır.

JSI (JavaScript Interface)

JSI, JavaScript kodunun C++ seviyesindeki objelere ve fonksiyonlara **doğrudan erişmesini** sağlayan yeni düşük seviye API'dir.

Temel Akış

1. JS kodu bir TurboModule fonksiyonunu çağırır:

```
Location.getCurrentPosition();
```

2. Bu çağrı JSON'a dönüştürülmez; doğrudan **C++ pointer üzerinden** ilgili native fonksiyon invoke edilir.
3. Native fonksiyon sonucu yine JSI üzerinden JS runtime'a döner.

Yeni Mimari'nin Avantajları

- JSON yoktur, **serialization overhead** ortadan kalkar.
- Sıfıra yakın gecikme (**low-latency**) ile çağrı yapılır.
- **Zero-copy** veri aktarımı mümkündür (özellikle medya verilerinde).
- Senkron fonksiyon çağrıları mümkündür.
- Daha yüksek FPS, daha akıcı animasyonlar ve gesture performansı sağlanır.

Zihin Modeli

```
JS Thread
  |
  | (C++ pointer call)
  v
JSI
  |
  v
TurboModules / Native Code
```

Eski ve Yeni Mimari Arasındaki Fark

- **Eski RN:** JS → JSON message → Bridge → Native
- **Yeni RN:** JS → JSI (C++) → Native

Özet

Eski mimaride JS ile native arasında JSON tabanlı, yavaş bir köprü bulunurdu. **Yeni mimaride** bu köprü kaldırılmış, JS kodu native fonksiyonları C++ üzerinden doğrudan çağırabilir hale gelmiştir. Bu mimari, performansı ciddi şekilde artırır ve modern RN mimarisinin temelidir.

React Native'in Limitasyonları

React Native, SPA modeli nedeniyle bazı durumlarda zayıftır:

- AppState gecikebilir.
- Bazı Android cihazlarda background event'leri engellenir.
- WebSocket bağlantıları background'da kopabilir.
- Kamera/video background'da otomatik kapanmaz.
- Timer'lar background'da çalışmaz.

Dikkat Gerektiren Durumlar

- Screen/audio recording → `onPause`'da durdurulmalı.
- WebSocket → background dönüşünde yeniden bağlanmalı.
- Kamera → background'da mutlaka kapatılmalı.
- Form state → unmount olursa kaybolmamalı.
- Location tracking → background permission gerekli.

Kavramsal Benzetme

React Native:

- Native lifecycle = Angular Component Lifecycle
- App background = Browser tab inactive
- Bridge = Zone.js benzeri event scheduler

Bu nedenle React Native uygulamaları **web SPA gibi** görünse de, arka planda **tamamen native lifecycle** ile yaşar.

React Native’de AppState Nedir?

React Native’deki **AppState**, uygulamanın işletim sistemi tarafından hangi yaşam döngüsü durumunda olduğunu (foreground, background, inactive) takip eden bir API’dir. Mobil uygulamalar sürekli olarak OS tarafından duraklatılabilir, arka plana alınabilir veya öldürülebilir. Bu nedenle uygulama durumlarını yönetmek için AppState kritik bir yapıdır.

1. AppState Durumları

- **active**
Uygulama ekranda görünür durumdadır ve kullanıcı etkileşimi mümkündür.
- **background**
Uygulama arka plandadır; kullanıcı tarafından görülmez. OS bu aşamada uygulamayı öldürebilir.
- **inactive**
iOS’ta sık görülen bir ara durumdur. Arama ekranı, kontrol menüsü, bildirim overlay’i gibi geçici durumlarda oluşur.

2. Örnek Kullanım

```
import { AppState } from 'react-native';

AppState.addEventListener('change', (state) => {
  console.log("Yeni durum: ", state);
});
```

3. AppState Neden Gereklidir?

Mobil işletim sistemleri (iOS ve Android) bellek ve CPU yönetimi gereği uygulamaları sık sık durdurur, dondurur veya tamamen sonlandırır. **AppState** bu geçişleri JS tarafında yönetebilmek için kullanılır.

- **OS uygulamayı arka plana attığında** işlemler durdurulabilir, timer’lar temizlenebilir, socket bağlantıları kapatılabilir.
- **Uygulama tekrar active olduğunda** veriler yeniden yüklenebilir, API çağrıları tazelenenebilir, sayfa yenilenebilir.
- **Background durumunda kaynak tüketimi azaltılabilir:** konum servisleri, sensör okuma, animasyonlar, audio/video oynatma vb.

4. Yaygın Kullanım Senaryoları

- Arka plana geçince **token yenilemeyi durdurmak**
- Uygulama tekrar öne gelince **API'ları yeniden fetch etmek**
- Video ve ses oynatmayı **otomatik durdurmak**
- Form verilerini **arka plana geçmeden önce saklamak**
- Konum, Bluetooth veya sensör okumalarını **arka planda kapatmak**

5. AppState Zihin Modeli

React Native, Native yaşam döngülerini AppState üzerinden JS katmanına taşır:

iOS Lifecycle	Android Lifecycle	React Native AppState
applicationDidBecomeActive	onResume	active
applicationDidEnterBackground	onPause	background
Geçiş durumları	Benzer geçişler	inactive

6. Özet

AppState, React Native uygulamasının foreground, background ve inactive durumlarını JS katmanında takip etmesini sağlayan yaşam döngüsü köprüsüdür. Mobil uygulamalarda OS uygulamayı sık sık durdurduğu için AppState, enerji yönetimi, veri yenileme, oturum yönetimi ve kaynak kontrolü için hayati öneme sahiptir.

Memory Pressure & Low Memory Kill

Mobil cihazlarda RAM sınırlıdır. Bu nedenle iOS ve Android işletim sistemleri, bellek kritik seviyelere düştüğünde uygulamalara baskı yapar, uyarır ve gerekirse uygulama sürecini tamamen sonlandırır. Bu duruma **memory pressure** denir.

1. Low Memory Warning

iOS, bellek seviyesi kritikleştiğinde `UIApplicationDidReceiveMemoryWarningNotification` üretir. Bu uyarı geliştiriciye:

- gereksiz cache'lerin temizlenmesi,
- büyük resimlerin bellekten atılması,
- geçici objelerin serbest bırakılması

gerektiğini bildirir.

Uyarı dikkate alınmazsa işletim sistemi kısa süre içinde uygulamayı **“Terminated due to memory pressure”** hatasıyla sonlandırır.

2. Background Task Kill

Uygulama arka plana geçtiğinde OS yalnızca birkaç saniyelik yaşam hakkı tanır. Bu süreden sonra uygulama:

- iOS'ta *suspended* durumuna alınır,
- Android'de LMK (Low Memory Killer) tarafından sınıflandırılır.

Bellek ihtiyacı artarsa OS en önce arka plandaki uygulamaları öldürür.

3. App Process'inin OS Tarafından Kesilmesi

İşletim sistemi aşağıdaki durumlarda süreci zorla sonlandırabilir:

- bellek yetersizliği,
- yüksek CPU tüketimi,
- battery saver modları,
- kontrolsüz timer veya animasyon kullanımı,
- memory leak.

Android loglarında genellikle `LowMemoryKill` mesajı, iOS crash loglarında `jetsamEvent` görülür.

4. Large Image Footprint

Diskte 3MB olan bir görsel RAM’de decode edildiğinde 20–50MB yer kaplayabilir. Örneğin:

$$1080 \times 1920 \times 4byte \approx 8.3MB$$

Ekranda birden fazla büyük görsel bulunması, uygulamanın kısa sürede memory pressure altında kalmasına yol açar. Bu nedenle:

- downsampling,
- WebP/AVIF formatları,
- FastImage,
- uygun cache stratejileri

kullanılmalıdır.

5. Memory Leak Detection

Memory leak’ler zamanla native heap’in büyümesine ve OS tarafından uygulamanın öldürülmesine neden olur.

Android Araçları

- LeakCanary
- Android Studio Memory Profiler

iOS Araçları

- Xcode Instruments (Leaks, Allocations, Memory Graph)
- Malloc Stack Logging

Özet

Bellek baskısı altında işletim sistemi uygulamayı önce uyarır, gerekirse arka planda dondurur ve en sonunda tamamen sonlandırır. Büyük resimler, bellek yönetimi hataları ve memory leak’ler mobil uygulamalarda en sık görülen sorunlardır.

React Native’de Memory Leak Detection

React Native uygulamalarında memory leak’ler hem JavaScript tarafında (Hermes heap) hem de native taraflarda (iOS/Android) oluşabilir. Bu nedenle leak tespiti yalnızca JS seviyesinde değil, platformun kendi araçlarıyla birlikte yapılmalıdır.

1. React Native’e Özel Araçlar

1.1 react-native-performance

Yeni mimari (JSI + TurboModules) ile uyumlu modern bir ölçüm aracıdır. Aşağıdaki leak türlerini tespit edebilir:

- JSI tabanlı referans sızıntıları,
- Component remount sonrası büyüyen heap,
- Render sonrası anormal memory trendleri.

1.2 react-native-memory-leak-listener

JavaScript tarafındaki GC yapılmamış nesneleri dinleyerek basit leak tespiti sağlar. Yalnızca JS heap üzerinde etkili olup native leak’leri yakalayamaz.

2. Android İçin Araçlar

2.1 LeakCanary (Önerilen)

Android’de React Native uygulamaları için en güvenilir leak tespit aracıdır. Aşağıdaki leak türlerini otomatik olarak yakalayabilir:

- Activity / Fragment leak,
- Context leak,
- View ve ViewGroup leak,
- React Native Surface / ViewManager leak,
- GestureHandler veya Navigation container leak.

2.2 Android Studio Memory Profiler

Java/Kotlin heap dağılımını, retained memory miktarını ve object lifecycle analizini gösterir.

3. iOS İçin Araçlar

3.1 Xcode Instruments

React Native ile en uyumlu ve en detaylı leak tespit aracıdır. Özellikle şu modüller kullanılmalıdır:

- **Leaks** (retain cycle tespiti)
- **Allocations** (heap büyümesini izleme)
- **Memory Graph** (strong reference zincirleri)

Tespit edilebilen leak türleri:

- UIView / UIViewController leak
- RCTBridge ve RCTModule leak
- Timer ve NotificationCenter leak
- Image decode kaynaklı leak

4. Hermes Runtime Memory Tools

Hermes 0.12+ ile JS heap snapshot alınabilir. Bu araçlar:

- JavaScript heap büyümesini,
- detached JS object'leri,
- reference cycle'ları,
- allocation timeline'ı

analiz etmeye yarar.

Heap snapshot, Flipper içinden de alınabilir.

6. Önerilen Mimari Yaklaşım

- **Android:** LeakCanary kullanılması önerilir.
- **iOS:** Xcode Instruments ile derin leak analizi yapılmalıdır.
- **JS Tarafı:** Hermes Heap Snapshot tercih edilmelidir.
- **Yeni Mimari:** JSI leak'leri için react-native-performance faydalıdır.

Özet

React Native'de leak tespiti yalnızca JS seviyesinde yapılamaz. Uygulama hem native heap hem de JavaScript heap üzerinde leak üretebileceği için, Android ve iOS'un kendi memory araçlarıyla birlikte analiz yapılması en doğru yöntemdir.

Mobile Threading Model

Mobil cihazlarda CPU çekirdeği, RAM ve batarya kısıtlı olduğundan, PC'deki gibi sınırsız thread açmak mümkün değildir. Bu nedenle işletim sistemi, thread'lerin kullanımını sıkı şekilde kontrol eder ve geliştiricinin doğru thread modelini benimsemesi gerekir.

1. UI Thread (Main Thread)

- Tüm kullanıcı etkileşimleri (touch, scroll, gesture)
- View çizimi ve layout hesaplamaları
- Animasyonlar
- Event dispatch işlemleri

UI thread bloklanırsa uygulama donar ve Android tarafında ANR (*Application Not Responding*) oluşur.

2. Worker Threads

CPU-bound işler için kullanılır:

- JSON parse
- veri işleme (filter, sort, map)
- image processing
- encryption ve compression

Sayıları sınırlıdır; aşırı thread oluşturmak battery drain ve context-switch maliyeti yaratır.

3. Background Tasks / Job Schedulers

Uzun süreli işler OS tarafından yönetilir:

- Android: WorkManager, JobScheduler, ForegroundService
- iOS: BGTaskScheduler, URLSession background transfers

Bu mekanizmalar enerji dostu şekilde zamanlanır.

React Native Özelinde Thread Modeli

React Native, hem JavaScript tarafını hem de native tarafı yürüttüğü için çok katmanlı bir thread modeline sahiptir.

1. JS Thread

JavaScript kodunun çalıştığı ana thread'dir.

- React komponentlerinin render mantığı
- state/props hesaplamaları
- timer'lar
- API callback'leri

JS thread'in bloklanması, UI'nın geç tepki vermesine neden olur.

2. UI Thread

Native arayüzün işlendiği thread'dir.

- touch/gesture event'leri
- layout ve render işlemleri
- native animasyonlar

3. Native Modules Thread

Native modüllerin (kamera, dosya sistemi, konum, kontaklar, Bluetooth vb.) çalıştığı thread'dir.

4. Background Threads

Arka plan havuzlarında şu işlemler yürütülür:

- ağ işlemleri (networking)
- dosya okuma/yazma
- image decoding
- Hermes GC thread'leri

Özet

Mobilde threading modeli dikkatle yönetilmelidir. UI thread'in bloklanması donmaya, JS thread'in bloklanması gecikmeye, native thread bloklarının ise modül hatalarına neden olur. React Native üç ana thread (JS, UI, Native Modules) ve çeşitli background thread havuzları üzerinden çalışır.

React Native’de Background Thread Açma Yöntemleri

React Native uygulamalarında JavaScript tarafı tek bir **JS thread** üzerinde çalışır. Bu nedenle ağır iş yüklerinin JS thread üzerinde çalıştırılması, UI gecikmelerine ve uygulama donmalarına neden olur. Background işlemleri çalıştırmak için aşağıdaki yöntemler kullanılır.

1. JS Worker Thread (Ek JS Thread Açmak)

1.1 react-native-threads

JavaScript tarafında ek bir worker thread oluşturmayı sağlar. Bu yöntem CPU-bound işlemlerin JS thread’den ayrılmasına yardımcı olur.

```
import { Thread } from 'react-native-threads';

const t = new Thread('./worker.js');
t.onmessage = (msg) => console.log(msg);
t.postMessage('start');

worker.js:

import { self } from 'react-native-threads';

self.onmessage = () => {
  let r = heavyComputation();
  self.postMessage(r);
};
```

Avantaj: JS thread’i bloklamadan hesaplama yapılabilir. Dezavantaj: Yeni mimari (JSI) tam uyumlu değildir.

2. JSI Tabanlı Worker (Yeni Mimari)

React Native 0.70+ ile birlikte JSI (JavaScript Interface) kullanılarak native tarafta thread’ler açılabilir. Zero-copy veri aktarımı ve yüksek performans sağlar.

2.1 C++ Tabanlı Worker Örneği

```
// MyWorker.cpp
void runHeavyTask(jsi::Runtime& rt, const jsi::Value* args, size_t count) {
  std::thread([](){
    long result = doHeavyMath();
    // callback ile JS’e gönderilir
  }).detach();
}
```

JavaScript tarafı:

```
import MyWorker from 'react-native-myworker';
MyWorker.runHeavyTask(() => console.log('done'));
```

Avantajlar:

- En hızlı background hesaplama yöntemi
- JSON/bridge overhead yoktur
- Native thread pool kullanır

3. Native Tarafında Thread Açmak

React Native, native modüller üzerinden arka plan thread'leri kullanmayı destekler.

3.1 Android (Kotlin/Java)

```
Thread {
    val result = heavyWork()
    reactContext
        .getJSModule(DeviceEventManagerModule
            .RCTDeviceEventEmitter::class.java)
        .emit("taskDone", result)
}.start()
```

3.2 iOS (Swift)

```
DispatchQueue.global(qos:.background).async {
    let result = heavyWork()
    self.sendEvent(withName: "taskDone", body: result)
}
```

JavaScript tarafı:

```
const emitter = new NativeEventEmitter(MyNativeModule);
emitter.addListener("taskDone", console.log);
```

Avantaj: Gerçek, tam arka plan thread kapasitesi. Dezavantaj: Native kod yazmayı gerektirir.

4. Headless JS (Android)

Android'de uygulama kapalı olsa bile arka planda JavaScript çalıştırılabilir. Uzun süreli veya planlı işler için uygundur.

Örnek

```
// index.js
AppRegistry.registerHeadlessTask('MyTask', () => async () => {
  await doSomething();
});

// Java
public class MyService extends HeadlessJsTaskService {
  @Override
  protected HeadlessJsTaskConfig getTaskConfig(Intent intent) {
    return new HeadlessJsTaskConfig("MyTask",
      Arguments.createMap(),
      5000,
      true);
  }
}
```

Kısıtlama: Sadece Android tarafından desteklenir.

5. OS Zamanlamalı Background İşleri

5.1 react-native-background-fetch

Operating System'in enerji optimizasyonlu job scheduler'ı ile çalışır.

```
BackgroundFetch.configure(
  { minimumFetchInterval: 15 },
  async () => {
    await syncData();
    BackgroundFetch.finish();
  }
);
```

Avantaj:

- App kapalı olsa bile çalışabilir
- OS batarya optimizasyonu uygular

Dezavantaj: Zamanlama OS tarafından kontrol edilir.

Özet Tablosu

İhtiyaç	Çözüm	Not
JS hesaplamalarını ayırmak	JS Worker Thread	Kolay kurulum
En hızlı background çalışma	JSI C++ Worker	Zero-copy, modern
Native API ile async iş	DispatchQueue / Thread	En stabil yol
App kapalıyken çalışmak	Headless JS	Android özel
OS zamanlamalı işler	BackgroundFetch	Enerji dostu

Genel Sonuç

React Native'de doğrudan yeni bir JS thread açmak mümkün değildir; ancak worker thread'ler, JSI tabanlı native thread'ler, Headless JS ve OS background job mekanizmaları kullanılarak arka planda görevler güvenli şekilde yürütülebilir.

ANR — Application Not Responding

Android işletim sistemi, **UI thread’in 5 saniye boyunca** yanıtsız kalması durumunda uygulamayı **ANR (Application Not Responding)** hatasıyla sonlandırır. Bu durum, kullanıcıya “uygulama yanıt vermiyor” uyarısı olarak yansır ve süreç çoğunlukla uygulamanın öldürülmesiyle sonuçlanır.

1. ANR’a Neden Olan Durumlar

ANR’ın temel nedeni UI thread’in bloklanmasıdır. UI thread tıkanıldığında:

- touch/scroll event’leri işlenemez,
- view redraw işlemleri durur,
- layout hesaplamaları yapılamaz,
- event loop kilitlenir.

React Native uygulamalarında aşağıdaki işlemler ANR’a sıkça neden olur:

- **Büyük JSON parse işlemleri**
- **Geniş array üzerinde map/filter/reduce**
- **Kriptografik işlemler** (hash, encrypt, PBKDF2)
- **Büyük görsel decode işlemleri**
- **Senkron depolama erişimi** (blocking I/O)

2. ANR’ın Temel Mekanizması

Android OS, UI thread’in **5 saniye boyunca input event’lerine yanıt vermemesini** kritik bir hata olarak kabul eder. Bu süre içinde:

- yeni frame üretilemez,
- kullanıcı eylemleri kuyruğa alınır ancak işlenmez,
- “looper” tıkanır.

Sonuç olarak OS, kullanıcı deneyimini korumak adına uygulamayı sonlandırır.

3. Çözüm Yaklaşımları

3.1 Background Thread Kullanımı

UI thread üzerinde ağır işler çalıştırmak yerine:

- native thread (DispatchQueue / Thread / Executor),
- worker thread (React Native Threads),
- JSI tabanlı C++ thread'leri

kullanılmalıdır.

3.2 JSI / TurboModules ile Hesaplamaların Native Tarafa Alınması

Yeni mimaride (JSI + TurboModules):

- JSON serileştirme yoktur,
- bridge tıkanması yaşanmaz,
- zero-copy memory sayesinde büyük veri akışı mümkündür.

Bu yöntem CPU-bound işlemlerin JS thread'den kaldırılmasını sağlar.

3.3 Batched Rendering ve UI Optimizasyonu

UI tarafında:

- gereksiz re-render'ların azaltılması,
- heavy component mount işlemlerinin bölünmesi,
- virtualized list bileşenlerinin kullanılması

UI thread üzerindeki yükü düşürerek ANR riskini azaltır.

4. Özet

ANR, Android'de UI thread'in 5 saniyeden uzun süre bloklanması sonucu uygulamanın OS tarafından öldürülmesidir. Ağır hesaplamalar, büyük JSON işlemleri ve senkron I/O gibi maliyetli işler her zaman background thread'lere taşınmalı, JSI tabanlı çözümler ve batched rendering ile UI yükü minimize edilmelidir.

Mobile Networking Limitations

Mobil cihazlarda ağ iletişimi, masaüstü veya sunucu ortamlarına göre çok daha değişken, kırılgan ve maliyetlidir. Bu nedenle mobil uygulamalar hem bağlantı kalitesindeki belirsizliklere hem de enerji ve veri tüketimi kısıtlarına göre tasarlanmalıdır.

1. Unstable Network (Kararsız Bağlantı)

Mobil cihazlar sürekli olarak:

- Wi-Fi $\leftrightarrow$ Mobil veri değişimi,
- zayıf sinyal bölgeleri,
- tünel/metro kesintileri,
- hücre değişimi (cell handoff)

gibi bağlantı dalgalanmalarına maruz kalır.
Bu durum, HTTP çağrılarında:

- paket kaybı,
- DNS hataları,
- response gecikmesi,
- TCP yeniden aktarımı

gibi maliyetlere yol açar.

2. Captive Networks

Bazı Wi-Fi ağları internet erişimi sağlamadan önce kullanıcıya bir doğrulama sayfası gösterebilir (örneğin otel, havaalanı). Bu durumdaki ağlara:

captive network

denir.

Uygulama internete bağlı görünse bile:

- istekler engellenir,
- DNS cevapsız kalır,
- HTTP redirect oluşur,
- TLS handshake başarısız olur.

3. Packet Loss (Paket Kaybı)

Mobil ağlarda paket kaybı PC ortamlarına göre çok daha yüksektir. Paket kaybı, özellikle TCP üzerinde:

- yeniden iletim (retransmission),
- artan gecikme,
- düşük throughput,
- bağlantı kopması

gibi etkiler yaratır.

4. Timeout Problemleri

Mobil ağlarda:

- yüksek gecikme (latency),
- dengesiz bağlantı,
- zayıflayan sinyal,
- cihazın uyku moduna geçmesi

timeout hatalarına neden olur. Bu durum mobil uygulamalarda doğru timeout konfigürasyonunu kritik hale getirir.

5. Cold Start DNS Lookup

DNS çözümü mobilde:

- captive network müdahalesi,
- DNS caching'in olmaması,
- rota değişimi,
- farklı hücreye geçiş

gibi nedenlerle soğuk başlangıç (cold start) maliyeti yaratabilir. DNS lookup süresi uygulamanın ilk isteğini ciddi şekilde yavaşlatır.

Mobil cihazlarda DNS çözümlemesi (Domain Name System lookup), masaüstü ortamlara kıyasla çok daha değişken ve maliyetlidir. **Cold start DNS lookup**, cihazın belirli bir domain için daha önce yapılmış bir çözümlemeye ait *DNS cache* bulunmaması durumunda, DNS'in sıfırdan çözülmesi gerektiği senaryodur.

Bu durum özellikle mobil ağda ilk API isteğinin yavaşlamasına neden olur.

1. Cold Start DNS Lookup Neden Olur?

- **Captive Network Müdahalesi**

Otel, havalimanı veya restoran Wi-Fi ağları interneti engelleyip kullanıcıyı giriş (login) sayfasına yönlendirmek için DNS trafiğini ele geçirir. Bu işlem sırasında DNS cache temizlenir ve cihazın tekrar DNS çözümlemesi gerekir.

- **DNS Cache'in Olmaması**

Mobil cihazlar bağlantı değişimlerinde (Wi-Fi kapatıp açmak, uçak moduna geçmek, SIM değişimi vb.) DNS cache'i temizleyebilir. Bu durumda ilk istek her zaman cold start DNS maliyetini taşır.

- **Rota Değişimi (Routing Change)**

Mobil ağlarda cihazın baz istasyonları arasında geçişi (handoff) ile birlikte network route yeniden oluşturulur. Yeni route, yeni DNS resolver kullanabilir ve cache geçersiz olur.

- **Hücre Değişimi (Cell Handoff)**

4G/5G ağlarında cihaz farklı bir hücreye geçtiğinde ağ bağlamı (network context) yeniden kurulur. Bu süreç DNS çözümlemesinin tekrar yapılmasına neden olur.

2. DNS Lookup Maliyeti

DNS çözümlemesi mobilde genellikle:

- 50–150 ms arasında (iyi bağlantı),
- 300–1000 ms arasında (zayıf bağlantı, captive ağ),

ek gecikme yaratabilir.

Bu nedenle mobilde ilk yapılan API isteğinin toplam süresinin büyük bir kısmı DNS çözümlemesinden kaynaklanır.

3. Cold Start DNS Probleminin Etkileri

- uygulama açılış süresinin artması,
- login veya ilk fetch operasyonlarının gecikmesi,
- timeout'ların daha kolay tetiklenmesi,
- kullanıcı deneyiminde yavaşlık algısı.

4. Optimizasyon Teknikleri

Cold start DNS gecikmesini azaltmak için aşağıdaki stratejiler kullanılabilir:

- **DNS Prefetch**
Uygulama açılır açılmaz düşük maliyetli bir “ping” isteği gönderilerek DNS çözümünün önceden yapılması sağlanabilir.
- **Persistent Connections (Keep-Alive)**
HTTP/2 veya HTTP/3 kullanılarak aynı bağlantı üzerinden birden fazla istek gönderilir; DNS çözümlemesi tekrar tekrar yapılmaz.
- **Custom DNS Resolver**
Cloudflare (1.1.1.1) veya Google DNS (8.8.8.8) üzerinden DNS-over-HTTPS (DoH) kullanılarak captive network müdahaleleri azaltılabilir.
- **Retry ve DNS Fallback**
DNS hataları için farklı resolver'lara fall-back mekanizması uygulanabilir.

Özet

Cold start DNS lookup, mobil cihazın belirli bir domain için DNS çözümlemesine sıfırdan başlaması durumudur. Ağ değişimleri, captive network'ler, DNS cache kaybı ve hücre geçişleri bu maliyeti artırır. Mobil uygulamalarda ilk API çağrısının gecikmesinin önemli bir bölümü DNS çözümleme süresinden kaynaklanır.

6. HTTP/2 Multiplexing

Mobilde HTTP/2'nin sağladığı çoklu istek (multiplexing) avantajı ağ koşulları bozulduğunda tersine dönebilir:

- bağlantı koparsa tüm stream'ler aynı anda düşer,
- tek bağlantı bozulursa tüm istekler etkilenir,
- yüksek RTT, tüm stream'lerin gecikmesine neden olur.

Bu nedenle mobilde:

- bağlantı havuzu (connection pool),
- daha kısa keep-alive süreleri,
- retry mekanizmaları

önem kazanır.

7. Retry ve Exponential Backoff

Mobil ağlar hata eğilimli olduğundan isteklerin güvenilir olması için:

- retry,
- jitter,
- exponential backoff

kullanılması gereklidir.

Örnek backoff fonksiyonu:

$$t = base \times 2^{attempt} + random(0, jitter)$$

Bu yöntem hem ağı hem de sunucuyu aşırı yüklemekten yeniden deneme yapmayı sağlar.

8. Offline-First Yaklaşımları

Mobil ağın doğası gereği uygulama her an:

- offline olabilir,
- paket kaybı yaşayabilir,
- bağlantı kalitesiz olabilir.

Bu nedenle offline-first yaklaşımı önemlidir:

- istekleri sıraya alma (queue),
- başarısız istekleri sonradan gönderme,
- yerel cache (AsyncStorage, MMKV, SQLite),
- veri tutarlılık stratejileri,
- optimistic UI

kullanılmalıdır.

Özet

Mobil ağlar kararsız, değişken ve enerji kısıtlıdır. Bu nedenle ağ işlemleri tek seferde çalışacağı varsayımıyla değil; paket kaybı, retry, DNS gecikmesi, captive network ve offline senaryoları göz önünde bulundurularak tasarlanmalıdır.

App Kill Scenarios

Web uygulamalarından farklı olarak mobil uygulamalar, işletim sistemi tarafından herhangi bir anda durdurulabilir veya tamamen sonlandırılabilir. Bu durum, donanım kısıtları (RAM, CPU, batarya), enerji tasarrufu ve sistem kararlılığı nedeniyle doğal bir davranıştır ve hata olarak kabul edilmez.

Mobil geliştiricilerin bu duruma hazırlıklı olması gerekir.

1. Memory Pressure (Bellek Baskısı)

Mobil cihazlarda RAM sınırlı olduğundan işletim sistemi, uygulamanın beklenenden fazla bellek kullanması durumunda:

- **memory warning** bildirir (iOS),
- arka plandaki uygulamaları dondurur,
- gerekli görülürse uygulama sürecini öldürür.

iOS log örneği: `terminated due to memory pressure`. Android log örneği: `LowMemoryKiller`.

2. Long-Running Background Tasks

Mobil işletim sistemleri, arka planda uzun süre çalışan işlere izin vermez. Aşağıdaki işlemler uygulamanın öldürülmesine neden olabilir:

- uzun süreli `setInterval` veya döngüler,
- yoğun CPU kullanan işlemler,
- büyük JSON parse operasyonları,
- büyük dosya veya resim işleme,
- sürekli network istekleri.

iOS, arka planda 30 saniyeden fazla CPU veya ağ işlemi yapan uygulamaları sonlandırır.

3. Idle Timer

Kullanıcı uzun süre etkileşimde bulunmazsa uygulama:

- önce **suspended** durumuna alınır,
- ardından bellek ihtiyacı oluştuğunda OS tarafından öldürülür.

Bu davranış özellikle Android’de çok daha agresiftir.

4. CPU Throttling

Cihaz aşırı ısındığında sistem:

- CPU frekansını düşürür (throttling),
- aşırı yük altındaki uygulamaları kapatabilir.

Özellikle kriptografi, görüntü işleme ve sürekli hesaplama yapan uygulamalar bu durumu tetikler.

5. Doze Mode (Android)

Android, cihaz hareketsiz kaldığında enerji tasarrufu için **Doze Mode**'a geçer:

- ağ istekleri kısıtlanır,
- arka plan görevleri durdurulur,
- timer'lar dondurulur,
- push bildirimleri gecikebilir.

Bu süreçte uygulama askıya alınabilir veya öldürülebilir.

6. App Nap (iOS)

iOS enerji tasarrufu için arka plandaki uygulamaları:

- tamamen durdurabilir,
- CPU kullanımını sıfıra indirebilir,
- background task'leri iptal edebilir.

7. Sonuç: Mobil Ortamda Uygulama Her An Öldürülebilir

Mobilde uygulamanın sonlandırılması normal bir davranıştır. Bu nedenle uygulamanın yalnızca RAM üzerinde çalışması güvenilir değildir.

8. Gerekli Mimari Önlemler

8.1 State Persistence (Durum Kalıcılığı)

Kritik veriler RAM yerine kalıcı depolamada tutulmalıdır:

- MMKV,
- AsyncStorage,
- SQLite / Realm / WatermelonDB.

8.2 Graceful Resume

Uygulama yeniden açıldığında kullanıcı:

- kaldığı ekrana,
- kaldığı veri durumuna,
- form içeriklerine,
- scroll pozisyonuna

geri dönebilmelidir.

8.3 Offline Cache

Bağlantı problemleri ve OS kesintileri için:

- cache-first stratejileri,
- stale-while-revalidate,
- local database mirror,
- request retry mekanizmaları

zorunludur.

8.4 Background Queue

Uygulama öldürülse bile görevlerin kaybolmaması için:

- upload queue,
- sync queue,
- retry queue,
- transactional offline storage

kullanılmalı ve işler disk üzerinde sıraya alınmalıdır.

Özet

Mobil işletim sistemleri uygulamaları bellek baskısı, CPU yükü, enerji tasarrufu veya arka plan kısıtlamaları nedeniyle spontan şekilde öldürebilir. Bu nedenle mobil uygulamalarda state persistence, offline cache ve background queue tasarımı zorunludur; aksi halde uygulama durumu ve iş akışları kolayca kaybolur.

Doze Mode (Android)

Android, cihaz uzun süre hareketsiz kaldığında (ekran kapalı, cihaz sabit, erişim yokken) otomatik olarak **Doze Mode** adı verilen enerji tasarrufu moduna geçer. Bu modda işletim sistemi, batarya tüketimini azaltmak amacıyla uygulamaların arka planda çalışmasını ciddi biçimde kısıtlar.

1. Doze Mode'un Temel Davranışları

Doze Mode'da aşağıdaki sistem bileşenleri kısıtlanır veya tamamen durdurulur:

- **Network erişimi engellenir**
HTTP istekleri, WebSocket bağlantıları, TCP bağlantıları ve HTTPS handshake işlemleri durdurulur. Uygulama bu süreçte ağ üzerinden herhangi bir veri gönderemez veya alamaz.
- **Background task'ler durdurulur**
JavaScript event loop, headless JS, servisler, React Native timer'ları, scheduling sistemleri ve sürekli çalışan döngüler işletim sistemi tarafından durdurulur.
- **Timer'lar dondurulur**
`setTimeout`, `setInterval` ve native zamanlayıcılar durdurulur. Zaman tabanlı fonksiyonlar çalışmaz.
- **Push notification'lar gecikebilir**
Google FCM bağlantısı düşük güç moduna alınır. Cihaz yeniden aktif olana kadar bildirimler ertelenebilir.

2. Suspended → Kill Mekanizması

Doze Mode'da uygulama çoğu zaman **suspended** (dondurulmuş) durumuna alınır. Bu durumda:

- uygulama CPU kullanamaz,
- event queue işlenmez,
- JavaScript thread durur,
- background servisleri çalışmaz.

Bellek baskısı olursa işletim sistemi uygulama sürecini haber vermeden öldürebilir. Bu davranış:

suspended → kill

şeklinde gerçekleşir ve uygulama yaşam döngüsü callback'leri (`onPause`, `onStop`, `onDestroy`) tetiklenmez.

3. Uygulamanın Yanlış Algısı

Doze Mode sırasında uygulama, geliştirici tarafından çalışıyor sanılabilir; fakat gerçekte:

- timer’lar durmuştur,
- network yoktur,
- background queue işlemiyordur,
- upload veya sync görevleri tamamlanmaz,
- uygulama durdurulmuş olabilir.

Daha sonra sistem kaynak ihtiyacı oluşursa süreç tamamen sonlandırılır.

4. Mimari Gereklilikler

Doze Mode nedeniyle mobil uygulamalar **her an öldürülebilir** varsayımıyla tasarlanmalıdır.

4.1 State Persistence Kritik durum bilgileri RAM yerine kalıcı depolamaya (MMKV, AsyncStorage, SQLite, Realm vb.) yazılmalıdır.

4.2 Graceful Resume Uygulama yeniden açıldığında kullanıcının kaldığı yerden devam edebilmesi için:

- navigation state,
- form içerikleri,
- offline veri,
- scroll pozisyonları

yeniden yüklenmelidir.

4.3 Offline Cache Ağ kısıtlamaları ve bağlantı kayıpları nedeniyle:

- cache-first,
- stale-while-revalidate,
- local database mirror,
- retry mekanizmaları

zorunludur.

4.4 Background Queue Upload veya senkronizasyon işlemleri için disk tabanlı bir kuyruk sistemi kullanılmalıdır. Uygulama öldürüldüğünde görevler kaybolmaz, yeniden açıldığında kaldığı yerden devam eder.

Özet

Doze Mode, Android'in batarya tasarrufu için uygulamaları dondurduğu ve gerekirse sessizce sonlandırdığı bir moddur. Bu nedenle mobil uygulamalar, RAM'e, timer'lara veya arka plan işleyişine güvenemez. State persistence, offline-first stratejileri ve disk tabanlı görev kuyukları mobil mimaride zorunludur.

Doze Mode’a Uygun Algoritmalar

Doze Mode, mobil cihazlarda enerji tasarrufu sağlamak amacıyla arka plan çalışmasını ciddi şekilde kısıtlar. Bu nedenle mobil uygulamalarda timer tabanlı döngüler, uzun süreli işlemler ve düşük öncelikli bildirimler Doze Mode sırasında çalışmaz veya işletim sistemi tarafından ertelenir. Bu durumlara uyum sağlamak için özel algoritma tasarımları kullanılmalıdır.

1. Timer Değil → Planlı Task (OS-Managed Scheduling)

Doze Mode’da:

- `setTimeout` ve `setInterval` çalışmaz,
- JavaScript event loop durdurulur,
- native zamanlayıcılar dondurulur.

Bu nedenle timer tabanlı yaklaşımlar güvenilir değildir.

Çözüm: OS Tarafından Yönetilen Görevler

Android: **WorkManager** veya **JobScheduler** iOS: **BGTaskScheduler**

Bu sistemler, işletim sisteminin uygun gördüğü enerji dostu zamanlarda uygulama görevlerini çalıştırır.

“Timer yerine planlı görev (scheduled task) kullanılmalıdır.”

2. Push Değil → High Priority Push

Doze Mode sırasında normal öncelikli bildirimler gecikebilir. Bunun nedeni:

- FCM bağlantısının düşük güç moduna alınması,
- heartbeat paketlerinin gönderilememesi,
- non-urgent bildirimlerin ertelenmesidir.

Bu durumda standart bildirimler zamanında ulaşmaz.

Çözüm: High Priority Push Notifications **High priority** FCM bildirimleri Doze Mode’u geçici olarak uyandırabilir ve bildirimin anında teslim edilmesini sağlar.

“Normal push bildirimleri yerine yüksek öncelikli push kullanılmalıdır.”

3. Long Tasks Değil → Batching + Snapshot

Doze Mode uzun süreli arka plan işlemlerini durdurur:

- büyük upload işlemleri,
- büyük JSON parse operasyonları,
- CPU yoğun hesaplamalar,
- uzun süreli senkronizasyon işlemleri.

Bu tür işlemler enerji politikaları nedeniyle kesintiye uğrar veya uygulamanın öldürülmesine yol açar.

Çözüm: Kısa Süreli Parçalı İşlem Tasarımı Uzun işlemler daha küçük parçalara bölünmelidir:

- büyük upload → küçük dosya segmentleri,
- büyük sync → küçük snapshot grupları,
- uzun hesaplama → parçalı batch işlemleri.

Bu yaklaşım Doze Mode'un CPU ve ağ kısıtlamalarını aşmayı sağlar.

“Uzun işlemler yerine küçük parçalara ayrılmış batch + snapshot yaklaşımı kullanılmalıdır.”

Özet

Doze Mode sırasında timer'lar çalışmaz, normal push bildirimleri gecikir ve uzun görevler durdurulur. Bu nedenle mobil uygulama algoritmaları planlı görevler, yüksek öncelikli bildirimler ve parçalı işlem modelleri ile tasarlanmalıdır.

Zamanlanmış Bildirimlerin Doze Mode Nedeniyle Gecikmesi

Android işletim sistemi, enerji tasarrufu amacıyla Doze Mode ve benzeri kısıtlamaları devreye alır. Bu modlarda zamanlanmış bildirimler (örneğin Notiffee ile planlanan local notifications), belirtilen zamanda tetiklenmeyebilir ve saatlerce gecikebilir. Bu hata uygulamanın veya Notiffee'nin davranışı değil; Android'in sistem politikalarıdır.

1. Sorunun Kaynağı: Doze Mode ve Alarm Gecikmesi

Android 6.0 (API 23) itibarıyla, cihaz hareketsiz kaldığında:

- CPU uyku moduna alınır,
- ağ bağlantıları durdurulur,
- zamanlayıcılar (timers) dondurulur,
- **AlarmManager** çağrıları ertelenir,
- uygulamalar arka planda askıya alınır,
- uygun bir “maintenance window” gelene kadar alarm tetiklenmez.

Bu nedenle:

01:00 için planlanan bir bildirim, saat 23:00'e veya daha ileri bir zamana ertelenebilir.

2. Android'de “Exact Time” Garanti Değildir

Aşağıdaki senaryolarda zamanlanmış bildirimler kesin olarak gecikir:

- Doze Mode aktifken,
- Battery Optimization açıkken,
- OEM agresif güç yönetimi (Xiaomi, Huawei, Oppo),
- App Standby Buckets sınırlı moddayken,
- AlarmManager “inexact” moda zorlandığında.

Sonuç: Notiffee ile planlanan local notifications, varsayılan Android ayarlarında tam zamanda tetiklenmez.

3. Çözüm Adımları

Kesin zamanlı bildirim elde edebilmek için aşağıdaki adımlar birlikte uygulanmalıdır.

3.1 setExactAndAllowWhileIdle() Kullanımı

Android'in en doğru (fakat yine de %100 garanti olmayan) yöntemi:

```
alarmManager.setExactAndAllowWhileIdle(  
    AlarmManager.RTC_WAKEUP,  
    triggerAtMillis,  
    pendingIntent  
);
```

Bu yöntem Doze Mode kısıtlamalarını minimum seviyeye indirir.

3.2 Battery Optimization Kapatılması

Battery Optimization açık olduğunda alarm gecikmesi çok daha agresiftir.

Kullanıcı ayarlarının açılması:

```
AndroidSettings.openBatteryOptimizationSettings();
```

Uygulamanın “Don't optimize” modunda olması zorunludur.

3.3 Android 12+ İçin Exact Alarm İzni

Android 12 (API 31) ve sonrası:

Tam zamanlı alarm çalıştırmak için özel izin gereklidir.

Manifest:

```
<uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />
```

Bu izin olmadan zamanlanmış bildirimler keyfi olarak ertelenir.

3.4 High Priority FCM Kullanımı (Uzaktan Bildirimler)

Yerel bildirimler yerine buluttan gelen bildirimlerde:

- normal öncelikli push → gecikir,
- **high priority push** → cihazı geçici olarak uyandırır.

```
{  
  "priority": "high",  
  "data": { "type": "ALARM" }  
}
```

Bu, Doze Mode kısıtlamalarını aşabilen tek garantili bildirim türüdür.

3.5 OEM Güç Optimizasyonlarının Kapatılması

Bazı üreticiler Android'den daha agresif optimizasyon uygular:

- Xiaomi,
- Huawei,
- Oppo,
- Vivo,
- Samsung (bazı modeller).

Bu cihazlarda:

- auto-start izinleri,
- background activity izinleri,
- unlimited battery mode,
- “lock app in recents” (Xiaomi)

açılmadan zamanlı bildirimler güvenilir çalışmaz.

4. Özet

Android Doze Mode zamanlanmış bildirimleri geciktirir. Bu gecikme sistemin tasarımından kaynaklanır ve Notifee'nin hatası değildir. Tam zamanlı bildirim elde etmek için: Battery Optimization'ın kapatılması, exact alarm izninin verilmesi, setExactAndAllowWhileIdle() kullanımı ve mümkünse high priority FCM bildirimleri zorunludur.

GPU vs CPU Rendering Pipeline

Mobil kullanıcı arayüzlerinde akıcı animasyonlar ve yüksek performanslı rendering için CPU ve GPU arasındaki iş bölümü kritik öneme sahiptir. UI thread (main thread) tıkanıldığında *jank*, *frame drop* ve kullanıcı deneyiminde gecikmeler oluşur. Bu nedenle animasyonlar mümkün olduğunca GPU'ya taşınmalı ve CPU yükü azaltılmalıdır.

1. Rendering Pipeline

Mobil rendering pipeline genel olarak üç temel aşamadan oluşur:

1. **Layout ve Measure** (CPU) View hiyerarşisinin hesaplanması, ölçüler, konumlar ve tarz özellikleri CPU tarafından işlenir.
2. **Painting** (CPU → GPU Hazırlığı) View'ların çizim komutları oluşturulur. Shadow, border, efekt gibi özellikler maliyet yaratabilir.
3. **Compositing** (GPU) GPU, çizim katmanlarını (layers) birleştirerek ekran çıktısını oluşturur. Bu aşama donanım hızlandırmalıdır.

2. Jank Neden Oluşur?

UI thread üzerindeki fazlalık yük aşağıdaki durumlara yol açar:

- frame sürelerinin 16.6ms'yi aşması (60 FPS için sınır),
- ağır layout hesaplamaları,
- senkron işlemler,
- CPU üzerinde yapılan animasyon hesaplamaları.

Sonuçta kullanıcı tarafında **jank** (takılma) ve **frame drop** görülür.

3. GPU'ya Taşınması Gereken İşler

Modern mobil arayüzlerde animasyon hesaplamalarının CPU'da yapılmaması gerekir. Aşağıdaki işlemler GPU üzerinde daha verimlidir:

- **off-main-thread animations** Ana thread'i bloklamadan animasyonların ayrı bir pipeline'da çalıştırılması.
- **hardware compositing** GPU katman birleşimi sayesinde daha akıcı geçişler.
- **opacity, transform, scale, translate, rotate** Bu özellikler GPU tarafından doğal olarak hızlandırılır.

4. Maliyetli Görsel Efektler

Aşağıdaki özellikler CPU ve GPU üzerinde yüksek maliyet oluşturur:

- **shadow rendering** Soft shadow (blur radius yüksek) GPU üzerinde pahalıdır.
- **blur efektleri** Gaussian blur gibi işlemler çoğunlukla CPU veya shader düzeyinde ağır hesaplama gerektirir.
- **overdraw** Ekranın aynı pikselinin birden fazla kez çizilmesi; gereksiz view katmanları, opacity overlay'leri, nested view yapıları performans kaybına neden olur.

5. React Native Özelinde Rendering

React Native'de rendering iki ana thread'e bölünmüştür:

- **UI Thread** Layout, ölçüm ve native UI rendering işleri.
- **JS Thread** Animasyonların veya state değişimlerinin hesaplanması.

JS thread bloklanırsa animasyonlar gecikir ve jank oluşur.

Bu nedenle React Native ekosisteminde GPU hızlandırmalı animasyonlar için özel kütüphaneler kullanılır.

6. Off-Main-Thread Animations İçin Kütüphaneler

6.1 Reanimated 2 Reanimated 2, *worklet* yapısı sayesinde animasyon hesaplamalarını JS thread yerine **UI thread** üzerinde çalıştırır ve gecikmeleri büyük ölçüde azaltır.

Avantajları:

- JS thread tıkanıklığından etkilenmez,
- 60 FPS animasyon garantiye daha yakın,
- native hızında gesture + animation kombinasyonu.

6.2 Skia (react-native-skia) Skia bir **GPU tabanlı 2D rendering engine**'dir. React Native içinde:

- custom drawing,
- shader efektleri,
- GPU hızlandırmalı canvas çizimi,

için kullanılır. Yüksek performanslı animasyonlar ve görsel efektlerde *CPU* → *GPU offload* sağlar.

7. Özet

Mobilde akıcı animasyonlar için CPU yerine GPU üzerinde çalışan compositing pipeline tercih edilmelidir. Shadow, blur ve overdraw gibi maliyetli işlemler dikkatle yönetilmelidir. React Native uygulamalarında Reanimated 2 ve Skia, animasyonların off-main-thread çalışmasını sağlayarak jank riskini büyük ölçüde azaltır.

iOS Sandbox vs Android Sandbox

Mobil işletim sistemlerinde **sandbox**, her uygulamanın diğer uygulamalardan, sistem bileşenlerinden ve dosya yapılarından izole şekilde çalışmasını sağlayan güvenlik modelidir. Amaç, veri gizliliğini korumak, uygulamalar arası etkileşimi sınırlamak ve sistem kararlılığını artırmaktır.

iOS ve Android sandbox modelleri benzer bir izolasyon mantığına sahip olmakla birlikte, uygulamaların özgürlük seviyeleri ve güvenlik yaklaşımları bakımından önemli farklılıklar içerir.

1. Sandbox Nedir?

Sandbox, bir uygulamanın yalnızca kendisine ait izole bir dosya sistemi, izin kümesi ve işlem bağlamı (process context) içerisinde çalıştırılmasıdır.

Bu model sayesinde:

- bir uygulama diğerinin dosyalarına erişemez,
- uygulamalar birbirinin RAM alanını okuyamaz,
- süreçler (processes) birbirini etkileyemez,
- veri sızıntısı veya kötü niyetli erişim engellenir.

Her uygulama “kendi küçük sanal bilgisayarında” çalışır.

2. iOS Sandbox Modeli

iOS, son derece kapalı ve katı bir sandbox mimarisi kullanır. Uygulamalar sıkı güvenlik ve gizlilik kuralları altında çalışır.

Başlıca Özellikler

- **Uygulama yalnızca kendi dizinine erişebilir**
/var/mobile/Containers/Data/Application/<UUID>/
- **Global file system erişimi yoktur**
Android'deki gibi ortak bir /sdcard alanı iOS'ta bulunmaz.
- **Cross-app communication son derece sınırlıdır**
iOS Intent benzeri mekanizmalara izin vermez. Uygulamalar neredeyse tamamen birbirinden izole çalışır.
- **Private API'lere erişim yasaktır**
Kodu imzalama (code signing) ve entitlements yapısı, uygulamanın Apple'ın izin vermediği hiçbir sistem kaynağına erişemeyeceği şekilde tasarlanmıştır.
- **Her extension bile kendi sandbox'ında çalışır**
Keyboard extension, Today widget, WatchOS companion app vb.

Sonuç

iOS sandbox modeli yüksek güvenlik sağlar ancak geliştirici özgürlüğünü sınırlar.

3. Android Sandbox Modeli

Android sandbox modeli Linux çekirdeğine dayanır. Her uygulama ayrı bir Linux kullanıcısı (UID) olarak çalışır.

Başlıca Özellikler

- **Her uygulama ayrı bir Linux user hesabıdır**
App A → UID 10123
App B → UID 10124
Dolayısıyla App A, App B'nin dosyalarına erişemez.
- **Cross-app communication daha esnektir**
Intent, Broadcast Receiver, Content Provider ve Bound Service gibi mekanizmalar uygulamalar arası iletişimi mümkün kılar.
- **External storage sandbox değildir**
/sdcard tüm uygulamalar tarafından erişilebilir durumdadır (izin verildiği sürece).
- **FileProvider gerekebilir**
Uygulamalar dosya paylaşımı için güvenli bir aracı kullanmalıdır.
- **OEM üreticileri sandbox davranışını değiştirebilir**
Xiaomi, Huawei, Oppo ve Samsung gibi üreticiler güç yönetimi ve izin politikalarında ek kısıtlamalar veya istisnalar ekleyebilir.

Sonuç

Android sandbox modeli daha esnek ve geliştirici dostudur; ancak iOS'a kıyasla daha geniş bir saldırı yüzeyi barındırır.

4. iOS vs Android Sandbox — Karşılaştırma Tablosu

Kriter	iOS	Android
Temel Model	App-specific container	Linux UID tabanlı izolasyon
Dosya Erişimi	Çok katı	Daha esnek
Cross-App Communication	Çok sınırlı	Çok açık (Intent vb.)
External Storage	Yok	Var (paylaşımlı)
Güvenlik Seviyesi	Çok yüksek	Yüksek ama esnek
OEM Etkisi	Minimal	Yüksek
Developer Özgürlüğü	Düşük	Yüksek

Özet

Sandbox, mobil uygulamaların birbirinden izole şekilde çalışmasını sağlar. iOS, kapalı ve yüksek güvenli bir sandbox modeliyle en sıkı izolasyonu sağlarken; Android daha esnek, geliştirici dostu ancak OEM etkisine açık bir sistem sunar.

13. App Size Optimization

Mobil uygulamalarda paket boyutunun büyümesi; indirilme oranının düşmesine, ilk açılış süresinin uzamasına ve cihazda daha fazla disk kullanımı oluşmasına neden olur. Bu nedenle APK/IPA boyutunu optimize etmek kritiktir.

Aşağıdaki teknikler hem Android hem iOS için yaygın olarak kullanılır.

1. Proguard / R8 (Android)

Amaç: Java/Kotlin bytecode'unu küçültmek, optimize etmek ve karıştırmaktır. R8, Proguard'ın modern sürümü olup aşağıdaki işlemleri yapar:

- **Shrinking** — Kullanılmayan sınıf ve metotların silinmesi
- **Minification** — Sembollerin küçültülmesi (ClassA → a)
- **Obfuscation** — Kodun okunamaz hale getirilmesi
- **Optimization** — Bytecode seviyesinde optimizasyon

Bu optimizasyonlar uygulama boyutunda genellikle %10–25 küçülme sağlar.

2. Hermes (React Native JavaScript Engine)

Hermes, mobil cihazlar için optimize edilmiş bir JavaScript motorudur. Varsayılan JSC motoruna göre daha küçüktür ve daha hızlı çalışır.

- JS bundle boyutunu azaltır
- Soğuk başlangıç süresini hızlandırır
- RAM kullanımını düşürür

Aktifleştirildiğinde ortalama 3–8 MB kazanç sağlar.

3. ICU Data Trimming

ICU, dünya dilleri, takvimleri ve sayı formatlarını içeren büyük bir kütüphanedir. Uygulamada yalnızca birkaç dil kullanılacaksa, diğer diller çıkartılarak boyut küçültülebilir.

- TR/EN gibi sınırlı dil desteği ile 2–7 MB tasarruf elde edilebilir.

4. Image Compression

Görseller çoğu mobil uygulamada en büyük boyutu kaplayan öğelerdir.

- PNG → **WebP** formatı
- JPEG optimizasyonu (mozjpeg, guetzli)
- SVG ile ikon kullanımı
- Splash / onboarding görsellerinin küçültülmesi

Gerçek dünyada en büyük küçülme genellikle burada sağlanır.

5. WebP Formatı

WebP, hem PNG hem de JPEG formatlarına göre daha küçük dosya boyutu üretir.

- PNG'ye göre %30–60 daha küçük
- JPEG'e göre %25–35 daha küçük
- Şeffaflık ve animasyon desteği

6. Native Libs Splitting

React Native ve diğer Native uygulamalarda .so uzantılı kütüphaneler farklı CPU mimarileri için ayrı ayrı bulunur:

- arm64-v8a
- armeabi-v7a
- x86
- x86_64

Hepsinin tek APK içinde bulunması boyutu gereksiz büyütür.

7. ABI Splits (Android)

ABI splits, her CPU mimarisi için ayrı APK üretmeyi sağlar. Böylece cihaz kendi mimarisine uygun ve minimum boyuttaki APK'yı indirir.

- Tek APK: 50–70 MB
- ABI split sonrası: Her biri 15–25 MB

Bu, Android tarafında en büyük boyut küçültmesini sağlayan tekniktir.

Genel Etki Sıralaması

Boyutu en çok küçülten optimizasyondan, en az küçültene doğru sıralama:

1. ABI splits
2. WebP + image compression
3. Hermes
4. Proguard / R8
5. ICU trimming
6. Native module cleanup
7. JS bundle minification

Pratik Öneriler

- Proguard / R8 aktif olmalı
- Hermes JS engine kullanılmalı
- Görseller WebP formatında olmalı
- Splash / onboarding görselleri küçültülmeli
- SVG ikon kullanılmalı
- Android için ABI splits aktif edilmeli
- Büyük JavaScript kütüphaneleri tree-shake edilmeli

14. Mobile-Specific Concurrency (Basit Anlatım)

Mobil cihazlarda CPU ve RAM sınırlı olduğu için, aynı anda çok fazla iş yapmak uygulamanın donmasına, animasyonların takılmasına ve veri tutarsızlıklarına neden olur. Bu nedenle concurrency, mobil geliştirmede kritik bir konudur.

Aşağıda mobilde bilinmesi gereken temel kavramlar sade bir şekilde açıklanmıştır.

1. Job Schedulers (Arka Planda Çalıştırıcılar)

Android (WorkManager): Uygulama kapalı olsa bile belirli işleri çalıştırmak için kullanılır.

iOS (BackgroundTasks API): iOS uygun gördüğünde kısa süreli arka plan çalışma süresi verir (10–30 saniye). Süre bitince işlemi durdurur.

2. Rate Limiting

Uygulamanın çok hızlı bir şekilde arka arkaya istek göndermesini engeller. Örneğin, her saniyede en fazla bir API isteği yapılması.

3. Debounce

Bir işlem ancak kullanıcı belirli bir süre durduğunda çalıştırılır. Örnek: Arama kutusunda kullanıcı yazmayı bıraktıktan 300ms sonra API çağrısının yapılması.

4. Throttle

Bir işlemin belirli aralıklarla çalıştırılmasını sağlar. Örnek: Scroll sırasında her pikselde değil, her 200ms’de bir event tetiklenmesi.

5. Batching (Toplu Gönderim)

Birden fazla küçük işin tek bir büyük iş olarak gönderilmesidir. Örnek: 10 log kaydının 10 ayrı istekte değil, tek bir POST isteğinde gönderilmesi.

6. Parallel vs Serial Queues

Serial Queue: İşler sırayla çalışır. UI işlemleri ve React Native JS thread bu yapıdadır.

Parallel Queue: Birden fazla iş aynı anda çalışabilir, ancak sayısı sınırlıdır. Arka plan resim işleme, network istekleri gibi işler burada yapılır.

7. Thread Pool Limitleri

Mobil cihazlarda sistem tarafından yönetilen sınırlı sayıda worker thread vardır (genellikle 4–8). Çok fazla thread açmak performansı düşürür.

8. Promise Queue Congestion (React Native)

React Native’de JS thread tektir. Aynı anda çok fazla iş (örneğin 20 API isteği) çalıştırılınca JS thread kilitlenir ve UI donmaya başlar.

9. Concurrency Hatalarının Sonuçları

- **Race Conditions** — Aynı veri farklı işlemlerce çakışır.
- **Freezes** — Uygulama kısa süreli donar.
- **JS Thread Block** — Tüm UI işlemleri durur.
- **Image Flicker** — Görseller titrer veya anlık kaybolur.
- **Stale Cache** — Eski veriler yeni veriyi ezebilir.
- **Dropped Frames** — Animasyonlar takılır (jank).

10. Mobil Concurrency’nin Altın Kuralı

UI (ana thread) yalnızca çizim, animasyon ve kullanıcı etkileşimini yönetmelidir. Tüm ağır işler arka plan queue’larına taşınmalıdır.

5. AppState / Lifecycle-Aware Data Flows

Mobil uygulamalar web uygulamalarından farklı olarak uykuya geçebilir, OS tarafından öldürülebilir, ağ bağlantısını kaybedebilir ve geri kazanabilir. Bu nedenle veri akışlarının uygulamanın yaşam döngüsüne duyarlı olması gerekir.

Aşağıda her bir davranışın detaylı açıklaması ve örnekleri verilmiştir.

1. App Sleep → Rehydrate State

Uygulama arka plana alındığında:

- JS thread durur,
- network işlemleri kesilir,
- timer'lar çalışmayı bırakır,
- OS RAM temizliği yaparsa state tamamen kaybolabilir.

Bu nedenle uykuya geçmeden önce state'in kalıcı depolamaya yazılması gerekir.

React Native Örneği

```
javascript AppState.addEventListener('change', nextState => { if (nextState === 'background') store.set('ui_state', JSON.stringify(globalState)); });
```

Uygulama yeniden açıldığında state geri yüklenir:

```
javascript const saved = store.getString('ui_state'); if (saved) setGlobalState(JSON.parse(saved));
```

Bu yaklaşım, kullanıcıların uygulamaya kaldıkları yerden devam edebilmesini sağlar.

2. App Resume → Refresh Data

Uygulama uzun süre arka planda kalmış olabilir ve veri artık güncel olmayabilir.

Resume olduğunda:

- token süresi dolmuş olabilir,
- liste verileri eski olabilir,
- unread count değişmiş olabilir,
- background task gerçekleşmemiş olabilir.

Bu nedenle aktif duruma geçince veri yenilemesi yapılmalıdır.

React Native Örneği

```
javascript AppState.addEventListener('change', state => { if (state === 'active') queryClient.invalidateQueries(); refetchUser(); refetchNotifications(); });
```

Bu yaklaşım WhatsApp, Instagram gibi uygulamalardaki “geri dönünce otomatik yenileme” davranışının aynısıdır.

3. Background Tasks → Queue Flush

Kullanıcı bir işlem başlatıp uygulamadan çıkabilir. Örnek:

- upload başlatıldı ancak kullanıcı başka uygulamaya geçti,
- offline event log birikti,
- video işleme devam ederken ekran kapandı.

Bu durumlarda kuyruk arka planda kısa süreliğine flush edilmelidir.

React Native Örneği

```
javascript let uploadQueue = [];  
function flushQueue() while (uploadQueue.length > 0) const job = upload-  
Queue.shift(); upload(job);  
AppState.addEventListener('change', state => if (state === 'background')  
flushQueue(); );  
Android tarafında WorkManager, iOS tarafında BackgroundTasks API bu  
tür senaryolar için kullanılır.
```

4. JS Thread Freeze → Recover

React Native'de JS thread tektir. Aşağıdaki işlemler thread'in donmasına neden olabilir:

- büyük JSON parse işlemleri,
- uzun döngüler,
- çok sayıda API isteğinin aynı anda yapılması,
- büyük görsellerin işlenmesi,
- yoğun re-render.

JS thread donduğunda UI durur ve veri “stale” hale gelebilir. Bu nedenle freeze sonrası kritik verilerin tekrar senkronize edilmesi gerekir.

Basit Freeze Detector

```
javascript let lastTick = Date.now();  
setInterval(() => const now = Date.now(); if (now - lastTick > 1000) // JS  
thread donmuş olabilir refetchCriticalData(); lastTick = now; , 500);
```

5. Network Change → Re-sync

Mobil cihazlarda ağ bağlantısı sürekli değişebilir:

- Wi-Fi → LTE,
- LTE → Edge,
- offline → online,
- captive portal senaryoları,
- yoğun bölgelerde paket kayıpları.

Bağlantı geri geldiğinde offline queue'nun gönderilmesi ve veri eşitlemesi yapılmalıdır.

React Native Örneği

```
javascript NetInfo.addEventListener(state => { if (state.isConnected) syncPendingMessages(); refetch(); });
```

Bu davranış Instagram, WhatsApp gibi uygulamalardaki “çevrimdışı yazılan mesajların geri gelince otomatik gönderilmesi” mekanizmasının aynısıdır.

Özet

- Uygulama uykuya geçerken state kaydedilmelidir.
- Resume olduğunda veriler yenilenmelidir.
- Arka plan geçişlerinde kuyruk işlemleri tamamlanmalıdır.
- JS thread donarsa kritik veriler tekrar senkronize edilmelidir.
- Ağ değiştiğinde offline queue ve veri yeniden eşitlenmelidir.

Mobilde yaşam döngüsü çok daha karmaşıktır çünkü:

- OS uygulamayı öldürebilir,
- JS thread donabilir,
- network değişebilir,
- bellek temizliğinde state kaybolabilir,
- uygulama saatlerce arka planda kalabilir.

Bu nedenle veri akışlarının yaşam döngüsüne duyarlı tasarlanması şarttır.

3D Secure Akışı, Uygulama Değişimi ve State Kaybı

Mobil uygulamalarda 3D Secure doğrulaması sırasında kullanıcı bankanın uygulamasına veya bankanın web arayüzüne yönlendirilir. Bu işlem sırasında uygulama arka plana alınır ve OS tarafından durdurulabilir veya tamamen sonlandırılabilir. Bu nedenle satın alma ekranının veya state'in kaybolması uygun şekilde yönetilmediğinde sık görülen bir problemdir.

Aşağıda bu davranışın neden gerçekleştiği ve doğru çözüm mimarisi detaylı biçimde açıklanmıştır.

1. Problemin Temeli: Mobil Yaşam Döngüsü

Kullanıcı bankanın uygulamasına yönlendirildiğinde:

- Uygulama arka plana alınır,
- iOS bu sırada uygulamayı **suspend** veya **terminate** edebilir,
- RAM boşaltılırsa navigation stack ve geçici state tamamen kaybolur,
- Uygulama geri gelindiğinde **fresh launch** başlatılabilir.

Bu nedenle satın alma ekranının “uçması” çoğu zaman **navigation state'in RAM'de tutulmasından** kaynaklanır. Mobil OS RAM temizliği yaptığında bu state silinir.

iOS ve Android Davranışları

iOS:

- Arka plan uygulamalarını agresif biçimde durdurur,
- Banka uygulamasına geçişte uygulamayı öldürmesi olağandır,
- Geri dönüş genellikle tamamen yeni bir uygulama başlatımıdır.

Android:

- Daha toleranslıdır ancak bellek baskısında uygulamayı öldürebilir.

Her iki platformda da RAM'de tutulan geçici state güvenilir değildir.

2. Neden State Kaybolur?

Yanlış tasarlanmış ödeme akışlarında:

- navigation stack yalnızca hafızada tutulur,
- ödeme bilgileri (product_id, amount, session) RAM'dedir,

- callback geldiğinde kullanılacak oturum bilgisi yoktur,
- kullanıcı uygulamaya döndüğünde eşleşecek bir state bulunamaz.

Sonuç:

Ödeme ekranı kaybolur.

Bu davranış normaldir; hata uygulamanın lifecycle-aware tasarlanmamasıdır.

3. Doğru Çözüm Mimarisi

A. Payment Session Kalıcı Depolamaya Yazılmalıdır

Geçici RAM yerine aşağıdaki depolama katmanlarından biri kullanılmalıdır:

- MMKV (hızlı key-value storage),
- Secure Storage,
- Backend-side session store.

Ödeme başlangıcında:

json "payment_session_id" : "abcd1234", "amount" : 150, "product_id" : 99, "status" : "pending"

Bu veri diske yazılmalıdır.

B. 3D Secure Dönüşünde Deep Link / Callback Yönetimi

Banka doğrulamasından sonra uygulama aşağıdaki gibi çağrılır:

text myapp://payment/callback?session_id=abcd1234&status=success

Uygulama tamamen yeni açılmış olsa bile:

- session_id yakalanır,
- MMKV veya backend'den ödeme state'i geri yüklenir,
- ödeme ekranı yeniden oluşturulur.

C. Navigation Stack'e Güvenilmemelidir

Navigation yapısı RAM tabanlı olduğu için güvenilir değildir. Bu nedenle ödeme ekranı:

- idempotent olmalı (yeniden oluşturulabilir),
- gerekli tüm state'i storage veya backend'den çekmeli,
- RAM kaybına dayanıklı olmalıdır.

D. Server-Side Session Önerilir

Büyük ölçekli uygulamalarda ödeme durumu tamamen backend tarafında tutulur:

```
text GET /payment-session/abcd1234 → "status": "success", "product":
```

99 Uygulama öldürülse bile geri döndüğünde backend durumu belirler.

4. Örnek Senaryolar

Yanlış Yaklaşım

- Ödeme state'i RAM'de tutulur,
- Kullanıcı 3D Secure için bankaya geçer,
- iOS uygulamayı öldürür,
- Navigation stack sıfırlanır,
- Kullanıcı uygulamaya döndüğünde ödeme ekranı yok olur.

Doğru Yaklaşım

1. Ödeme başlarken session kalıcı olarak kaydedilir.
2. Kullanıcı bankaya yönlendirilir.
3. Uygulama öldürülse bile önemli değildir.
4. Bankadan dönüş deep link ile alınır.
5. Session ID üzerinden backend'den ödeme durumu çekilir.
6. UI ödeme ekranını yeniden oluşturur.

5. Sonuç

3D Secure akışı sırasında uygulama değişimi, uygulamanın RAM'de saklanan state'ine güvenilmesini imkânsız kılar. Bu nedenle:

- Ödeme state'i kalıcı depolama veya backend üzerinde tutulmalı,
- Deep link callback mutlaka işlenmeli,
- Navigation stack'e asla güvenilmemeli,
- UI tamamen yeniden kurulabilir şekilde tasarlanmalıdır.

Bu yaklaşım, modern e-ticaret uygulamalarında (Trendyol, Getir, Hepsiburada, Amazon) kullanılan standart çözümdür.

Mobilde Thread, Worker, Queue Kavramları ve React Native’de Ağır İş Yönetimi

Mobil cihazlarda CPU, RAM ve enerji sınırlıdır. Bu nedenle concurrency mimarisi, web ve backend sistemlerinden çok daha farklı çalışır. Bu doküman thread, worker, queue, JS thread, native worker thread ve ağır iş offloading stratejilerini React Native bağlamında detaylı biçimde açıklar.

1. Thread Nedir?

- **Thread**, CPU’nun çalıştırdığı bağımsız bir yürütme birimidir.
- Mobil cihazlarda genellikle 4–8 çekirdek bulunur; aynı anda sınırlı sayıda thread çalışabilir.
- Her thread farklı işler yapabilir: UI çizimi, network isteği, resim decode, ses oynatma vb.

2. Worker Nedir?

- **Worker**, belirli bir ağır iş için ayrılmış özel bir thread türüdür.
- Örnek: resim sıkıştırma worker’ı, video encode worker’ı, JS motor worker’ı.
- Her worker bir thread’dir; fakat her thread bir worker değildir.

3. Queue Nedir?

- **Queue**, işler için bekleme sırasıdır.
- Thread, queue içindeki işleri sırayla alır ve yürütür.

Örnek queue:

```
[ is1, is2, is3, is4 ]
```

Queue = işleri tutar, thread = işleri çalıştırır.

4. Serial Queue

- İşler **tek sıra halinde** çalışır.
- Aynı anda iki iş asla çalışmaz.
- React Native JS Thread ve UI thread bu yapıdadır.

Sıra:

```
is1 → is2 → is3 → is4
```

5. Parallel Queue

- Birden fazla thread kullanır.
- İşler paralel çalışabilir ancak sayısı sınırlıdır.
- Örnek: resim decode, video işleme, native I/O.

6. React Native’de Thread Yapısı

RN geliştiricisi manuel thread oluşturmaz. RN kendi thread mimarisini sağlar:

- **JS Thread (Tek)** — tüm JavaScript kodu bu serial queue’da çalışır.
- **Shadow/Layout Thread** — Flexbox layout hesapları.
- **Native Modules Thread Pool** — native modüllerin ağır işleri.
- **UI/Main Thread** — çizim, animasyon, touch.
- **Render Thread (GPU)** — animasyon motorları.

JS Thread tektir; bloklanırsa UI donar.

7. Mobilde Rastgele Thread Açmak Neden Tehlikelidir?

- Mobil cihazda çok fazla thread oluşturmak:
 - yüksek CPU kullanımı,
 - pil tüketimi,
 - frame drop,
 - animasyon takılması,
 - OS tarafından throttling,
 - uygulamanın öldürülmesi

Bu nedenle thread sayısı sınırlı ve kontrollü olmalıdır.

8. JS Thread Neden Serial Queue Gibi Çalışır?

JS motoru tek threadlidir. Sıra:

`is1 → is2 → is3 → ...`

- Bir iş bitmeden diğerine geçilemez.
- Ağır bir iş eklenirse tüm queue bloke olur.
- UI olayları (press, scroll) sıraya girer ve geç çalışır.

Örnek:

```
javascript console.log("is 1"); doHeavyLoop(); // 300ms blok console.log("is 2");
```

Bu sürede UI donar.

9. React Native’de Worker Mantığı

RN’de “worker”, native modüllerin arka planda kullandığı thread havuzudur.

- Ağır işler JS thread yerine native worker thread’lerde yürütülür.
- JS sadece komut gönderir; iş native tarafında yapılır.

Örnek:

```
javascript const result = await NativeImageCompressor.compress(file);
```

Bu işlem JS thread’i değil, native worker’ı meşgul eder.

10. JS Thread’e Ağır İş Yaptırmanın Sonuçları (Yanlış Yöntem)

- donan ekranlar,
- input gecikmesi,
- scroll takılması,
- dropped frames,
- kullanıcı deneyiminde ciddi bozulma.

Örnek kötü kullanım:

```
javascript for (let i = 0; i < 2000000000; i++)
```

11. JS Thread’e Ağır İş Yaptırmak İçin Doğru Yaklaşımlar

1. Ağır işi native module’a offload etmek

- image/video processing
- crypto
- pdf işlemleri

2. JSI / Turbo Modules — C++ tabanlı yüksek performans

3. Dedicated JS Workers

- react-native-multithreading
- react-native-threads

4. WebView içinde Worker (fallback)

12. Doğru Yöntem Karşılaştırması

Senaryo	Yöntem
Image/video processing	Native Module
ML Inference	JSI / C++
Büyük JSON Parse	JS Worker
Offline Search	JS Worker
Crypto	JSI
I/O işlemleri	Native File Threads

13. Özet

- Mobil concurrency “çok thread” değil, **doğru queue yönetimidir**.
- JS Thread tektir ve serial queue gibi çalışır.
- Ağır işler her zaman native worker thread'lere aktarılmalıdır.
- Rastgele thread oluşturmak mobilde verimsiz ve tehlikelidir.
- RN'de geliştirici doğrudan thread açmaz; RN mimarisi işleri uygun thread'e yönlendirir.

Mobilde Thread Kullanımı ve Ağır İşler İçin Doğru Mimari

1. “Thread Açmaktan Kaçınmak Lazım”

Mobilde kendi başına yeni thread oluşturmak risklidir:

- CPU küçük ve sınırlıdır.
- OS agresif biçimde arka plan işlemlerini öldürür.
- Fazla thread → yüksek context-switch → cihaz kasmaya başlar.
- Enerji tüketimi artar.
- Android/iOS zaten hazır thread havuzları sunar.
- 10 tane thread açmak performans getirmez; tam tersi UI'ı bozabilir.
- Animasyonlar takılır, UI donar, frame düşer.

2. “Thread Kullanmak Gerekmez”

Mobil uygulamalar zaten çok sayıda hazır thread ile çalışır:

- UI Thread
- JS Thread
- Layout Thread
- Render Thread
- Native Worker Thread Pool

Burada amaç:

Yeni thread yaratmak değil, mevcut thread havuzlarını doğru kullanmaktır.

3. Doğru Yaklaşım: Ağır İşleri Native Worker Pool’a Offload Etmek

Yanlış

“Bir tane yeni thread açayım, büyük JSON parse yapayım.”

Bu, JS thread’in bloklanmasına ve UI donmasına sebep olur.

Doğru

Ağır işi native worker pool'a offload etmek.

React Native örneği: `javascript await NativeModule.doHeavyTask();`
Bu kodda:

- JS thread ağır işi yapmaz.
- Native taraf kendi worker thread havuzundan bir thread seçer.
- İş bitince sadece sonuç JS'e döner.

4. Özet

- **Kendin thread oluşturma → HAYIR.**
- **Mevcut worker pool'u kullan → EVET.**
- **Ağır işi JS thread'e verme → HAYIR.**
- **Ağır işi native worker veya JS Worker'a at → EVET.**

RN'de zaten:

- JS Thread → tek
- UI Thread → tek
- Worker Pool → RN tarafından yönetilir

Sen sadece işi doğru kişiye verirsin.

5. Hangi İşler Hangi Thread'de Olmalı?

- UI işi → UI Thread
- Normal JS işi → JS Thread
- Ağır CPU işi → native worker / JSI / JS Worker
- Ağır I/O → native FS worker
- Image/Video compression → native worker
- Büyük JSON parse → JS Worker
- Network batching → background queue

6. Sıfır Riskli Kural

React Native'de kendi thread'ini yaratmaktan uzak dur. Native ve React Native zaten yeterince thread açar.

7. Backend vs Device — Ağır İş Nereye Ait?

Backend’de Yapılması Mantıklı Olan İşler

- Büyük dosya işleme (PDF → text, OCR)
- Video transcode
- Image optimization (Sharp, Cloudinary)
- Büyük JSON transform
- Arama index oluşturma (Elastic, Meilisearch)
- Kripto işlemleri
- Büyük matematiksel işlemler

Cihazda Yapılması Zorunlu Olan Ağır İşler

- Image compression (30–80MB → 2MB)
- Video compression (100–300MB videolar)
- Kamera frame processing (QR, yüz tespiti, AR)
- Offline JSON parse
- Offline search
- Sensitive data işlemleri (HealthKit, fotoğraf ML)

Özet Tablo

İş Türü	Backend’e Taşınır mı?	Neden?
Video compression	H	Çok büyük dosya, maliyetli
Image compression	H	Büyük boyut, UI beklemez
Camera ML	H	Real-time gerekir
Sensitive data	H	Mahremiyet
Offline features	H	İnternet yoksa ne olacak?
Büyük JSON parse	E	Ama device verisiyse mecburi device
PDF processing	E	Mantıklı
Audio → Text	E	Maliyet & hız durumu
Thumbnail generation	E	Backend yapabilir

8. Sonuç

- İş cihazdaki dosya üzerinde ise → **device tarafında native worker**.
- İş server verisi üzerinde ise → **backend**.
- Real-time + privacy + offline gerekiyorsa → **kesinlikle native**.

9. Worker Thread’ler Nasıl Oluşur?

Sen Oluşturmazsın → Kütüphane Oluşturur

Örneğin:

```
javascript await ImageCompressor.compress(image);  
Arka planda:
```

- JS → native fonksiyon çağırır.
- Native → kendi thread havuzundan bir thread seçer.
- Ağır iş o thread’de yapılır.
- JS sadece sonucu alır.

Android Örneği (Gerçek Kod)

```
java executorService.submit(() -> { Bitmap bitmap = BitmapFactory.decodeFile(path);  
File output = compressBitmap(bitmap); promise.resolve(output.getAbsolutePath());  
});
```

iOS Örneği

```
swift DispatchQueue.global().async { let output = doHeavyTask() resolve(output)
```

10. Son Net Cümle

React Native’de ağır iş = native worker thread yapar. Developer thread açmaz, kütüphane açar.

16) Jank, Overdraw ve Frame Time Budget

Frame Time Budget

Mobil cihazlarda akıcı bir kullanıcı deneyimi için hedeflenen hız **60 FPS**'tir. Bu nedenle her frame'in,

$16.6ms$

içerisinde tamamlanması gerekir. UI thread bu süre içinde:

- layout,
- measure,
- drawing,
- compositing

adımlarını bitiremezse **jank** oluşur.

Jank

UI thread'in 16.6 ms limitini aşması sonucunda:

- scroll takılması,
- animasyonların zıplaması,
- kullanıcı input'larının gecikmesi

gibi belirtiler ortaya çıkar.

Overdraw

Overdraw, aynı pikselin birden fazla kez çizilmesidir. Örneğin:

- arka plan,
- üzerine view,
- üzerine shadow,
- üzerine opacity layer

gibi katmanlar üst üste geldiğinde UI thread'in işi artar ve jank oluşur.

React Native'de derin view hiyerarşileri (deep nested views) doğal olarak **overdraw** ve buna bağlı **performans kaybı** üretir.

Bridge Bottleneck (React Native’ye Özel Performans Alanı)

React Native’de JS kodu ile Native kod arasında bir ”köprü” (bridge) bulunur. Bu köprü JSON tabanlı mesajlaşma ile çalıştığı için dar boğaz oluşturur.

Bridge’in Sorunları

- büyük JSON veri aktarımı *blocking* olur,
- çok sayıda event gönderimi frame drop üretir,
- JS thread uzun süre meşgul kaldığında UI donabilir.

Modern Çözümler

Bridge dar boğazını aşmak için React Native’in yeni mimarisi aşağıdaki yaklaşımları kullanır:

- **JSI (JavaScript Interface):** JS tarafının native fonksiyonlara C++ üzerinden doğrudan erişmesi; köprünün baypas edilmesi.
- **TurboModules:** Modüllerin daha hızlı (*synchronous*) çalışabilmesini sağlar.
- **Fabric Renderer:** Yeni nesil UI renderer; daha az jank ve daha verimli layout/render aşamaları.

Kısa Özet

Bridge Darboğazı — büyük JSON + çok event + yoğun JS işleri = Frame Drop
Yeni mimaride: JSI + TurboModules + Fabric – Köprüsüz Hızlı İletişim

Nested View (İç İçe View) Problemi ve Çözümleri

1) Nested View Nedir?

Aşağıdaki gibi bir yapıya **nested view** (iç içe view) denir:

```
<View>                                ← 1. katman
  <View>                              ← 2. katman
    <View>                            ← 3. katman
      <Text>                          ← 4. katman
    </Text>
  </View>
</View>
</View>
```

View'lar derinleştikçe buna **deep nested view** denir.

2) React Native'de Neden Sorun?

Her bir view, UI thread üzerinde aşağıdaki aşamalardan geçer:

- **Layout** (flexbox hesapları)
- **Measure** (genişlik/yükseklik ölçümü)
- **Draw** (çizim, background, border)
- **Composite** (ekrana yerleştirme)

Derinlik arttıkça her frame için yapılan iş de artar:

$1 \text{ katman} \Rightarrow 1 \text{ islem}$, $5 \text{ katman} \Rightarrow 5 \text{ islem}$, $20 \text{ katman} \Rightarrow 20 \text{ islem}$

UI thread bu işlemleri **16.6 ms** içinde bitiremezse **jank** oluşur.

3) Deep Nested View = Jank

```
<View>
  <View>
    <View>
      <View>
        <View>
          <Text>Merhaba</Text>
        </View>
      </View>
    </View>
  </View>
</View>
```

Bu yapı, **layout**, **measure** ve **render** yükünü katlar.

4) Overdraw ile Birleşince Daha Kötü

Aşağıdaki durumda aynı piksel birden fazla kez çizilir:

```
<View style={{backgroundColor: 'white'}}>
  <View style={{backgroundColor: 'white'}}>
    <View style={{backgroundColor: 'white'}}>
```

Bu durum:

$$Overdraw + DeepNestedLayout = Jank$$

üretir.

5) Neden React Native’de Daha Sık Görülür?

- Her tasarım <View> ile başlıyor.
- Tailwind / Styled Components ekstra wrapper ekliyor.
- Icon + Text + Container kombinasyonları katman artırıyor.
- Tasarımcılar kutu içinde kutu kullanmaya eğilimli.

Bu nedenle 10–20 katman *çok yaygındır*.

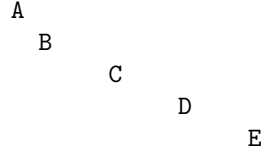
6) Örnek Kötü Render Ağacı

```
<View> (1)
  <View> (2)
    <View> (3)
      <View> (4)
        <LinearGradient> (5)
          <View> (6)
            <Shadow> (7)
              <View> (8)
                <Pressable> (9)
                  <AnimatedView> (10)
                    <Text> (11)
                    </Text>
                  </AnimatedView>
                </Pressable>
              </View>
            </Shadow>
          </View>
        </LinearGradient>
      </View>
    </View>
  </View>
</View>
```

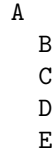
Bu yapı **çok pahalıdır**.

7) Çözüm: Derinliği Azalt, Genişliği Artır

Derin yapı (kötü):



Yüzeysel yapı (iyi):



Aynı UI, daha az hesaplama.

8) Pratikte Yapılması Gerekenler

- Gereksiz wrapper view'ları kaldır.
- Shadow + Opacity iç içe kullanmaktan kaçın.
- Gradient + Shadow + Animation kombinasyonunu azalt.
- Animated.View içinde çok fazla child bulundurma.
- FlatList içinde derin render ağacı oluşturma.
- Reanimated/Skia kullanarak UI thread'e offload et.
- Tasarımı 1:1 taşımak yerine mobil için optimize et.

9) Kaç Katman Güvenli?

$5 \text{ katman} \Rightarrow \text{geneldegüvenli}$

$10 + \text{ katman} \Rightarrow \text{jankriskiyüksek}$

Önemli olan views sayısı değil, view derinliğidir.

10) Son Özet

Nested view sorun değildir; deep nested view sorundur.

Amaç view sayısını değil, view derinliğini azaltmaktır.

20 WebView Kullanımı: Performans Problemleri ve Çözümler

1) WebView Nedir?

WebView, mobil cihaz içinde çalışan **mini bir tarayıcıdır**. Bu nedenle bir WebView:

- ayrı bir JavaScript engine,
- ayrı bir DOM ağacı,
- ayrı bir layout engine,
- ayrı bir render pipeline,
- ayrı thread ve process kaynakları

kullanır.

Bu özellikler nedeniyle WebView mobilde en **pahalı** UI elemanlarından biridir.

2) Neden 20 WebView Felakettir?

a) Çoklu Thread ve Process Açılması

Her WebView,

- UI bağlantısı,
- WebCore thread,
- JS thread,
- IO thread,
- GPU tarafında bir render pipeline

oluşturur. 20 WebView = 20 mini tarayıcı.

b) Aşırı Bellek Kullanımı

Tek bir WebView genellikle:

$$20-70MBRAM$$

kullanır. Bu nedenle:

$$20 \times 50MB = 1GBRAM$$

kullanımı çok sık görülür.

Düşük ve orta seviye cihazlarda bu doğrudan **OOM (Out of Memory)** **crash** üretir.

c) UI Thread Üzerindeki Yük

Her WebView şu aşamalardan geçer:

- measure,
- layout,
- style calculation,
- DOM update,
- painting,
- compositing,
- GPU rasterization

20 WebView, tüm bu işlemleri 20 kez tekrarlattığı için UI thread'in 16.6 ms frame bütçesini aşması *kaçınılmazdır*:

$$FrameTime > 16.6ms \Rightarrow Jank + Donma$$

d) JS Thread Çakışması

React Native'de:

- RN JS thread
- WebView'in kendi JS engine'i

eşzamanlı çalışır. 20 WebView, JS scheduler üzerinde yoğun çakışma ve **ANR** (Application Not Responding) üretir.

e) GC (Garbage Collector) Baskısı

20 farklı JS engine (V8 veya JSC) ayrı GC çalıştırır. Bu, UI thread'de **freeze/pause/stall** oluşturur.

f) CPU Kullanımının Aşırı Artması

Çoklu WebView kullanımı cihazda:

$$120\% - 150\% CPU (multi - core utilization)$$

gibi değerlere yol açabilir. Ardından cihaz ısınır, throttling başlar ve sistem uygulamayı öldürür.

3) Sonuç: 20 WebView Kullanılamaz Bir Ekran Üretir

Böyle bir ekran:

- takılır (jank),
- donma yaşar,
- scroll çalışmaz,
- input gecikir,
- ANR üretir,
- RAM aşımıyla çöker,
- yüksek enerji tüketir,
- iOS watchdog tarafından öldürülür.

Bu yapı **mobilde çalışabilir değildir**.

4) Doğru Yaklaşımlar (Çözümler)

- **Tek WebView kullanıp iç navigasyonu içeride yönetmek**
- **WebView clustering:** 1 WebView → çok sayfa
- **Lazy loading:** sadece ekranda görünen WebView'ları yüklemek
- **Virtualization:** görünmeyeni unmount etmek
- **Native render kullanmak:** HTML yerine native component
- **Sadece tek aktif WebView çalıştırmak**
- **Slider/Carousel içinde WebView kullanmaktan kaçınmak**

5) Son Özet

Aynı ekranda 20 WebView kullanmak mobil dünyada ciddi bir tasarım ve performans hatasıdır.

Bu, mini tarayıcıların 20 kez çalışması anlamına gelir ve cihazın doğal limitlerini aşar.

Mobilde Arka Planda Kesin Çalışan Tek Mekanizma

1) Gerçek: Uygulamanın Kodu Arka Planda Çalışmaz

Android ve iOS'ta uygulama arka plana geçtiğinde:

- zamanlayıcılar,
- döngüler,
- setTimeout / setInterval,
- JavaScript thread,
- background işlemler

tamamen durdurulur.

Hiçbir uygulama kendi kodunu sürekli arka planda çalıştıramaz. Sadece OS'in izin verdiği belirli görevler tetiklenebilir.

2) Android'de Arka Planda (Kısmen) Çalışabilenler

- AlarmManager (setExactAndAllowWhileIdle)
- WorkManager (OS izin verirse)
- JobScheduler (belirli koşullarda)
- Foreground Service (bildirim gösterilirse)
- BroadcastReceiver (bazı sistem event'leri)

Ancak bunların hiçbiri **%100 garantili değildir**. Kesin öldürülmeyen tek yapılar:

OS'in kendisi sistem servisleri

örneğin: LocationService, Alarm subsystem, Network stack.

3) iOS'ta Arka Planda Çalışabilenler

iOS daha kısıtlayıcıdır. Arka planda yalnızca OS'in tetiklediği işler çalışır:

- APNs push notification servisi (kesin çalışır)
- VoIP push
- Background Fetch (OS isterse)
- Background Tasks (OS planlarsa)
- Location updates (region, significant-change)
- Bluetooth / MFi event'leri

Temel kural:

“Senin kodun arka planda çalışmaz. OS isterse uyandırır.”

Notiffee Zaman Kayması (Alarm Drift) Problemi

1) Doze Mode Kaymanın En Büyük Sebebidir

Android cihaz uzun süre ekranda değilse **Doze Mode** devreye girer:

- timer'lar ertelenir,
- background task'ler uyutulur,
- alarm'lar sadece belirli bakım dilimlerinde (maintenance window) çalıştırılır,
- exact zamanı OS kendisi seçer.

Bu nedenle:

23:00 → 17:00

gibi büyük zaman kaymaları tamamen normaldir.

2) OEM Battery Killers

Samsung, Xiaomi, Oppo, Huawei gibi cihazlarda agresif güç tasarrufu:

- AlarmManager'ı geciktirir,
- WorkManager'i iptal eder,
- JobScheduler'ı erteler,
- local notification'ları öldürür.

Bu üreticiler, scheduled notification'ları değiştirir.

3) Exact Alarm İzni Yoksa OS Saati Yuvarlar

Android 12+ için exact alarm izni gereklidir:

```
<uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM"/>
```

Bu izin yoksa OS zamanı rastgele yakın bir slota taşır:

23:00 → 23:10, 22:45, *ertesigün*16:00, *hattalaysonra*17:00

4) Cihaz Uzun Süre Uyku Modundayla

Cihaz uzun süre Doze'da veya kullanılmamışsa:

- alarm kuyruğa alınır,
- ilk uygun zamanda çalıştırılır,
- OS zamanlamayı rastgele seçer.

5) React Native ve Notiffee'nin Suçu Değildir

Notiffee ve RN yalnızca Android'in API'lerini kullanır:

- AlarmManager
- WorkManager
- JobScheduler

OS izin vermezse drift olur. Kütüphanenin kontrol edemeyeceği bir davranıştır.

6) Çözüm Yolları

- **Exact alarm permission** eklemek.
- **Battery optimization whitelisting** istemek.
- Kesin garanti gerekiyorsa: **Foreground Service**.
- En güvenilir yöntem: **server-side scheduled push**.

20) Backpressure on Mobile Streams

Mobilde kamera, mikrofon, BLE sensörleri veya konum servisleri gibi **sürekli akan veri kaynaklarında** tipik bir problem oluşur:

$$\text{Üretici}(\text{producer}) \gg \text{Tüketici}(\text{consumer})$$

Üretici çok hızlı veri (frame/audio/event) gönderirken tüketici yavaş kalırsa:

- buffer şişer,
- memory artar,
- frame queue dolar,
- uygulama donar veya çöker.

Bu duruma **backpressure** denir.

Çözüm Teknikleri

- **Bounded Queues:** Kuyruk büyüklüğünü sınırlamak; fazla frame gelince eski frame'i atmak.
- **Frame Skip:** Her frame'i işlemek yerine belirli oranla frame atlamak.
- **Adaptive Bitrate:** Video kalitesini işlem gücüne göre dinamik düşürmek.
- **Audio Resampling:** Ses verisini daha düşük sample-rate'e dönüştürmek.

- **Event Sampling / Throttling:** GPS, BLE veya sensor event'lerini seyrekletmek.

Backpressure yönetimi mobil streaming mühendisliğinin temel yapı taşlarından biridir.

21) Mobile Security Concepts

Mobil güvenlik, backend güvenliğine ek olarak cihaz seviyesinde özel koruma katmanları gerektirir.

Cihaz Bazlı Güvenlik Bileşenleri

- **Secure Enclave (iOS) / Keystore (Android):** Kriptografik anahtarlar donanım güvenlik modülünde saklanır.
- **Keychain:** Kimlik bilgileri ve token'ların şifreli depolanması.

Cihaz Manipülasyonu Tespiti

- **Jailbreak / Root Detection:** Cihazın güvenlik sınırlarının aşılmasını tespit etmek.
- **Tamper Detection:** Uygulama dosyalarının değiştirilmesini algılamak.

Ağ Güvenliği

- **SSL Pinning:** Sertifika sahteciliği ve MITM saldırılarını engellemek için istemcinin beklenen sertifikayı doğrulaması.
- **Certificate Injection Detection:** Ortamda sonradan eklenmiş sahte CA sertifikalarını tespit etmek.

Kod ve Binary Koruması

- **Obfuscation (R8):** Kodun okunabilirliğini azaltarak reverse engineering'i zorlaştırmak.
- **Anti-Debugging / Anti-Hooking:** Frida, Xposed, LLDB gibi araçlarla müdahale girişimlerini engellemek.

Replay Attack Önlemleri

Mobil token'lar açısından:

- kısa ömürlü access token,
- refresh token rotation,

- nonce / request-id kullanımı,
- token binding (cihaza bağlı token)

Replay saldırılarını engellemek için kritik önlemlerdir.

Özet

Mobil güvenlik, sadece backend doğrulaması değil; cihaz, ağ, binary ve token seviyesinde çok katmanlı koruma gerektirir.

26) Mobile Compatibility Matrix

Web’de uygulama yalnızca tarayıcı üzerinde çalışır; ancak mobil dünyada çok geniş bir uyumluluk matrisi vardır:

Parçalanma (Fragmentation) Faktörleri

- **Android API Levels:** Her versiyonun farklı izin, güvenlik ve API davranışları vardır.
- **OEM Custom ROM’lar:** Xiaomi, Samsung, Oppo, Huawei kendi işletim sistemi katmanlarını ekler.
- **Device Fragmentation:** Binlerce farklı cihaz modeli ve ekran boyutu.
- **CPU ABI:** arm64-v8a, armeabi-v7a gibi farklı mimariler.
- **GPU Differences:** Mali, Adreno, PowerVR gibi farklı GPU sürücüler.
- **Vendor-Specific Bugs:** Xiaomi background kill, Samsung camera freeze gibi üreticiye özel hatalar.

Bu nedenle mobilde test matrisi web’e kıyasla çok daha karmaşıktır

Mobil Test Matrisi (Mobile Compatibility / Device Test Matrix)

Mobil uygulamalarda **test matrisi**, uygulamanın farklı **cihaz + işletim sistemi + ekran + donanım + OEM** kombinasyonlarında nasıl çalıştığını doğrulamak için oluşturulan kapsamlı uyumluluk tablosudur.

Web yalnızca tarayıcı çeşitliliğine sahipken, mobilde yüzlerce cihaz ve varyasyon vardır; bu nedenle test süreci çok daha karmaşıktır.

1. Android OS Versiyonları

Her Android cihaz aynı işletim sisteminde çalışmaz. Yaygın sürümler:

- Android 8
- Android 9
- Android 10
- Android 11
- Android 12
- Android 13
- Android 14

Her sürüm; izinler, background limitleri, notification davranışları gibi farklı API sınırlandırmalarına sahiptir.

2. iOS Versiyonları

Apple ekosistemi daha sade olsa da farklı versiyonlar uyumluluk gerektirir:

- iOS 15
- iOS 16
- iOS 17
- iOS 18

Her sürümde WebView, izin akışları ve UI davranışları değişebilir.

3. Cihaz Üreticileri (OEM)

Android dünyasında en büyük farklılık üreticilerin OS'i modifiye etmesidir:

- Samsung
- Xiaomi
- Huawei
- Oppo
- Vivo
- Realme
- OnePlus
- Google Pixel

Özellikle Xiaomi ve Huawei, agresif **background kill** kuralları ile bilinir.

4. Ekran Boyutları ve DPI (Density)

Mobil cihazlar farklı ekran ve piksel yoğunluklarına sahiptir:

- Küçük ekran: 4.7"
- Orta ekran: 6.1"
- Büyük ekran: 6.7"
- Tablet: 8"–12"
- Katlanabilir cihazlar (foldable)

DPI seviyeleri:

- mdpi
- hdpi
- xhdpi
- xxhdpi
- xxxhdpi

Yanlış çözünürlük kullanımı bulanık ikon ve bozuk layout'a neden olabilir.

5. Donanım Farkları (CPU/GPU/RAM)

Cihaz donanımı geniş bir skaladadır:

- 2–3 GB RAM → düşük segment
- 4–6 GB RAM → orta segment
- 8+ GB RAM → yüksek segment

GPU farklılıkları (Mali, Adreno, Apple GPU) özellikle animasyonlarda performans farkına sebep olur.

6. Permission (İzin) Güvenlik Davranışları

Her üretici izin modellerini farklı uygulayabilir:

- Kamera / mikrofon izinleri
- Background location
- Battery optimization istisnaları
- Auto-start izinleri (özellikle Xiaomi/Huawei)

7. Network (Bağlantı) Testleri

Mobil uygulamanın farklı ağ koşullarında test edilmesi gerekir:

- WiFi
- 4G/5G
- Kötü bağlantı (Edge)
- Offline senaryoları
- VPN bağlantısı
- Captive portal (otel, havaalanı ağları)

8. Neden “Test Matrisi” Denir?

Çünkü çok çeşitli kombinasyonlar vardır. Aşağıdaki tablo, bu karmaşayı özetler:

Cihaz	OS	DPI	CPU	OEM Ayarı
Samsung S23	Android 14	xxhdpi	Exynos	Battery optimize off
Xiaomi Redmi	Android 12	xhdpi	Snapdragon	Background kill on
Pixel 7	Android 14	xxhdpi	Google Tensor	Pure Android
iPhone 12	iOS 17	@3x	Apple A14	Stable background

Özet

Mobil test matrisi, uygulamanın farklı cihaz, OS, ekran, üretici, donanım ve ağ kombinasyonlarında sorunsuz çalışmasını garanti altına almak için oluşturulan kapsamlı bir uyumluluk planıdır.

Web’de yalnızca tarayıcı çeşitliliği varken, mobilde yüzlerce donanım ve OS kombinasyonu vardır.

Bu nedenle mobil test süreci **web’e göre 10 kat daha karmaşıktır**.

Touch Latency, Input Delay & Gesture Conflicts

Mobil platformlarda her dokunma etkileşimi, kullanıcının parmağı ekrana değdiği andan UI'nin tepki verdiği ana kadar bir dizi aşamadan geçer. Bu zincire **touch pipeline** denir.

Touch Pipeline İşleyişi

1. Touch Down

Kullanıcının parmağı ekrana temas eder. Cihaz bu ham olayı (raw touch event) işletim sistemine iletir.

2. Hit Test

OS + framework, bu dokunuşun *hangi* UI elemanına ait olduğunu belirler. (Örneğin: Button mı? ScrollView mı? Pan gesture mı?)

3. Gesture Recognizer

Dokunuşun bir *gesture* olup olmadığı anlaşılır: **tap**, **press**, **pan**, **swipe**, **long press**, **scroll** vb.

4. Responder Chain

Hangi bileşenin olaya yanıt vereceği belirlenir. Eğer birden fazla aday varsa, çakışma çözümü yapılır (conflict resolution).

5. Render Update

Gesture onaylanır, ilgili UI güncellenir ve ekrana yeniden çizim yapılır.

Bu aşamaların tamamı tek bir kritik hatta bağlıdır: **UI Thread**. UI thread bloklandığında tüm dokunma zinciri gecikir (input delay).

Sonuç Olarak Ortaya Çıkan Problemler

• Pan vs Scroll Conflict

Hem yatay hem dikey gesture aynı anda algılanmaya çalışıldığında hangi yönün baskın olacağı belirlenemez. Örneğin: Yatay slider içinde dikey scroll yapmak.

• Nested Scroll Issues

İç içe scroll yapılmış yapılarda (ör. yatay kart slider + dikey sayfa), hangi scroll view'ın dokunuşu devralacağı üzerinden çatışma oluşur.

• Press Delay / Input Gecikmesi

UI thread meşgulse:

- buton geç yanıt verir,
- dokunuş algılanmaz gibi görünür,
- uzun basma yanlışlıkla devreye girer,
- kullanıcı “uygulama dondu” zanneder.

- **Gesture Priority Conflicts**

Aynı dokunuşu farklı gesture recognizer'lar yakalamaya çalışabilir. Örnek:

- `ScrollView` scroll etmek ister,
- `PanGestureHandler` çekme (drag) yapmak ister,
- `TapGesture` tıklamak ister.

Doğru öncelik belirlenmezse UI **kararsız ve tahmin edilemez** davranır.

Önemli Sonuç

Dokunma gecikmesi (**touch latency**) modern mobil uygulamalarda kullanıcı deneyimini direkt etkileyen kritik konulardan biridir. UI thread'in bloklanması, yanlış gesture öncelikleri ve nested yapılarıdaki kararsızlıklar, input kalitesini ciddi şekilde düşürür.

Asset Resolution & DPI Density

Mobil cihazlar farklı DPI değerlerine sahiptir:

mdpi, hdpi, xhdpi, xxhdpi, xxxhdpi

iOS tarafında:

@1x, @2x, @3x

Yanlış çözünürlükte görsel kullanmak şu hatalara yol açar:

- bulanık ikonlar,
- düşük kaliteli UI,
- piksel kayması,
- farklı cihazlarda tutarsız görünüm.

React Native’de En Sık Yapılan Hata

Tek çözünürlükte görsel eklemek (ör. sadece @1x). Bu durumda yüksek DPI cihazlarda ikonlar bulanık görünür.

Doğru Yaklaşım

Tüm gerekli DPI versiyonlarını sağlamak:

- image.png
- image@2x.png
- image@3x.png

Offline-First Conflict Resolution

Mobil uygulamalarda **offline-first** yaklaşımı, kullanıcının internetsizken de veri oluşturabilmesini veya veriyi düzenleyebilmesini sağlar. İnternet bağlantısı geri geldiğinde bu lokal değişiklikler sunucuya gönderilir.

Eğer aynı veri hem cihazda hem sunucuda farklı şekillerde değişmişse bir **çatışma (conflict)** oluşur. Bu çatışmanın nasıl çözüleceğine *conflict resolution* denir.

Aşağıda çatışmayı çözmek için kullanılan başlıca yöntemler sade bir şekilde açıklanmıştır.

Çözüm Stratejileri

- **Vector Clocks:** Her değişikliğe mantıksal bir zaman etiketi eklenir. Böylece hangi değişikliğin daha yeni veya daha eski olduğu kolayca anlaşılır. Daha yeni olan genelde kazanır.
- **CRDT (Conflict-Free Replicated Data Types):** Aynı verinin birden fazla cihazda farklı şekillerde değişebilmesine izin veren ve internet geldiğinde bu değişiklikleri **otomatik olarak birleştiren** veri yapılarıdır. Google Docs tarzı sistemler bu yöntemi kullanır.
- **Last-Write-Wins:** En basit kuraldır: En son yapılan değişiklik kazanır. Bazı durumlarda veri kaybı oluşturabilir ama küçük uygulamalarda çok yaygındır.
- **MVCC (Multi-Version Concurrency Control):** Verinin birden fazla versiyonu tutulur. Local değişiklik eski bir versiyon üzerine yapılmışsa, sunucudaki yeni versiyonla karşılaştırılır ve çatışma tespit edilir.
- **Optimistic Merge:** Local değişiklik ile server değişikliğinin çarpışmadığı varsayılır. Eğer farklı alanlara dokunmuşlarsa otomatik olarak birleştirilir.
- **Custom Conflict Rules:** Bazı alanlar için özel kurallar tanımlanabilir: fiyat → server kazansın, açıklama → local kazansın, tarih → daha yeni tarih kazansın, gibi uygulamaya özel stratejiler.

Android Native Crash Bileşenleri

React Native uygulamalarında birçok hata **Java/Kotlin/ObjC/Swift/C++** tarafında oluşur. Bu nedenle Android tarafında native crash analizinde aşağıdaki üç yapı kritik öneme sahiptir.

1) Tombstones

Tanım: Native crash gerçekleştiğinde Android tarafından otomatik oluşturulan düşük seviye *core dump* dosyalarıdır.

İçerik:

- thread durumları
- register değerleri
- call stack
- memory map bilgileri

Bu dosyalar `/data/tombstones/` dizininde bulunur ve crash'in gerçek nedenini ortaya çıkarır.

2) NDK Crash Logs

Tanım: C/C++ kodunda oluşan *native* hataların loglarıdır.

Neye işaret eder?

- out-of-bounds array erişimi
- yanlış pointer kullanımı
- buffer overflow
- heap corruption

3) Sinyal Tabanlı Çökmeler

Tanım: Android, belirli tür hatalarda uygulamayı sinyallerle sonlandırır.

Örnekler:

- **SIGSEGV** — illegal memory access
- **SIGABRT** — abort() çağrısı veya fatal durum
- **SIGBUS** — yanlış hizalanmış bellek erişimi

Bu sinyaller, crash'in hangi kategoride olduğunu anlamak için önemlidir.

Özet

- **Tombstones:** Çökme anındaki eksiksiz sistem görüntüsü.
- **NDK Logs:** C/C++ kaynaklı hata analizi.
- **SIGSEGV/SIGABRT:** Hatanın türünü gösteren sinyaller.

iOS Native Crash Bileşenleri

React Native uygulamalarında iOS tarafındaki birçok çökme, **Objective-C / Swift / C++** kaynaklıdır. Crash analizi için aşağıdaki yapılar kritik öneme sahiptir.

1) Crash Logs

Tanım: iOS, bir uygulama çöktüğünde otomatik olarak *crash log* üretir. Bu loglar Xcode veya cihazdan alınabilir.

İçerik:

- crash nedeni (exception message)
- thread call stack
- register değerleri
- kullanılan framework'ler

Bu loglar, hatanın hangi native fonksiyonda oluştuğunu belirlemek için ilk kaynaktır.

2) dSYM ve Symbolication

Tanım: Release iOS build'lerinde call stack *obfuscate* olur (sayı ve adresler). dSYM dosyası, bu adresleri okunabilir fonksiyon isimlerine dönüştürmek için kullanılır.

Amaç:

$$MemoryAddress \longrightarrow Human - ReadableFunctionName$$

Gerekçe: Symbolication yapılmadan crash log'u anlamak *mümkün değildir*.

3) Native Exception Türleri

Örnekler:

- **EXC_BAD_ACCESS** — illegal hafıza erişimi
- **EXC_CRASH** — fatal runtime error

- **SIGSEGV / SIGABRT** — C tabanlı çökme sinyalleri

Bu hatalar genellikle:

- yanlış pointer kullanımı
- null referans erişimi
- hatalı bridging
- yanlış thread kullanımından

kaynaklanır.

4) Memory Debugging Araçları

iOS, native hafıza hatalarını tespit etmek için güçlü araçlar sunar.

- **Instruments** → **Leaks**: hafıza sızıntılarını tespit eder
- **Instruments** → **Allocations**: RAM kullanımını gösterir
- **Instruments** → **Zombies**: use-after-free hatalarını gösterir
- **Malloc Stack Logging**: hatalı malloc/free çağrılarını bulur

Özet

- **Crash Logs**: Çökme raporlarının temel kaynağı.
- **dSYM**: Logların okunabilir hale gelmesi için zorunlu.
- **Native Exceptions**: Hafıza ve thread kaynaklı çökme türleri.
- **Instruments**: iOS'ta native debugging için ana araç seti.

Native Crash Debugging — Üç Kritik Kavram

1) Native Memory Leak Detection

Tanım: Native tarafta (Java, Kotlin, Swift, ObjC, C++) hafıza ayrılır fakat iş bitince geri bırakılmaz. Bu durum zamanla RAM'in dolmasına ve uygulamanın OS tarafından öldürülmesine yol açar.

Örnek:

Bir view oluşturulur fakat hiçbir zaman serbest bırakılmaz.

Sonuç: Uygulama yavaşlar, memory usage sürekli artar ve sonunda crash oluşur.

Araçlar:

- iOS: Instruments (Leaks)
- Android: LeakCanary, MLeaks

2) Heap Corruption Analysis

Tanım: Native kodda yanlış hafıza bölgesine yazılması sonucunda hafızanın “bozulmasıdır”. Genellikle C/C++ tabanlı modüllerde görülür.

Wrong Write - Memory Corruption - Random Native Crash

Örnek: `c int arr[5]; arr[10] = 99; // sınır dışı yazma → heap corruption`

Sonuç: Uygulama beklenmedik şekilde çöker, crash stack anlamsız görünebilir.

3) Bridge Error Analysis (React Native)

Tanım: React Native'de JS ↔ Native arasındaki köprü üzerinden yanlış tip, çok büyük veri veya aşırı sayıda event gönderilmesi sonucunda köprünün çökmesidir.

Örnek Hata Senaryoları:

- JS tarafı büyük bir JSON gönderir (10-20 MB)
- Native taraf yanlış veri tipi bekler
- Çok fazla event kısa sürede köprüye gelir

Sonuç: Köprü (bridge) kilitlenir ve uygulama **native crash** verir. Bu crash JS tarafında görünmez; yalnızca Xcode logları veya Android tombstone içinde gözüktür.

Özet

- **Memory Leak:** Hafıza bırakılmaz → RAM dolar → crash.
- **Heap Corruption:** Yanlış hafıza alanına yazılır → sistem bozulur.
- **Bridge Error:** JS ve native arasındaki veri aktarımı çöker.

32) Mobile Rendering Architecture

Mobil uygulamalarda render mimarisi, web'deki DOM tabanlı modelden tamamen farklıdır. iOS ve Android'in her biri kendi grafik motoruna, kendi animasyon sistemine ve kendi kompozisyon pipeline'ına sahiptir.

React Native ise bu iki farklı pipeline'ın üzerinde çalışır ve kendi layout engine'ini kullanır.

Bu nedenle her platformda performans karakteristiği farklıdır.

iOS Render Pipeline

iOS'un tamamı **Core Animation** üzerine kuruludur. UI güncellemeleri, animasyonlar ve katman işlemleri CPU değil **GPU tarafından** işlenir.

- **Core Animation:** iOS animasyonları aslında "kod" değildir; GPU'ya gönderilen bir *timeline* tanımıdır. Bu yüzden iOS animasyonları son derece akıcıdır.
- **CALayer Tree:** Her UIView'in altında bir *CALayer* bulunur. iOS, View ağaçlarını değil, Layer ağaçlarını render eder. Bu katman yapısı GPU için optimize edilmiştir.
- **Metal / OpenGL Pipeline:** Son kademe (fragment + vertex shader) GPU üzerinde hesaplanır. Blur, shadow ve opacity gibi efektler burada uygulanır.

Animasyonlar çok akıcıdır çünkü GPU tarafından hesaplanır.

Android Render Pipeline

Android'de grafik motoru **Skia**'dır. Tüm 2D çizimler (shape, text, shadow, radius) Skia ile yapılır.

- **Skia:** Chrome ve Flutter'ın da kullandığı grafik motoru. Tüm çizimler CPU üzerinde hazırlanır, GPU'ya gönderilir.
- **Render Thread:** UI thread sadece *ne yapılacağını* hesaplar. Render Thread bu bilgiyi GPU'ya aktarılacak hale getirir (display list üretir).
- **Hardware Composer:** Android'in ekran kartı sürücüsü katmanıdır. Render edilmiş layer'ları nihai ekrana birleştirir (composition).

Shadow + Blur + Nested View - Jank riski artar

React Native Katmanı

React Native, iOS ve Android'in *kendi* pipeline'larına dokunmaz. Bunun yerine layout ve JS-native iletişimini yönetir.

- **Yoga Layout:** RN'in Flexbox tabanlı layout engine'i. Tüm ölçüm (measure) ve yerleşim (layout) hesapları burada yapılır.
- **RCTBridge Batching:** JS thread'deki değişiklikler tek tek değil, *batch halinde* native'e gönderilir. Bu performans kazandırır ancak aşırı veri gönderilirse **bridge bottleneck** oluşur.
- **Fabric Renderer:** Yeni RN mimarisi (JSI tabanlı). Daha hızlı commit aşaması, concurrent rendering, daha az bridge bağımlılığı sağlar.

React Native = İki farklı pipeline'ın üzerinde çalışan ayrı bir katman

Neden Önemli?

Farklı render pipeline'ları nedeniyle:

- animasyon hızları
- gesture tepkiselliği
- shadow/blur performansı
- nested view davranışı
- compositing maliyeti

iOS ve Android'de tamamen farklı olabilir.

Bu nedenle mobil optimizasyon *platforma özel* düşünülmelidir.

Mobile Pipeline Stalls (CPU–GPU Sync Bottlenecks)

Mobil render hattı iki ana bileşenden oluşur: CPU Pipeline + GPU Pipeline. Normalde CPU bir frame'i hazırlarken GPU bir önceki frame'i işler. Ancak belirli işlemler GPU'nun **bitirmesini bekler**, bu da pipeline'ın durmasına neden olur.

CPU–GPU Senkron Bloklama Sebepleri

- **Ağır shadow** (soft shadow, elevation)
- **Blur efektleri** (Gaussian blur)
- **Opacity layers** (katmanlı şeffaflık)
- **Clipping / mask işlemleri** (overflow: hidden)
- **Synchronous layout** (ani measure çağrıları)

Bu işlemler GPU tarafında uzun süren hesaplamalar gerektirir ve CPU, bir sonraki frame'i hazırlayamadan önce GPU'yu beklemek zorunda kalır.

Mobilde bu durum doğrudan *lag* üretir; web'de ise pipeline bağımsızlığı nedeniyle çok daha az hissedilir.

Main Thread Checker (iOS)

iOS, UI güncellemeleri için ayrılmış **main thread** üzerinde tehlikeli işlemleri tespit eden yerleşik bir kontrol mekanizmasına sahiptir. Bu sistem Xcode üzerinde uyarı üretir.

Tehlikeli İşlem Örnekleri

- **Disk IO** (dosya okuma/yazma)
- **Ağır malloc/free** (yoğun bellek tahsisi)
- **Synchronous networking** (blocking HTTP)

Örnek iOS Hatası

```
Main Thread Checker: UI API called on a background thread
```

Bu uyarılar, performansın ciddi şekilde etkilendiğinin göstergesidir.

Java/Kotlin Garbage Collector (Android)

Android'in Garbage Collector (GC) mekanizması, bellek temizliği yaptığında **UI thread'inin durmasına** neden olabilir.

GC'nin Tetiklenme Nedenleri

- **Büyük nesne tahsisi** (large arrays, images)
- **Sürekli alloc/free döngüsü**
- **Aşırı state değişimi** (RN setState spam)
- **Büyük JSON parse** (string → object)

Native Deadlocks

Deadlock, iki veya daha fazla thread'in birbirini bekleyerek sonsuz kilitlenme durumuna girmesidir. Bu hata çökmeye neden olmaz, ancak uygulama *donmuş* görünür.

Common Deadlock Senaryoları

- **UI Thread - JS Thread** karşılıklı lock beklemesi
- **Gesture Recognizer**'in kilitlenmesi (scroll vs pan çatışması)
- **Audio/Video stream mutex deadlock** (eşzamanlı buffer erişimi)
- **Camera warm-up deadlock** (kamera thread'i initialization'da takılır)

İki thread de birbirini beklediği için hiçbir zaman ilerleme olmaz.

Belirti

- uygulama donar
- scroll çalışmaz
- UI tepkisizdir
- ancak crash logu oluşmaz!

Deadlock = Sessiz, log'suz, en tehlikeli çökme türü

JNI / JSI Boundary Costs

React Native'de JS kodu ile Native kod arasında veri geçişi yapılırken **marshalling** maliyetleri oluşur.

Eski Köprü (Bridge/JNI) Mimarisi

- JS objeleri JSON'a çevrilir
- Java/Kotlin tarafına aktarılır
- C++ katmanına (JNI) geçilir
- Tekrar JS tarafına dönülür

Bu işlem: JSON serialize + copy + type-convert gibi pahalı aşamalar içerir.

JSI'nın Avantajı

JSI, JS ↔ Native iletişimde:

- **zero-copy transfer** (veri kopyalamadan)
- **pointer paylaşımı**
- **minimum marshalling**

kullanarak performansı ciddi şekilde iyileştirir.

Binder Transactions (Android IPC)

Android'de tüm bileşenler arası iletişim (Activity, Service, ContentProvider)

Binder IPC üzerinden gerçekleşir

Binder payload limit = 1 MB

Bu sınır aşılsa: TransactionTooLargeException

Thermal Throttling

Mobil cihaz ısındığında (CPU/GPU sıcaklık eşiği aşılnca): CPU frequency azalır, GPU speed azalır

Bu, animasyonların ve uygulama akıcılığının düşmesine neden olur.

Isınmaya Sebep Olan İşlemler

- Yüksek FPS (60–120 FPS)
- Kamera streaming (4K video feed)
- Crypto işlemleri (hashing, encrypt/decrypt)
- Video encode/decode
- Sensor fusion (GPS + IMU + ML)

In-App Browser ve WebView Multiprocess Architecture

Mobil uygulamalarda WebView, web içeriklerini uygulama içinde göstermek için kullanılır. Modern WebView yapısı (Chrome WebView, Safari WKWebView) monolitik değildir; **çok süreçli (multi-process)** bir mimaride çalışır.

Bu model, masaüstü tarayıcıların güvenlik ve performans prensipleri mobil tarayıcılara taşınarak oluşturulmuştur.

WebView'in Çok Süreçli Yapısı

WebView içindeki işlemler farklı süreçlere ayrılarak izole edilir:

- **Rendering Process:** HTML, CSS, layout, paint, style, reflow ve DOM operasyonları gerçekleştirilir. Tüm UI manipülasyonları bu süreçte yapılır.
- **Network Process:** HTTP istekleri, cookie yönetimi, redirect işlemleri ve TLS handshake bu süreçte olur. Render sürecinden tamamen ayrıdır.
- **GPU Process:** Layer compositing, shader çalıştırma ve piksel işleme GPU tarafında yapılır.

- **Main App Process:** React Native veya native uygulama kodu burada çalışır. WebView bu sürecin içinde sadece bir *container* olarak bulunur.

Bu ayrım, bir süreç çökerse diğerlerinin çalışmaya devam etmesini sağlar.

Avantajlar

- **Process izolasyonu sayesinde daha az crash:** Web içeriklerinin düşmesi, uygulamanın kendi sürecini etkilemez. Sadece WebView rendering süreci yeniden başlatılır.
- **Daha güvenli çalışma modeli:** Cross-site scripting, exploit girişimleri veya kötü amaçlı JS kodu ana uygulama sürecine erişemez.
- **Daha iyi performans:** Ağ trafiği, layout hesapları ve GPU işleri farklı süreçlerde yürütüldüğü için CPU yükü dengelenir.
- **UI Responsiveness artar:** Network process yavaşlarsa bile UI thread bundan etkilenmez.

Dezavantajlar

- **Yüksek bellek tüketimi:** Her bir süreç ayrı memory footprint oluşturduğu için toplam RAM kullanımı artar. Eski cihazlarda WebView açmak 150–350 MB ek RAM tüketimine yol açabilir.
- **IPC maliyeti (Inter-Process Communication):** Süreçler arası mesajlaşma (render network GPU) **serialization** ve **copy** maliyeti yaratır. Bu nedenle WebView içinde büyük JSON transferi pahalıdır.
- **Gesture/hit-test çatışmaları:** WebView içindeki olay yakalama (touch, scroll, pinch) ana uygulamanın gesture sistemiyle çakışabilir. Özellikle React Native scroll view içerisinde WebView kullanıldığında *nested gesture conflict* oluşur.
- **Daha karmaşık debugging:** WebView crash olduğunda stack trace ana süreçte görünmez; crash genellikle rendering process içinde olur.

React Native Perspektifi

React Native’de WebView, uygulamanın kendi JS ve native kodundan tamamen ayrı bir render pipeline’a sahiptir.

- RN layout (Yoga) WebView içeriğine karışamaz.
- WebView içindeki JS, RN JS thread’iyle *aynı değildir*.
- WebView kendi JS motoruna, CSS engine’ine ve DOM yapısına sahiptir.
- PostMessage iletişimi süreçler arası olduğu için maliyetlidir.

- Bir ekranda birden fazla WebView kullanmak RAM + GPU yükünü katlar.

Sonuç

WebView'in çok süreçli yapısı şu anlama gelir:

- yüksek izolasyon = daha kararlı
- yüksek RAM kullanımı = daha pahalı
- IPC maliyeti = daha yavaş iletişim
- gesture çatışmaları = dikkatli kullanım gerektirir

React Native uygulamalarında WebView kullanımı, tamamen ayrı bir tarayıcı motoru entegre etmek anlamına geldiği için, performans ve bellek açısından dikkatli bir mimari planlama gerektirir.

Mobil Performans Metrikleri: TTI, TTFB, TBT

Web'de olduğu gibi mobilde de performans ölçümü **metrik tabanlı** yapılır.

Kritik Mobil Metrikler

- **TTI (Time To Interactive)**: Uygulamanın etkileşime hazır olduğu zaman.
- **TTFB (Time To First Byte)**: İlk network byte'ının gelmesi.
- **TBT (Total Blocking Time)**: JS thread'in kilitli olduğu toplam süre.
- **Startup Latency**: App'in açılış süresi.
- **First Render Time**: İlk UI'nın çizilme süresi.
- **Input Delay**: Dokunmaya verilen gecikme.

Bu metrikler iyileştirilmeden mobil uygulama gerçek anlamda optimize edilmiş sayılmaz.

Navigation Stack Depth Issues

Mobil platformlarda navigation **stack tabanlı** çalışır (push, pop, replace). Doğru yönetilmediğinde ciddi problemler yaratır.

Tipik Problemler

- **Improper Pop:** Yanlış ekrandan pop yapılması, stack'in bozulmasına sebep olur.
- **Orphan Screens:** Bir ekran stack'te görünür fakat artık erişilebilir değildir (navigation ağacı bozulmuş).
- **Unmounted Screens Still Listening:** Unmount olmuş ekranın event listener'ları temizlenmez → memory leak.
- **Navigation Closures Leak:** React Navigation'ın closure yapısı nedeniyle ekran kapanınca bile bazı referanslar RAM'de kalır.
- **Gesture Conflicts:** Swipe-to-go-back ile kendi gesture'larının çakışması.
- **Remount vs Focus:** Ekran yeniden mount edilmesi gerekirken sadece focus değişir → state yanlış görünür.

Bu alan **navigation engineering** olarak ayrıca ele alınır.

Mobile Data Layer Inconsistency

Mobil veri katmanı backend'den daha karmaşıktır çünkü cihaz offline olabilir, ağ zayıf olabilir, cache farklı cihazlarda farklı state taşır.

Karşılaşılan Sorunlar

- **Stale Cache:** Eski verinin gösterilmesi (TTL sürelerinin yanlış ayarlanması).
- **Multiple Sources of Truth:** API yanıtı, local DB, RAM cache, Redux/MMKV birbirinden farklı olabilir.
- **Out-of-Sync Store:** Offline değişiklikler henüz sync edilmemiştir.
- **Partial Write / Partial Sync:** Veri kısmen yazılır; uygulama farklı bir state gösterir.
- **Optimistic UI Rollback:** Hata geldiğinde geri alma (undo) kurgusu yapılmaz → kullanıcı hatalı state görür.
- **Version / eTag Mismatch:** Server'ın son versiyonu ile cihazdaki versiyon tutmaz.

Bu nedenle mobilde *hydration* → *dehydration* stratejileri kritik önem taşır.

Mobile Connection State Modeling

Mobil bağlantı durumu **sadece online/offline değildir**. Gerçek hayatta çok farklı network durumları bulunur.

Bağlantı Durumları

- **Captive Portal**: Otel, havaalanı ağında login ekranı gerekir.
- **Weak WiFi**: WiFi bağlı görünür ama veri yoktur.
- **Slow Cellular**: 3G veya düşük LTE hızında API timeout'ları oluşur.
- **Flaky LTE**: Bağlantı saniyelik kopup geri gelir.
- **Switching Networks**: WiFi LTE geçişlerinde socket ve stream çöker.
- **VPN**: DNS çözümleri, routing ve latency değişir.
- **Background Network Cut**: OS arka planda ağı tamamen kapatır.
- **Metered Network (Android)**: Veri maliyetli olduğu için OS throttling uygular.

Yanlış modelleme, uygulamanın "çökmüş gibi" algılanmasına yol açar.

Audio Focus & Media Session Conflict

Mobil ses sistemi paylaşımlıdır. Başka bir uygulama müzik çalışırsa senin uygulaman ses çalamaz veya ses seviyesi düşürülür.

Temel Kavramlar

- **Audio Focus Gain/Loss**: Uygulama odak kazanır veya kaybeder.
- **Ducking**: Ses kısılır (navigasyon talimatları gibi senaryolarda).
- **Background Audio Session**: Uygulama arkadayken bile ses çalmaya devam etme.
- **Interruptions**: Arama veya bildirim sırasında ses kesilir.
- **Bluetooth Events**: Kulaklık tuşu, bağlantı kopması, mikrofon yönlendirmesi gibi olaylar.

Voice/video chat uygulamalarında kritik önem taşır.

Camera HAL (Hardware Abstraction Layer)

Her cihazın kendi kamera sürücüsü vardır. Bu nedenle Android kamera dünyası tutarsızlıklarla doludur.

Sorunlar

- **Sensor Orientation Differences:** Cihazın kamerayı fiziksel olarak farklı konumlandırması.
- **Autofocus Bug'ları:** Bazı modellerde odak kilitlenmez.
- **Exposure Compensation Farkları:** Parlaklık ayarları cihazdan cihaza değişir.
- **Frame Delivery Order:** Bazı cihazlar frame'leri ters sırada veya gecikmeli teslim eder.
- **Manufacturer-Specific Pipelines:** Samsung, Xiaomi, OnePlus kendi özel kamera pipeline'larına sahiptir.

Bu yüzden kamera modülleri tüm cihazlarda aynı davranmaz.

Geofencing & OS-Level Aggressive Kill

Android üreticilerinin güç optimizasyon politikaları geofencing'i gerçek hayatta çalıştırmayı çok zorlaştırır.

OEM Davranışları

- **Xiaomi / Huawei / Oppo:** Arka planda GPS'i keser, JobScheduler'ı öldürür.
- **Background Task Kill:** Uygulama kapalıyken konum güncellemesi gelmez.
- **Doze Mode Etkisi:** Geofencing event'leri ciddi gecikmeye uğrar.
- **OS-Level GPS Throttling:** Sık konum talebi yapan uygulamalar sınırlandırılır.

In-App Purchase (IAP) Edge Cases

Mobil uygulamalarda In-App Purchase (IAP) akışı, **Apple App Store** ve **Google Play Billing** gibi harici sistemlere bağlıdır. Bu nedenle hata oranı yüksektir ve beklenmedik durumlar sıkça ortaya çıkar.

Aşağıda IAP sürecinde en sık karşılaşılan kritik edge-case senaryoları yer almaktadır.

1) User Cancels

Kullanıcı satın alma ekranını açar fakat işlem sırasında **geri** veya **iptal** tuşuna basar. Bu durumda `purchaseCancelled` event'i yönetilmezse UI yanlış durumda kalabilir.

2) Pending Purchase

Bazı ödeme yöntemleri (ör. kredi kartı doğrulaması, aile paylaşımı onayı) satın almayı **pending** durumda bırakabilir. Bu durumda kullanıcı ürünü satın almış görünür fakat ürün teslim edilmemiştir.

3) Refund (Geri Ödeme)

Kullanıcı App Store veya Play Store üzerinden para iadesi alır.

- iOS: refund notifications
- Android: Google Play Billing ACK / revoke events

Altyapı bu durumu takip etmezse kullanıcı ödemesiz premium kullanmaya devam eder.

4) Restore Failure

Restore purchases işlemi sırasında:

- ağ kopabilir
- kullanıcı Apple hesabından çıkmış olabilir
- Play Store / App Store servisleri çalışmayabilir

Bu durumda UI yanlış bilgi gösterir.

5) Signature Mismatch

Android tarafında purchase token doğrulanırken:

- signature mismatch
- invalid token

- tampered receipt

hataları çok yaygındır. Backend doğrulaması yapılmazsa fraud açığı oluşur.

6) Connectivity Loss

İşlem sırasında kullanıcı:

- interneti kapatır
- WiFi LTE geçişi yapar

Bu durumda satın alma event'i gelir ama ürün teslim edilmez.

7) Platform-Side Errors

App Store / Google Play bazen şu hataları üretir:

- billing unavailable
- service disconnected
- storekit error
- purchase in progress

Bu hatalar uygulama kodu kaynaklı değildir; platform kararsızlığıdır.

8) Wrong Receipt Validation

Backend tarafında yanlış doğrulama yapılırsa:

- eski receipt kabul edilir
- expire olmuş subscription aktif görünür
- sandbox receipt production ortamında reddedilir

9) Device Clock Drift

Cihazın saati yanlış ayarlı ise:

- trial süreleri yanlış hesaplanır
- abonelik bitişi UI'da hatalı görünür
- satın alma “expired” diye algılanır

10) Trial Expiration

Deneme süresi (free trial) bittiğinde:

- yenileme başarılı olamayabilir
- kullanıcı ödeme yöntemini değiştirmiş olabilir
- bankanın 3D Secure doğrulaması eksik olabilir

Bu durumda kullanıcı “premium gitti” sanır ve support talepleri artar.

Sonuç

In-App Purchase süreci çok fazla harici sisteme bağlı olduğu için, edge-case senaryoları bilinmeden yapılan IAP entegrasyonları genellikle yüksek crash/bug oranı ile sonuçlanır.

Sağlam bir IAP akışı için:

- backend receipt validation,
- pending/restore yönetimi,
- refund event handling,
- doğru UI state yönetimi

zorunludur.

Layout Thrashing (Measure–Layout Feedback Loops)

Mobil UI layout sistemi pahalıdır. Yanlış kullanımda **ölçüm (measure)** ve **yerleşim (layout)** döngüleri birbirini tetikleyerek performans çöküşü yaratır.

Nedenleri

- **Sürekli measure() çağrısı:** Component yeniden ölçülür ve tüm alt elemanlar tekrar hesaplanır.
- **Parent → Child → Parent döngüsü:** Yoga/Flexbox parent ve child boyutlarını karşılıklı etkileyerek tekrar tekrar layout pass tetikler.
- **Forced Layout Pass:** State değişimiyle tüm ekran yeniden layout'a zorlanır.
- **Nested Flexbox Karmaşası:** Çok iç içe flex yapıları recursive layout zinciri oluşturur.

Sonuç

- frame drop
- CPU spike
- battery drain

Bu problem React Native + Flexbox kombinasyonunda çok yaygındır.

Texture Upload Bottlenecks

Bir görsel (texture) GPU'ya yüklenirken UI thread durabilir.

Neden

- Büyük PNG/JPEG decode işlemi
- GPU'ya veri yükleme (texture upload)
- Base64 inline image kullanımı
- Dynamic image swapping

Çözüm

- WebP kullanımı
- Görsellerin preloading yapılması
- Lazy decode teknikleri
- Texture atlas kullanımı

Double Buffering / Triple Buffering

Mobil render pipeline'ı üç aşamada çalışır:

1. CPU bir frame hazırlar.
2. GPU frame'i işler.
3. Ekran frame'i gösterir.

Bu üç aşama **senkronize değilse**:

- stutter
- jitter
- partial frame artifacts

Oyun dünyasında kritik olduğu kadar mobil UI animasyonlarında da önemlidir.

Hotspot Detection (CPU Hot Path Analysis)

Uygulamanın yüzde 80 CPU kullanımını oluşturan kod genellikle **sadece %10–20 satırdır**.

Araçlar

- Android Profiler
- Xcode Instruments
- Flamegraph
- Hermes Profiling

Sonuç

Hot-path optimize edilmezse:

- battery drain
- jank
- FPS düşüşü

Proximity & Ambient Light Sensors

Mobilde iki sensör kritik rol oynar:

- **Proximity Sensor:** Yüz yakinken ekran kapanır (ör. telefon görüşmesi).
- **Ambient Light Sensor:** Ortam ışığına göre parlaklık ayarı.

Kamera, AR, navigation ve ışık kontrolü yapan uygulamalarda bu sensörlerin davranışlarını bilmek zorunludur.

Bounded Memory Pools (Slab Allocators)

Mobilde memory allocation pahalıdır. Bu nedenle:

- **Object Pooling:** Sık kullanılan nesnelerin yeniden kullanılması.
- **Pre-allocated Buffers:** Önceden tahsis edilen sabit boyutlu bellek blokları.
- **Slab Allocation:** Sayfa boyutunda (page-sized) bellek havuzları.

Bu teknik mobil uygulamalarda performans kazancı sağlar.

Intermittent Crash Patterns

Bazı mobil crash'ler **tekrar edilemez**. Bunun nedeni:

- race conditions
- nondeterministic threading
- GC zamanlaması
- sensor event timing
- network flakiness
- OS background kill

Mobil debugging deterministik değildir — olay dizisine bağlıdır.

Touch Sampling Rate & High-Refresh Displays

Modern telefonlarda iki farklı hız kavramı vardır:

- **Display Refresh Rate (Hz)**: Ekranın bir saniyede kaç kere yenilendiği.
- **Touch Sampling Rate (Hz)**: Ekranın bir saniyede kullanıcı dokunuşunu kaç kez örneklediği.

Bunlar birbirinden bağımsızdır.

Örnek Donanım Kapasiteleri

- Ekran yenileme: 90 / 120 / 144 Hz
- Dokunuş örnekleme: 120 / 180 / 240 Hz

Bu şu anlama gelir: Ekran saniyede 120 kere çizilir, ama dokunuş 240 kez ölçülebilir.

Touch Pipeline (Dokunma İşleme Hattı)

Mobil işletim sistemlerinde bir **touch event**'i şu aşamalardan geçer:

1. **Touch Sampling**: Donanım parmağın konumunu 120–240 Hz hızla toplar.
2. **Event Dispatch**: OS, bu veriyi UI frameworküne gönderir.
3. **Hit Testing**: Hangi komponentin bu dokunuşu alacağı belirlenir.
4. **Gesture Recognizers**: Pan, scroll, long-press, tap gibi tanımlayıcılar yarışır.
5. **Responder Chain**: Event hangi bileşene iletilecek karar verilir.
6. **Re-Render / Layout Update**: UI thread gerekirse layout ve render günceller.
7. **Display Refresh**: Ekran 16.6 ms (60 Hz) veya 8 ms (120 Hz) sonunda frame'i gösterir.

Zorluk: Pipeline Zamanlaması

Eğer herhangi bir aşama şu sınırı aşarsa:

$$FrameBudget = 16.6ms(60FPS) \quad veya \quad 8.3ms(120FPS)$$

UI gecikmeye başlar.

React Native Özelindeki Sorun

React Native’de tüm gesture işlemleri **JS thread** üzerinden geçtiği için:

- JS thread bloklanırsa → dokunuş *geç* işlenir.
- Layout thrash olursa → scroll takılır.
- Re-render ağırsa → frame skip oluşur.
- Event queue dolarsa → gesture drop yaşanır.

Neden Yüksek Hz Cihazlarda Daha Belirgin?

120 / 144 Hz bir ekranda her frame için:

$$\frac{1}{120} \approx 8.3ms$$

$$\frac{1}{144} \approx 6.9ms$$

UI thread’in hata toleransı düşer. Bu nedenle yüksek refresh rate cihazlarda:

- en küçük JS lag = daha belirgin gecikme
- drop edilen frame = gözle daha net fark edilir
- micro-jank = hissedilir

Sorun Örnekleri

- **Input Lag:** Buton geç yanıt verir.
- **Scroll Delay:** Scroll “takılarak” başlar.
- **Gesture Drop:** Pan/Swipe hareketi hiç algılanmaz.
- **Rubber-band Effect:** Dokunuş ile ekran hareketi arasında gecikme.

Çözüm Teknikleri

- **Reanimated 2 / Gesture Handler:** Gestures → UI thread’e taşınır (JS bloklansa bile çalışır).
- **Batching & Throttling:** Çok fazla gesture callback’i azaltılır.
- **JS Thread Offloading:** Ağır iş Native module veya Worker’a taşınır.
- **Minimal Re-render:** setState kullanımını azaltma, memoization.
- **FlatList windowing optimization:** Yüksek refresh rate’te scroll performansı artar.
- **Avoid Layout Thrashing:** Measure çağrılarını azaltmak.

Sonuç

Touch sampling rate yüksek olduğunda cihaz dokunuşu çok hızlı toplar; ancak UI thread bu hızda işleyemezse sorun olur. Bu nedenle yüksek Hz ekranlarda RN uygulamalarında gesture optimizasyonu kritik önem taşır.

App Wake Locks (Android)

Wake-lock Android'in cihazı uyanık tutma mekanizmasıdır.

Türleri

- **Partial Wake Lock:** CPU uyanık kalır, ekran kapanır.
- **Full Wake Lock:** Ekran ve CPU açıktır.

Yanlış kullanımda:

- cihaz sabaha kadar uyanık kalır
- batarya hızla tükenir

Google Play App Standby Buckets

Android uygulamaları kullanım sıklığına göre kategorize eder:

- Active
- Working Set
- Frequent
- Rare
- Restricted

Her kategori farklı:

- ağ erişimi
- arka plan çalışması
- alarm zamanlaması

limiti taşır. Bu nedenle aynı uygulama farklı cihazlarda farklı davranır.

App Preload Optimization (OEM)

Bazı cihazlar (Samsung, Xiaomi) uygulamayı preload eder:

- kaynakları önceden yükler
- DEX optimizasyonu yapar
- caching / warming uygular

Etkiler

- cold start hızlanır
- bazı race condition'lar farklılaşır
- farklı cihazlarda farklı başlangıç davranışı görülür

Google Play Store Ranking Faktörleri

Google Play sıralaması, uygulamanın indirilme sayısından çok **kullanıcı davranışı, performans metrikleri ve etkileşim sinyallerine** dayalıdır. Aşağıda sıralamayı etkileyen en önemli 10 faktör, büyüklüğe göre sıralanmış şekilde verilmiştir.

1. Retention Rate (Geri Dönüş Oranları)

Google Play'in en güçlü sinyallerinden biridir.

- D1 retention
- D7 retention
- D30 retention

Kullanıcının uygulamayı düzenli açması sıralamayı doğrudan yükseltir.

2. User Engagement (Kullanıcı Etkileşimi)

Kullanıcının uygulamada geçirdiği süre ve gezindiği ekran sayısı önemlidir.

- ekran derinliği,
- günlük kullanım süresi,
- aksiyon sayısı

3. Crash Rate & ANR Rate

Yüksek ANR ve crash oranı sıralamayı dramatik şekilde düşürür. Google Play konsolu bunları *ranking killer* olarak işaretler.

4. Review Score & Review Quality

Sadece puan değil, yorum içeriği NLP ile analiz edilir.

- pozitif kelime oranı,
- yorumu yazma tarihi,
- kullanım bağlamı

5. Uninstall Rate

Kullanıcının uygulamayı yükledikten kısa süre sonra silmesi negatif ranking sinyalidir.

6. Download Velocity & Trend

Son 30 gün içinde indirme artışı ve rakiplere göre hızlanma, Google Play’de yükselmeye neden olur.

7. Keyword Relevance (ASO)

Başlık, açıklama ve asset içerikleri Google NLP ile analiz edilir.

- başlık anahtar kelimeleri,
- açıklama yoğunluğu,
- görsel metinler

8. Update Frequency

Düzenli güncelleme, uygulamanın bakımda olduğu sinyali verir ve sıralamayı pozitif etkiler.

9. Regional Behavior

Sıralama ülkeye özeldir. Türkiye’de:

- retention,
- uninstall rate,
- local keyword performansı

ayrı ayrı hesaplanır.

10. Store Listing Conversion Rate

Kullanıcı mağaza sayfanı görüp indirmeye karar veriyor mu?

- ikon,
- screenshot,
- preview video,
- kısa açıklama

CTR (click-through rate) doğrudan ranking sinyalidir.

Sonuç

Kullanıcının uygulamayı sık açması (**retention + engagement**) sıralamayı artırır, ancak tek başına yeterli değildir. Crash oranı, uninstall oranı, yorum kalitesi, ASO ve indirme hızı ile birlikte değerlendirilir.

Native Heap Fragmentation

Mobil uygulamalarda iki tür hafıza bulunur:

- **JS Heap** — Küçük, JavaScript kodunun çalıştığı alan.
- **Native Heap** — Bitmap, kamera buffer'ları, codec, GPU, C++/ObjC/Swift belleklerinin tahsis edildiği büyük alan.

Native heap, sürekli farklı boyutlarda bellek tahsis (allocate) ve serbest bırakma (free) işlemleri nedeniyle **parçalı** bir hale gelir. Bu duruma **heap fragmentation** denir.

Sorunun Temel Tanımı

Fragmentation oluştuğunda toplam RAM boş görünse bile, sistem büyük bir bellek bloğunu *tek parça* hâlinde bulamadığı için tahsis işlemi başarısız olur.

RAM boş gözükür **ama** büyük blok halinde boş alan bulunamadığı için
“allocation failed” hatası alınır.

Neden Oluşur?

- Büyük/küçük sürekli bitmap decode/free işlemleri
- Kamera buffer'larının sık tahsis edilmesi
- Lottie, GIF, WebP decoding işlemleri
- Büyük JSON parsing için native buffer tahsisi
- Video frame decoding (MediaCodec, AVFrame)
- Texture allocate/free (OpenGL/Metal)
- RN bridge'de küçük sürekli buffer tahsisleri

React Native'de Neden Daha Sık Görülür?

React Native aynı anda birden fazla native alt sistemi kullanır:

- Hermes/JSC JS engine (JS Heap)
- Yoga layout engine
- Skia / Image pipeline
- Bitmap decoding
- Bridge buffer'ları

- Texture ve GPU yüzeyleri

Bu bileşenlerin hepsi **aynı native heap** üzerinde çalıştığı için, farklı boyutlarda sık tahsis/free işlemleri büyük oranda fragmentation oluşturur.

Sonuçlar

- **RAM boş görünür** fakat büyük bir blok bulunamadığı için allocation başarısız olur.
- **Camera init fail** (preview buffer tahsisi yapılamaz)
- **Large bitmap decode fail**
- **OOM crash** (“Failed to allocate large object”)
- **libc mmap errno 12 (ENOMEM)** hataları

Neden JS Tarafında Görünmez?

React Native debug ekranında yalnızca **JS Heap** görünür. Native heap kullanımı veya fragmentation hiçbir yerde gösterilmez. Bu nedenle geliştirici RAM kullanımının düşük olmasına rağmen crash yaşar.

Optimizasyon Önerileri

- Büyük bitmap’leri decode etmeden önce downscale et (Android `inJustDecodeBounds`)
- Gereksiz `ImageView`’ları `unmount` ederek erken free edilmesini sağla
- `FastImage` ile disk/memory cache kullan
- Lottie yerine mümkün olduğunda WebP animasyon kullan
- Kamera preview resolution’ını düşür
- Skia kullanılıyorsa PNG/JPG yerine WebP/AVIF tercih et
- Sık `allocate/free` yapan blur, shadow, dynamic radius gibi pahalı view’lardan kaçın

Kısa Özet

Native heap fragmentation = RAM boş görünmesine rağmen, tek parça büyük bir bellek alanı bulunamadığı için kamera, bitmap veya GPU işlemlerinin **başarısız olmasıdır**. React Native’de en sık görülen gizli crash sebeplerinden biridir.

Mobile Clock Drift / NTP De-Sync

Cihaz saati yanlış olduğunda:

- token expiration hataları
- payment signature mismatch
- scheduled task yanlış zamanlanır
- push notification yanlış saatte gelir
- sync conflict ortaya çıkar

Saat sapması mobilde çok sık rastlanan gerçek bir problemdir.

iOS RunLoop Internals

iOS'ta tüm UI işlemleri **RunLoop** üzerinde çalışır. RunLoop bir döngüdür ve her iterasyonda şu aşamalardan geçilir:

- **Timers:** Zamanlayıcılar işlenir.
- **Input Sources:** Touch, gesture, network gibi event kaynakları.
- **Dispatch Queues:** GCD ile kuyruklanan işler.
- **Render Commit:** UI güncellemeleri Core Animation'a iletilir.
- **Idle:** Boşta kalan süre.
- **Sleep:** İş yoksa thread uyur.

Yanlış aşamada ağır iş çalıştırmak **frame kaybına** neden olur.

Android Looper / Handler MessageQueue

Android UI Thread'in kalbi:

- **Looper:** Sonsuz döngü.
- **MessageQueue:** Tüm UI işlerinin kuyruğu.
- **Handler:** Mesaj gönderme aracı.

Her UI güncellemesi bir **Message** olarak kuyruğa girer. Queue tıkanır → **ANR (Application Not Responding)** oluşur.

GPU Tile-Based Rendering

Mobil GPU'lar çoğunlukla **tile-based rendering** kullanır:

- ekran küçük “tile” bloklarına bölünür,
- her tile GPU’da ayrı hesaplanır,
- memory-bandwidth ihtiyacı azalır.

Yanlış kullanım:

- aşırı blending,
- overdraw,
- büyük opacity yüzeyleri,
- pahalı shadow.

Tile mimarisine ters UI → **jank**.

Display Compositors

Ekrana çizimi yöneten ana bileşenler:

- **iOS: Core Animation**
- **Android: SurfaceFlinger**

Bu compositor, “layer tree”yi birleştirerek ekran çıktısını oluşturur. Aşağıdaki işlemler compositor seviyesinde ciddi maliyet üretir:

- opacity,
- transform,
- clip,
- z-index layering.

Memory-Mapped Files (mmap)

Büyük dosyalar RAM'e tam yüklenmez; **sayfa** bazlı okunur.

Kullanım alanları:

- offline maps,
- dictionary apps,
- ML model loading,
- video segment buffers,
- büyük JSON parsing.

Avantajlar:

- çok düşük RAM kullanımı,
- hızlı random access,
- hızlı startup.

Shader Warm-Up

Mobil GPU'lar, çeşitli UI efektlerini çizmek için **shader** adı verilen küçük GPU programlarını kullanır. Bu shader'lar ilk kullanımda derlenir (*runtime shader compilation*). Bu derleme işlemi genellikle **ilk karede jank** veya kısa süreli donma oluşturur.

Shader Nedir?

Shader, GPU'nun bir görsel elemanı nasıl çizeceğini tanımlayan küçük programdır. Örnek shader türleri:

- Blur shader
- Gradient shader (linear, radial)
- Shadow shader
- Mask / Animated border shader
- Skia, Lottie veya SVG için özel shader'lar

Bu shader'lar:

- OpenGL (GLSL)
- Metal shader language
- Vulkan SPIR-V
- Skia'nın SkSL dili

gibi GPU pipeline'larında derlenir.

Neden Jank Oluşur?

Shader compile işlemi **ilk kullanım anında** gerçekleşir. Bu işlem 2-30 ms (bazı cihazlarda 50 ms) sürebilir. Mobilde 60 FPS için:

$$Framebudget = 16.6ms$$

Dolayısıyla shader compile süresi bu eşiği aşarsa **ilk animasyon karesinde frame drop** yaşanır.

Hangi UI Elemanlarında Shader Jank Görülür?

- Blur view'ların ilk görünmesi
- Gradient (linear/radial) içeren view'ların ilk çizimi
- Shadow veya elevation oluşturulduğunda
- Animated border veya mask'li view'lar
- Lottie / Skia animasyonlarının ilk frame'i
- Clipped, masked veya yarım köşeli özel layer'lar

Shader Warm-Up Nedir?

Shader warm-up, shader'ın kullanıcı arayüzünde görünmeden önce arka planda bir kere çalıştırılmasıdır. Bu sayede GPU shader'ı önceden derler ve cache'e alır.

Shader **ilk kez görünmeden** önce derlenirse, animasyonlar tamamen pürüzsüz olur.

Warm-Up Teknikleri

1. Görünmez Render (Offscreen Warm-Up)

Shader kullanacak bir view, opacity=0 veya ekran dışında olacak şekilde bir kere çizilir:

- Blur view görünmez şekilde mount edilir.
- Gradient view offscreen render edilir.
- Mask/shadow içeren view küçük bir alanda çalıştırılır.

GPU shader'ı derler fakat kullanıcı bunu görmez.

2. Android ShaderWarmUp API

Android (API 24+) için özel warm-up mekanizması:

```
kotlin class MyShaderWarmUp : ShaderWarmUp() override fun onDraw(canvas: Canvas) canvas.drawRect(...) canvas.drawCircle(...) canvas.drawGradient(...)
```

3. Flutter ShaderWarmUp Override

```
dart class MyWarmUp extends ShaderWarmUp @override void warmUpOn-  
Canvas(Canvas canvas) final paint = Paint()..shader = myGradient; canvas.drawRect(Rect.fromLTWH(0,  
0, 100, 100), paint);
```

4. iOS CALayer Pre-Composition

- Gradient layer
- Shadow path
- Mask layer

görünmez bir view üzerinde bir kere çizilir.

Warm-Up Yapılmadığında Sonuç

- İlk karede **jank**
- Modal açılışlarında takılma
- Blur/gradient gibi efektlerde ilk kullanım donması
- Lottie/Skia animasyonlarının ilk frame'de drop yaşaması

Warm-Up Yapıldığında Sonuç

- Animasyonlar ilk kareden itibaren pürüzsüz olur
- Blur/gradient/shadow efekti kesintisiz başlar
- Modal ve page transition'lar native seviyede akıcı olur

Kısa Özet

Shader warm-up, GPU'nun shader'ı kullanıcı görmeden önce derlemesini sağlayarak, ilk karede yaşanan jank'i tamamen ortadan kaldıran kritik bir performans optimizasyonudur.

Precomputed Layout Caches

Büyük listelerde (FlatList, RecyclerView):

- cell height önceden hesaplanır,
- ölçüm cache'lenir,
- layout deterministik tutulur.

Böylece:

- no remeasure,
- no layout thrash,
- hızlı “jump scroll”.

App Sandbox Escape Vectors

Mobil uygulamalar sandbox içindedir. Tipik escape hataları:

- insecure WebView JS bridge,
- hatalı deep-link parsing,
- uygunsuz file provider izinleri,
- arbitrary file read/write,
- Keychain/Keystore yanlış kullanımı.

Bunlar mobil güvenliğin temel taşlarıdır.

Mobile Snapshot State Restoration (iOS)

iOS, bir uygulama arka plana geçtiği anda mevcut ekranın **snapshot** olarak adlandırılan bir görüntüsünü alır. Bu snapshot, App Switcher’da gösterilmek ve uygulama yeniden açıldığında ilk frame gelene kadar hızlı bir görsel oluşmasını sağlamak için kullanılır.

Snapshot Ne Zaman Alınır?

- Kullanıcı Home tuşuna bastığında,
- App arka plana geçtiğinde,
- App Switcher açıldığında,
- Uygulama sistem tarafından askıya alınmadan hemen önce.

Snapshot dosyaları genellikle `Library/Caches/Snapshots` altında tutulur ve uygulama kapalıyken dahi diskte kalabilir.

Yan Etkiler

1. Eski UI'nın Restore Edilmesi

Uygulama yeniden açıldığında gerçek UI yüklenene kadar eski snapshot kısa bir süre görünebilir. Bu durum:

- eski bir ekranın “flaş” şeklinde görünmesine,
 - kapatılmış bir modalın görünür gibi olmasına,
 - yüklenmemiş state'in yanlış algılanmasına
- neden olabilir.

2. Hassas Bilgilerin Görüntülenmesi

Snapshot içinde aşağıdaki bilgiler bulunabilir:

- Kullanıcı adı ve profil bilgileri,
- Banka hesap bakiyesi,
- Kart numaralarının görüntülenen kısımları,
- Mesajlar, QR kodlar veya ödeme ekranları,
- Gizli içerikler veya özel sayfalar.

Bu snapshot, App Switcher'da görünür ve cihaz jailbreak edilirse dosya sisteminden okunabilir.

3. Yanlış State Sunulması

Snapshot görüntüsü, uygulamanın gerçek durumu yüklenmeden önce kısa süre görüldüğü için:

- “*Para gönderme başarılı*” gibi bir ekran anlık olarak görünüp kaybolabilir,
- Loading ekranı yerine eski bir ekran görünebilir,
- Kullanıcı arayüzünde karışıklığa neden olabilir.

Bankalar Neden Blur/Obfuscation Kullanır?

Bankacılık ve güvenlik odaklı uygulamalar snapshot'ın hassas içeriği göstermemesi için, uygulama arka plana girerken ekranın üstüne blur veya solid bir maske ekler. Bu maske snapshot tarafından yakalanır ve gerçek içerik gizlenmiş olur.

Örnek iOS uygulaması:

```
swift func applicationDidEnterBackground(application : UIApplication){letblurView = UIVisualEffectView
func applicationWillEnterForeground(application : UIApplication){window?.viewWithTag(999)?.removeFromSuperview}
React Native tarafında bu işlem:
```

- `AppState` ile arka plana giriş,
- Üste bir `BlurView` veya mask eklenmesi,
- Foreground'da maskenin kaldırılması

şeklinde uygulanır.

Kısa Özet

iOS, uygulama arka plana geçerken ekranın snapshot'ını alır. Bu snapshot hassas bilgiler içerebileceği veya yanlış UI gösterebileceği için, bankalar ve hassas uygulamalar blur/obfuscation yöntemiyle görüntüyü maskeleyerek gizlilik ve UX tutarlılığı sağlar.

React Native Mimarisi

Bu doküman React Native'in yeni mimarisini oluşturan temel kavramları (Fabric, JSI, JNI, TurboModules, Shadow Tree, Gesture (Dokunma) Handler) ve modern mobil işletim sistemlerinde kullanılan bellek yapılarını (stack, heap, JS heap, C++ heap, GPU heap, ART heap, fragmentation) bütüncül ve sistemsal bir düzende açıklar.

React Reconciler (JS)

Tanım

React Reconciler, kullanıcı arayüzünün hangi bölümlerinin yeniden oluşturulması (re-render) gerektiğine karar veren **mantık katmanıdır**. UI'yı çizmez; yalnızca *hangi değişikliklerin* yapılacağına karar verir.

Görevi

Bir component yeniden render edilince:

1. Eski Fiber ağacı (previous tree)
2. Yeni Fiber ağacı (next tree)

karşılaştırılır (diffing). Bu karşılaştırmanın sonucunda şu **mutations** oluşturulur:

- node ekleme
- node silme
- node güncelleme
- node taşıma

Bu değişiklik listesi, Renderer katmanına iletilir.

Reconciler vs Renderer

React Native'de Reconciler yalnızca *karar mekanizmasıdır*. Gerçek UI güncellemelerini yapan ise:

- **Fabric Renderer**
- veya (eski mimaride) **UI Manager**

Reconciler:

- Ne değişti?
- Hangi component yeniden render edilmeli?
- Hangi node silinmeli / güncellenmeli / eklenmeli?

Renderer:

- Reconciler'ın bildirdiği değişiklikleri native UI'ya uygular.
- View oluşturur, günceller, layout hesaplatır.

Neden "Reconcile" Denir?

Çünkü yeni UI ile eski UI *uzlaştırılır* (reconcile edilir). Amaç: minimum değişiklik ile UI'yı güncellemek.

Fiber Node Nedir?

Tanım

Fiber Node, her bir React componentinin bellekteki *kışisel temsilidir*. UI'da görünen her component, Reconciler tarafından yönetilen bir Fiber Node ile temsil edilir.

Fiber Tree

Tüm Fiber Node'lar hiyerarşik bir yapı oluşturur:

```
App
  Header
    Title
  Body
    Card
    Button
```

Bu yapıda her kutu, bir **Fiber Node** demektir.

Fiber Node İçeriği

Bir Fiber Node, ilgili component hakkında aşağıdaki bilgileri taşır:

- **type** — Component türü (Function Component, View, Text vs.)
- **pendingProps** — Henüz işlenecek yeni props
- **memoizedProps** — Son render'a ait props
- **stateNode** — Component'in karşılık gelen native UI node'u
- **child / sibling / return** — Fiber ağaç bağlantıları
- **flags** — Hangi değişikliğin yapılması gerektiği (update, delete, placement)
- **lanes / priority** — React 18 concurrency öncelik bilgileri

Fiber Node'un Görevi

1. Reconciler, eski ve yeni Fiber Node'ları karşılaştırarak gerekli UI değişikliklerini hesaplar.
2. Renderer (ör: Fabric), Fiber Node'da belirtilen değişimleri native UI üzerinde uygular.

Neden Önemlidir?

Fiber mimarisi sayesinde React:

- render işlemine ara verebilir ve sonra devam edebilir (interruptible rendering)
- iş yüklerini önceliklendirebilir (lanes, priority)
- daha büyük UI ağaçlarında jank'ı azaltabilir
- async rendering yapabilir

Fiber Node = Her React componentinin bellekteki temsilidir; Reconciler bu Fiber Node'lar üzerinden karşılaştırır ve hangi UI değişikliklerinin uygulanacağına karar verir.

JSI (JavaScript Interface)

JSI, React Native'in yeni mimarisinde JavaScript kodu ile C++ katmanı arasında **yüksek performanslı bir köprü** sağlayan düşük seviyeli bir arabirimdir. Eski Bridge mimarisindeki JSON tabanlı mesajlaşmayı ortadan kaldırarak, **pointer düzeyinde doğrudan erişim** sunar.

Amaç

JSI'nin temel amacı, JavaScript tarafının:

- C++ objelerine,
- TurboModule fonksiyonlarına,
- Native bileşen durumlarına,
- Hermes (veya JSC) motoru içindeki runtime yapısına

doğrudan, hızlı ve senkron şekilde erişebilmesini sağlamaktır.

Öne Çıkan Özellikler

- **Serileştirme gerektirmez:** Eski Bridge gibi JSON stringify/parse yoktur.
- **Senkron çağrılar:** Native fonksiyonlar anında cevap verebilir.
- **Düşük gecikme:** UI-thread ile JS-thread arasında minimum overhead.
- **TurboModules desteği:** Tüm yeni native modüller JSI üzerinde çalışır.
- **HostObjects desteği:** C++ objeleri JS tarafında gerçek JS objesi gibi kullanılabilir.
- **Cross-runtime esnekliği:** Hermes, JSC ve gelecekteki tüm motorlardan bağımsız çalışır.

JSI, JavaScript motorunun *runtime* yapısına erişebilir ve bu runtime üzerinden C++ fonksiyonlarını JavaScript fonksiyonu olarak export eder.

Bu süreçte:

1. JS tarafı bir fonksiyon çağırır.
2. JSI runtime, bu çağrıyı C++ HostFunction'a yönlendirir.
3. HostFunction native kodu işletir.
4. Sonuç JSI üzerinden JS'e geri döner.

Eski Bridge ile Karşılaştırma

- Bridge **async** + **JSON** modeline dayanırdı.
- JSI ise **sync** + **pointer-level** bir yapıdır.
- Bridge yüksek latency yaratırdı; JSI **nanosaniye seviyesinde** çalışır.
- Bridge batching yaparken; JSI **anında işleme** yapabilir.

Sonuç

JSI, React Native'in yeni mimarisini mümkün kılan temel teknoloji olup, Fabric Renderer, TurboModules ve concurrent rendering gibi modern özelliklerin tamamı bu katman üzerine inşa edilmiştir.

Fabric Renderer Nedir?

Tanım

Fabric Renderer, React Native'in yeni mimarisinde Reconciler tarafından üretilen UI değişikliklerini (mutations) **gerçek Native UI bileşenlerine uygulayan** modern çizim motorudur. Eski mimarideki *UI Manager* katmanının tamamen yerini alır.

Temel Görevleri

Fabric, Reconciler'ın belirlediği UI güncellemelerini şu adımlarla yönetir:

1. Yeni UI node'larını oluşturur (UIView, ViewGroup vb.)
2. Yoga ile layout hesaplamalarını gerçekleştirir
3. Shadow Tree'yi güncelleyip Native Tree ile senkron tutar
4. Stil, prop ve event güncellemelerini native katmana uygular
5. Commit Phase sırasında tüm değişiklikleri ekrana yansıtır

Özellikleri

- **Senkron iletişim:** JSI üzerinde çalıştığı için Bridge gerekmez.
- **Düşük gecikme:** JSON serileştirmesi olmadığı için hızlıdır.
- **Shadow Tree yönetimi:** Hesaplama ve rendering için ayrılmış bir veri yapısı.
- **Yoga destekli layout:** Flexbox tabanlı ölçüm ve konumlandırma.
- **Concurrent Rendering uyumlu:** React 18 ile tam entegredir.
- **Platform bağımsız:** iOS ve Android için tek tasarım.

Shadow Tree ve Native Tree

Fabric, iki ayrı veri yapısı kullanır:

- **Shadow Tree:** UI'nın hesaplama ve layout için kullanılan hafif, sanal temsildir.
- **Native Tree:** Platformun gerçek UI node'larıdır (UIView, CALayer, ViewGroup vb.).

Fabric bu iki ağacı sürekli olarak *senkron* tutar.

Eski Mimari ile Karşılaştırma

- Bridge + UIManager → **asenkron, JSON tabanlı, yavaş**
- Fabric → **senkron, pointer-level, yüksek performanslı**
- Bridge batching yapardı; Fabric **anında commit** edebilir.
- Eski mimari jank yaratırdı; Fabric **low-latency rendering** sağlar.

Sonuç

Fabric Renderer, React Native'in modern mimarisinin merkezinde yer alır. JSI üzerinden yüksek hızlı etkileşim, Yoga tabanlı layout, Shadow Tree yönetimi ve concurrent rendering desteği ile, önceki nesil UI Manager'ı tamamen geçersiz kılarak çok daha performanslı bir UI oluşturma süreci sağlar.

Yoga Layout Engine Nedir?

Tanım

Yoga, React Native tarafından kullanılan, Facebook tarafından geliştirilmiş bir **Flexbox tabanlı layout (yerleşim) motorudur**. Tüm UI bileşenlerinin genişlik, yükseklik ve ekrandaki konumu Yoga tarafından hesaplanır.

Amacı

Yoga, Shadow Tree üzerindeki her node için:

- boyut hesaplama (width, height),
- konum belirleme (x, y),
- margin/padding/border çözümleri,
- Flexbox kurallarının uygulanması

gibi tüm layout işlemlerini yönetir.

Yoga'nın Katıldığı Mimari Süreç

FiberTreeReconcilerShadowTreeFabricYogaNodeTreeLayoutCalculationNativeUI

Hesapladığı Flexbox Özellikleri

Yoga, CSS Flexbox modelinin sadeleştirilmiş bir implementasyonunu içerir. Aşağıdaki özellikler Yoga tarafından hesaplanır:

- `flexDirection`
- `flexGrow`, `flexShrink`, `flex`
- `justifyContent`
- `alignItems`, `alignSelf`
- `flexWrap`
- `padding`, `margin`, `border`
- `minWidth`, `maxWidth`
- `minHeight`, `maxHeight`

Temel Özellikleri

- **C++ implementasyonu:** Yüksek performanslıdır.
- **Deterministic:** Her zaman aynı girdilerden aynı layout çıkar.
- **Platform bağımsız:** iOS ve Android için tek algoritma.
- **Layout caching:** Gereksiz tekrar hesaplamayı engeller.
- **Measure function:** Dinamik olarak ölçülen (ör. Text) bileşenler için özel ölçüm fonksiyonları sağlar.

Shadow Tree ile İlişkisi

Her React Native UI bileşeni için:

- bir **Shadow Node**,
- bu shadow node içinde bir **Yoga Node**

bulunur. Yoga, Shadow Tree üzerindeki tüm Yoga Node'larını hesaplar ve sonuçları Native Tree'ye aktarılır.

Neden Kritiktir?

- React Native'in tüm layout sistemi Yoga'ya dayanır.
- Flexbox davranışlarının tamamı Yoga'nın algoritmasına bağlıdır.
- Performans optimizasyonu (özellikle listeler ve animasyonlarda) Yoga mantığını bilmeyi gerektirir.

Tek Cümle Özet

Yoga = React Native'in Flexbox tabanlı matematiksel layout motorudur; her bileşenin boyutunu ve konumunu hesaplayarak UI'nın ekrana doğru şekilde yerleşmesini sağlar.

React Native Render Pipeline (Dikey Akış)

1. JS Kodundan Ekrana

```
React Component (JS)
|
v
React Reconciler (JS)
|
v
Fiber Node oluşturulur
|
v
Props ve State çözülür
|
v
JSI (JS -> C++)
|
v
Fabric Renderer (C++)
|
v
Shadow Tree (C++)
|
v
Yoga Layout Engine
|
v
Layout hesaplama (width, height, position)
|
v
Shadow Tree Diff
|
v
Fabric Mount Instructions
|
v
UI Manager (C++)
|
v
UI Thread
|
v
Native View Hierarchy (UIView / ViewGroup)
|
v
```

```
GPU Render Pipeline
|
v
Tile Rendering
|
v
Display Buffer
|
v
Ekрана çizim
```

2. Gesture / Touch Pipeline

```
Kullanıcı dokunur
|
v
Touch Sampling (donanım)
|
v
Hit Test (UI Thread)
|
v
Gesture Handler (native)
|
v
JS Thread (gerekirse)
|
v
React Reconciler
|
v
setState / useState
|
v
JSI
|
v
Fabric Renderer
|
v
Shadow Tree update
|
v
Yoga Layout
|
v
Diff
|
v
Mount Instructions
|
v
UI Thread
|
v
Native View güncellemesi
|
```

v
GPU Render
|
v
Ekranda animasyon görsel olarak güncellenir

3. Reanimated / UI Thread Animasyon Akışı

```
Reanimated Worklet (UI Thread)
|
v
Frame-by-frame hesaplama
|
v
Style/Layout güncellemeleri
|
v
Shadow Tree update
|
v
UI Thread commit
|
v
GPU güncellemesi
|
v
Animasyon ekranda görünür
```

4. TurboModule / Native Module Çağrısı

```
JS Kod (MyModule.doSomething())  
|  
v  
JSI  
|  
v  
C++ TurboModule  
|  
v  
JNI  
|  
v  
Java/Kotlin Native Module  
|  
v  
Android / iOS Native API  
|  
v  
Sonuç döner  
|  
v  
JNI  
|  
v  
C++  
|  
v  
JSI  
|  
v  
JS tarafına geri dönüş
```

TurboModules

TurboModules, eski Bridge tabanlı NativeModules yerine JSI tabanlı, yüksek performanslı modül yapısıdır.

Avantajları

- JSI üzerinden hızlı iletişim
- Senkron native API çağrıları
- Auto-codegen desteği

Gesture Handler

Varsayılan JS tabanlı gesture sistemi yavaş olduğu için, Gesture Handler, UI thread üzerinde çalışan gerçek native gesture motorudur.

- JS thread'e bağımlı değildir.
- Pan / Tap / Press / Pinch native düzeyde işler.
- Reanimated ile birleşince animasyon + input tamamen UI thread'dedir.

Bellek Modelleri ve Heap Türleri

1. Stack Bellek

Küçük, geçici fonksiyon değişkenleri stack üzerinde tutulur.

```
cpp void foo() int x = 5; // stack
```

Özellikler:

- Küçük ve hızlıdır.
- Fonksiyon bitince otomatik temizlenir.

2. Heap Bellek

Heap, büyük ve dinamik verilerin tutulduğu RAM bölgesidir.

- Dinamik allocation (new/malloc)
- Büyük objeler
- Out-of-memory hataları burada olur

3. JS Heap

JS engine (Hermes/JSC) tarafından yönetilir.

- React state
- JS objeleri, closure'lar
- Genellikle küçüktür (20–60 MB)

4. Java Heap (ART Heap)

Android Runtime (ART) tarafından yönetilir.

- Activity/Fragment/Compose objeleri
- Java/Kotlin tarafındaki state
- GC ile yönetilir

5. Native Heap (C++ Heap)

React Native'in kritik kısmıdır.

- TurboModules
- JSI runtime
- Skia

- Camera / bitmap buffers
- Fabric/Shadow Tree C++ objeleri

En tehlikeli sorun: **heap fragmentation**.

6. GPU Heap

GPU driver tarafından yönetilir.

- textures
- shaders
- framebuffers
- render targets

GPU heap dolarsa:

- shader compile fail
- texture allocate fail
- jank ve frame drop

7. Heap Fragmentation

Fragmentation, heap'in küçük parçalı boşluklara bölünmesidir.

Boş Alan = [50KB] + [40KB] + [70KB] ancak Gerekli = 150KB

Olası sonuçlar:

- Kamera init fail
- Large bitmap decode fail
- Native OOM

Hybrid Thread Models (React Native)

Modern React Native (Fabric + TurboModules) mimarisi, eski köprü (bridge) modelinden farklı olarak çoklu thread üzerinde çalışan **hibrit bir yapı** kullanır. Bu yapı, yanlış thread üzerinde yapılan state güncellemeleri veya işlemler sonucunda jank, race condition veya crash oluşturabilir.

React Native'deki Temel Thread'ler

- **JS Thread** — Uygulamanın JavaScript mantığının çalıştığı alan.
- **UI Thread** — Native UI güncellemelerinin işlendiği ana thread.
- **Shadow Tree Thread** — Fabric layout ve ölçüm hesaplamalarının yürütüldüğü thread.
- **Native Modules Threads** — Klasik native modüllerin çalışabileceği ayrı background thread'ler.
- **TurboModules Threads** — JSI tabanlı TurboModules için concurrent native thread'ler.
- **JSI Background Threads** — C++ tabanlı JSI işlemleri için kullanılan ek background worker thread'ler.

Bu çoklu thread yapısı performans artışı sağlasa da eşzamanlılık (concurrency) sorunlarını önemli ölçüde artırır.

Neden Kritik?

Eski mimaride yalnızca iki ana thread vardı:

- JS Thread
- UI Thread

Yeni mimaride:

- Layout hesaplamaları ayrı bir thread'de,
- Native modüller farklı queue'larda,
- TurboModules senkron/async çalışabilir,
- JSI pointer tabanlı memory modeli kullanır,

Bu nedenle thread güvenliği çok daha kritiktir. Yanlış thread'de işlem yapmak *race condition*, *invalid memory access* ve *segfault* gibi hatalara yol açabilir.

Neden Yanlış Thread’de setState Crash Üretir?

React’ın render döngüsü ve fiber mimarisi yalnızca **JS Thread** üzerinde güvenli şekilde çalışır:

- Hook’lar,
- Reconciliation,
- Render scheduling,
- Batching,
- React internal state,

hep JS thread’e bağlıdır.

Bu nedenle başka bir thread’den yapılan:

`setState()`, `dispatch()`, `useState()` güncellemeleri

React’ın çalışma bağlamı aktif olmadığı için hataya yol açar.

Tipik crash mesajları:

- `Cannot update from outside React-managed thread`
- `JSI: attempted to access value from invalid runtime`
- `Fabric: commit called from wrong thread`

Jank Nereden Gelir?

- **UI Thread block** — Scroll/pan animasyonlarında drop.
- **JS Thread block** — Gesture event’lerinin gecikmesi.
- **Shadow Tree yarışması** — Layout ve commit çakışması.
- **Senkron TurboModule çağrısı** — JS thread kilitlenmesi.

Bu hatalar özellikle:

- modal açılışlarında,
- scroll + animation kombinasyonlarında,
- gesture + JS yoğun işlem eşzamanlılığında,
- JSI tabanlı native modüllerde

daha sık görülür.

Örnek Tipik Sorunlar

- “First-load jank”
- “Gesture lag”
- “Touch responder hell”
- ScrollView ile nested pan çakışmaları
- Splash sonrası UI freeze
- JSI concurrency crash’leri
- Fabric commit race condition hataları

Önerilen Çözümler

- State güncellemelerini **yalnızca JS Thread** üzerinde yap.
- TurboModules içindeki C++ kodunda thread safety (mutex/atomic) sağla.
- UI Thread’i ağır işlemlerden uzak tut.
- Layout karmaşıklığını azalt.
- Reanimated 2/3 kullanarak animasyonları UI Thread’e taşı.
- Gesture Handler v2 ile native gesture pipeline’ı kullan.
- Native modüllerde doğru queue/dispatcher yapılarını kullan.

Kısa Özet

React Native modern mimarisinde JS, UI, Shadow Tree, TurboModule ve JSI thread’leri paralel çalışır. Yanlış thread’de yapılan bir state güncellemesi hem crash hem de jank üretir; doğru eşzamanlılık yönetimi kritik önem taşır.

OS Power Profiles

Cihaz güç profilleri:

- Performance,
- Balanced,
- Power Save,
- Ultra Power Save.

Bunlar CPU scheduler'ı değiştirir. Etkiler:

- animasyon davranışları,
- kamera performansı,
- BLE scanning,
- background task kısıtlaması.

GPU Render Pass Optimization

Mobil GPU'lar (özellikle PowerVR, Adreno, Mali) **tile-based deferred rendering** (TBDR) mimarisini kullanır. Bu mimaride ekran, küçük *tiles* (karo) bloklarına bölünür ve her tile bağımsız olarak işlenir. Her tile için GPU birden fazla “render pass” yürütür. Bu pass'lerin her biri ek maliyet demektir.

Maliyeti Artıran Render İşlemleri

- **Shadows (Gölge Hesaplamaları):** Her gölge, genellikle Gauss blur kernel'i ile hesaplanır. Bu işlem tile başına onlarca ek örnekleme (*samples*) gerektirir. Mobil GPU'larda bu işlem, toplam render süresini katlayabilir.
- **Nested Views (İç İç Yapılar):** Her nested view, potansiyel:
 - ek clipping,
 - ek compositing,
 - ek overdraw

yaratır. Overdraw, aynı pikselin birden fazla kez çizilmesidir.

- **Animated Clipping:** Border radius, mask path veya clip rect animasyonla değişiyorsa, GPU her frame bunu *yeniden hesaplamak* zorundadır. Bu, özel bir “offscreen pass” oluşturur ve oldukça pahalıdır.

Neden Bu İşlemler Pahalıdır?

- Render pass sayısı arttıkça tile başına işlenen *fragment* sayısı artar.
- Blending, gölgeler ve clip işlemleri, GPU'nun **fill-rate** limitlerine çarpar.
- Her ek pass, hafıza bant genişliğini (memory bandwidth) tüketir.
- Mobil GPU'lar CPU kadar güçlü değildir; özellikle fragment shader yükü arttığında “thermal throttling” devreye girer.

Sonuç: Performans Kaybı

Bu maliyetlerin birikimi şunlara yol açar:

- **frame drop (FPS düşüşü),**
- **jank (UI takılması),**
- **yüksek GPU tüketimi → batarya tüketimi,**
- **termal throttling → performansın sürekli düşmesi,**
- **gecikmeli animasyonlar ve input lag.**

Zero-Copy Buffer Transfer

Tanım

Zero-copy, verinin (görsel, audio, JSON buffer'ı) kopyalanmadan, sadece **pointer paylaşımı** üzerinden bir katmandan diğerine aktarılmasıdır. Normalde buffer transferi sırasında:

- memcpy
- marshalling
- serialization
- tekrar decode

gibi maliyetli işlemler gerçekleşir.

Zero-copy, bu adımların tamamını **bypass ederek** hem CPU hem de batari maliyetini düşürür.

Neden Önemlidir?

- 20–50 kat performans artışı sağlar.
- GC baskısını azaltır.
- UI thread üzerinde freeze / jank riskini düşürür.
- JS ↔ Native köprüsünde (Bridge) tıkanmayı azaltır.

React Native Bağlamı

- JSI (JavaScript Interface) zero-copy transferi destekler.
- Skia tabanlı rendering motorları (ör. React Native Skia) texture buffer'larını doğrudan GPU'ya aktarabilir.

GPU Overdraw Heatmaps

Tanım

Overdraw, bir pikselin ekranda bir frame içerisinde kaç kez çizildiğini gösterir. Mobil GPU'larda overdraw GPU maliyetini dramatik biçimde artırır.

Overdraw Seviyeleri

- **0x** — İdeal durum
- **1x** — Normal
- **2x** — Opacity, shadow veya nested view kaynaklı
- **3x** — Yanlış hiyerarşi, performans problemi
- **4x+** — Kritik performans hatası (katastrofik)

React Native Etkisi

Flexbox tabanlı UI hiyerarşisi:

- nested Views
- opacity overlay
- zIndex manipülasyonları
- shadow + border kombinasyonları

ile kolayca 3x-4x overdraw üretir.

Heatmap Araçları

Android:

- Developer Options → Debug GPU Overdraw

iOS:

- Xcode Instruments → Core Animation → Color Offscreen-Rendered

Adaptive Bitrate Ladders (ABR)

Tanım

Adaptive Bitrate (ABR), video akış kalitesinin anlık network koşullarına göre dinamik olarak değiştirilmesi tekniğidir.

Tipik Bitrate Ladder

- 144p
- 240p
- 360p
- 480p
- 720p
- 1080p

Mobil Network Problemleri

- LTE → 3G düşüşleri
- ani bandwidth düşüşleri
- congestion (tıklanma)
- captive network geçişleri
- WiFi ↔ LTE geçişi

React Native Etkisi

Birçok React Native video player:

- ABR algoritması içermez,
- sadece bir kalite seçer veya sabit bitrate oynatır,
- buffer underrun yaşatır,
- network değişiminde drop / freeze üretir.

React Native Concurrent Mode (Experimental)

Amaç

React Native'in yeni mimarisi (Fabric) ile birlikte, React 18'in concurrent rendering sistemini mobile taşımaktır.

Özellikler

- **Interruptible rendering** — UI render işlemi gerektiğinde durdurulur.
- **Selective hydration** — sadece görünen bölgeler interaktif hale getirilir.
- **Priority-based updates** — scroll gibi işlemler yüksek önceliklidir.
- **Background rendering** — ağır UI işlemleri arkaplanda yürütülür.

Avantajlar

- Scroll sırasında pürüzsüzlük
- jank'ın azalması
- büyük listelerde daha akıcı performans
- render işlemlerinin frame bütçesine uygun dağıtılması

Hermes Bytecode Optimizations

Hermes Motorunun Optimizasyonları

- **Bytecode preloading** — bundle daha hızlı yüklenir.
- **Inline caching** — sık çağrılan fonksiyonlar optimize edilir.
- **String interning** — string duplication azaltılır.
- **Register-based VM** — stack-based VM'e göre daha hızlıdır.
- **Hidden classes** — JS objeleri için içsel sınıf yapıları oluşturur.
- **JIT-less optimization** — düşük hafıza cihazlarda avantaj sağlar.

Yanlış Yapılandırmanın Etkileri

- uygulama açılışı yavaşlar (cold start)
- JS heap leak'leri
- bridge üzerinde tıkanma

Doğru Yapılandırmanın Etkileri

- %20-%30 performans artışı
- daha hızlı startup
- daha düşük GC baskısı

Native Module Lifetime Management

Tanım

React Native Native Modules, hem JS runtime hem de native runtime tarafından kontrol edilen karmaşık yaşam döngülerine sahiptir.

Yaşam Döngüsü Aşamaları

- **Instantiation** — modül oluşturulur.
- **Memoization** — modül cache edilir.
- **Thread Affinity** — modülün hangi thread'de çalışacağı belirlenir.
- **JS Binding** — JS ile native fonksiyonlar bağlanır.
- **Teardown** — modül bellekten temizlenir.
- **Weak references** — JS tarafında referans kaybı olup olmadığı izlenir.

Hatalı Yönetimin Sonuçları

- Memory leak (özellikle kamera modülleri)
- Zombie instances (silinmiş ama yaşayan modüller)
- Crash (çift free, use-after-free)
- Thread conflict (yanlış thread'de çalıştırma)

Özellikle Hassas Modüller

- Camera modules
- Audio recording modules
- Bluetooth / BLE modules
- Video encoders

Emülatörler ve Simülatörler

Bu bölümde, mobil geliştirme sürecinde sık kullanılan emülatör ve simülatör kavramlarını, Android emülatörlerinin pratik kullanımını, React Native ile etkileşimini ve tipik sorun-çözüm kalıplarını sistematik biçimde ele alıyoruz. Amaç; geliştiricinin hem günlük pratikte kullanabileceği hem de altında yatan mantığı anlayabileceği bir çerçeve sunmaktır.

Emülatör ve Simülatör Arasındaki Fark

Emülatör (Emulator)

Emülatör, gerçek bir cihazın donanım ve işletim sistemi davranışını mümkün olduğunca *birebir taklit etmeye* çalışan sistemdir. Android emülatör:

- Gerçek bir Android işletim sistemi *image*'i çalıştırır.
- Sanal bir cihaz (telefon/tablet) donanımı simüle eder.
- CPU seviyesinde emülasyon veya donanım hızlandırmalı sanallaştırma kullanır.
- React Native geliştirme sürecinde, Windows ve macOS ortamlarında ana test cihazı olarak kullanılır.

Simülatör (Simulator)

Simülatör, donanımı tam olarak emüle etmekten ziyade, hedef ortamın davranışını *yaklaştıran* bir model sunar. iOS Simulator:

- Sadece macOS üzerinde çalışır.
- iOS'in simüle edilmiş bir sürümünü çalıştırır; gerçek donanımı birebir taklit etmez.
- Çoğu senaryoda yeterli olsa da, bazı donanım seviyesindeki davranışları gerçeğe göre farklılık gösterebilir.

Pratik düzeyde, React Native geliştiricisi için:

- Windows ortamında ana odak: **Android Emulator**.
- macOS ortamında: Android Emulator + iOS Simulator birlikte kullanılabilir.

AVD (Android Virtual Device) Mantığı

Her Android emülatörü, bir *sanal cihaz profili* (*Android Virtual Device – AVD*) üzerinden tanımlanır. Bu profil, aşağıdaki bileşenlerden oluşur:

- **Cihaz modeli:** Örneğin Pixel 4, Pixel 7, tablet profilleri.
- **Android sürümü:** API seviyesi (ör. API 29, 30, 33, 34).
- **Google Play / Google APIs:** Emülatörün Google Play veya Google servisleri ile gelip gelmediği.
- **Donanım kaynakları:** RAM miktarı, dahili depolama, ekran çözünürlüğü.
- **Ekran yönü ve yoğunluğu:** *portrait* / *landscape*, dpi ve çözünürlük değerleri.

Performans ve Donanım Hızlandırma

Emülatör performansını belirleyen en kritik unsur, donanım hızlandırmadır.

Sanallaştırma Teknolojileri

- BIOS/UEFI seviyesinde **sanallaştırma** (Intel VT-x, AMD-V) açık olmalıdır.
- Windows ortamında Hyper-V, WSL2, HAXM, WHPX gibi teknolojiler, zaman zaman emülatörle çakışabilir.
- Emülatör ciddi şekilde yavaşsa, çoğu zaman problem sanallaştırma ayarları veya çakışan hypervisor'lardır.

AVD Performans Ayarları

AVD ayarlarında dikkat edilmesi gereken temel noktalar:

- **Graphics:** Mümkünse *Hardware* (*GPU*) hızlandırma seçilmelidir.
- **RAM:** Çok düşük RAM emülatörü yavaşlatır, çok yüksek RAM ise ana makineyi zorlayarak genel sistemi yavaşlatır.
- **Açık emülatör sayısı:** Aynı anda birden fazla AVD çalıştırmak, geliştirme makinesini gereksiz yere zorlar. Gündelik iş akışında genellikle tek AVD yeterlidir.

Ağ (Network) ve localhostlocalhost Meselesi

Emülatör, ayrı bir cihaz gibi davrandığı için `localhost` kavramı geliştirici makinesinden farklıdır. Önemli nokta:

- Emülatör içinden `http://localhost:3000` çağrıldığında, bu adres emülatörün kendi içindeki `localhost`'u ifade eder; geliştirici makinesini değil.

Android emülatörünün, host makineye (geliştiricinin bilgisayarına) erişmesi için özel bir IP kullanılır:

- **Android Emulator** için: `10.0.2.2`

Örneğin, backend geliştirme makinesinde `http://localhost:3000` adresinde çalışıyorsa, emülatör içinden şu adres kullanılmalıdır:

`http://10.0.2.2:3000`

Bu ayrımı yapmamak, mobil geliştirmede en sık görülen ağ hatalarından biridir.

Emülatör ve Gerçek Cihaz Arasındaki Farklar

Emülatörlerin Avantajları

- Hızlı geliştirme döngüsü: Derleme sonrası uygulamayı emülatörde hemen yeniden yükleyebilme.
- Farklı ekran boyutları, çözünürlükler ve Android sürümlerini denemeyi kolaylaştırması.
- Fiziksel cihazı sürekli kablo ile bağlama ihtiyacını azaltması.

Emülatörlerin Sınırlılıkları

- Zayıf donanımlı bilgisayarlarda performans ciddi şekilde düşebilir.
- Gerçek donanım davranışını birebir yansıtmayabilir:
 - Sensörler (GPS, akselerometre, jiroskop) sınırlı veya simüle edilmiştir.
 - Dokunma hissi, gesture tepkileri ve klavye davranışları gerçek cihazlardan farklı olabilir.
- İzinler, kamera erişimi, bildirimler ve üreticiye özel davranışlar, gerçek cihazlarda farklı sonuçlar üretebilir.

İşlemci Mimarileri, Sistem İmajları ve Sanallaştırma

Emülatör performansını ve stabilitesini belirleyen en kritik teknik bileşenlerden biri, **işlemci mimarisi** ve kullanılan **sistem imajı (system image)** türüdür. Bu bölümde:

- geliştirici makinesinin işlemcisi (Intel/AMD/Apple Silicon),
- Android emülatör sistem imajlarının mimarileri (x86, x86_64, ARM),
- sanallaştırma katmanı (virtualization layer)

arasındaki ilişkiyi kavramsal olarak açıklıyoruz.

Host Makine İşlemcisi (Intel, AMD, Apple Silicon vb.)

Geliştirici bilgisayarınızın markası (MSI, Lenovo, Dell vb.) teknik olarak ikincil önemdedir; asıl kritik olan, alttaki **işlemci mimarisidir**:

- **Intel x86_64** veya **AMD x86_64** işlemciler (masaüstü / laptopların büyük kısmı).
- **ARM tabanlı işlemciler** (özellikle Apple Silicon – M1, M2, M3).

Android emülatörler, bu işlemci mimarisinin üzerine bir sanal cihaz katmanı kurar. Bu nedenle:

- Intel/AMD bir makinede, x86/x86_64 tabanlı sistem imajları genellikle *en hızlı* çalışan emülatörleri üretir.
- ARM tabanlı bir makinede (örneğin Apple Silicon), ARM veya ARM uyumlu sistem imajları tercih edilir; x86 emülasyonu ek performans maliyeti oluşturabilir.

Android Sistem İmajı Mimarileri

Android Virtual Device (AVD) oluştururken seçtiğiniz *system image*, temelde iki önemli boyuta sahiptir:

1. **API seviyesi** (Android 12, 13, 14 vb.).
2. **Mimari türü (ABI)**:
 - x86
 - x86_64
 - armeabi-v7a (32-bit ARM)
 - arm64-v8a (64-bit ARM)

x86 / x86_64 İmajlar

- Intel/AMD masaüstü/laptop işlemcileri ile doğal olarak uyumludur.
- Donanım hızlandırması (hardware acceleration) ile birlikte **çok daha yüksek performans** sunar.
- Geliştirme ortamı Windows veya klasik x86_64 tabanlı macOS ise, *günlük geliştirme için ilk tercih* genellikle x86_64 imajlardır.

ARM Tabanlı İmajlar

- Gerçek Android cihazların çoğu ARM tabanlı olduğu için, *ülätör bu mi-mariyi emüle edebilir.*
- *x86 bir host makinede ARM imajı çalıştırmak*, ek emülasyon katmanı gerektirdiği için *belirgin performans kaybına neden olur.*
- *Apple Silicon gibi ARM tabanlı host sistemlerde ise, ARM imajlar doğal olarak daha verimlidir.*

Geliştirici açısından özet kural:

- Host işlemci **x86_64** ise $\Rightarrow$ mümkün olduğu kadar **x86_64 system image** kullan.
- Host işlemci **ARM** (örneğin Apple Silicon) ise $\Rightarrow$ ARM tabanlı imajlar daha doğal ve genellikle daha hızlıdır.

Sanallaştırma Katmanı (Virtualization Layer)

Android emülatör, sistem imajını çalıştırmak için işletim sisteminin sunduğu sanallaştırma özelliklerini kullanır. Bu katman:

- **BIOS/UEFI** seviyesinde *donanım sanallaştırmasının* (Intel VT-x, AMD-V) açık olmasına bağlıdır.
- Windows tarafında Hyper-V, WSL2, WHPX gibi hypervisor'lar ile etkileşim içindedir.

Pratik Etkiler

- Eğer BIOS/UEFI'de sanallaştırma kapalıysa, emülatör ya hiç başlamaz ya da *son derece yavaş* çalışır.
- Bazı hypervisor'lar (Hyper-V, WSL2 vb.) aktifken, eski tip hızlandırma sürücüler (ör. HAXM) ile çakışma oluşabilir.
- Emülatörle ilgili “aşırı yavaş”, “takılıyor” veya “başlamıyor” türü sorunlarda, çoğu zaman kök sebep işlemci gücünden ziyade **sanallaştırma yapılandırmasıdır**.

Bilgisayarınızın Markası (MSI, Lenovo vb.) Neden İkincil Önemdedir?

Laptop veya masaüstünüzün MSI, Lenovo, Dell vb. olması, teknik anlamda doğrudan emülatör mimarisini belirlemez. Önemli olan:

- Altındaki işlemci mimarisi (Intel/AMD x86_64 veya ARM),
- BIOS/UEFI’de sanallaştırmanın açık olup olmadığı,
- İşletim sistemindeki hypervisor yapılandırması (Hyper-V, WSL2 vb.),
- AVD’de seçtiğiniz system image’in bu mimariyle uyumu.

Dolayısıyla “MSI kullanıyorum, emülatör neden yavaş?” sorusunun cevabı marka değil, çoğunlukla:

- yanlış seçilmiş system image (ARM imaj + x86_64 host),
- kapalı sanallaştırma,
- aşırı düşük RAM/GPU ayarı

gibi konfigürasyon problemleridir.

Önerilen Minimal Bilgi Seti

Bir mobil geliştiricinin emülatör–işlemci mimarisi tarafında *bilmesi yeterli olan* özet noktalar şunlardır:

1. Kendi makinesinin işlemci mimarisi nedir? (Intel/AMD x86_64 mi, yoksa ARM tabanlı mı?)
2. AVD oluştururken **x86_64** imaj seçmek, x86_64 host üzerinde en iyi performansı sağlar.
3. BIOS/UEFI’de sanallaştırma (Intel VT-x / AMD-V) açık olmalıdır.
4. Hyper-V, WSL2 vb. ortamların emülatörle çakışma ihtimali vardır; sorun yaşıyorsa bu katmanlar kontrol edilmelidir.
5. ARM system image’ler genellikle *gerçek cihaza yakın* mimari sunar, fakat x86 host üzerinde performans maliyeti yaratabilir.

Native Kütüphaneler ve .so Dosyaları (ABI Uyumunun Önemi)

Android tarafında **native kütüphaneler**, derlendiğinde genellikle **.so** (*shared object*) uzantılı dosyalar üretir. Bu dosyalar:

- C/C++ (NDK) ile yazılmış kodların derlenmiş hâlidir,
- belirli bir işlemci mimarisi (ABI) için üretilmiştir,
- APK/Bundle içinde **lib/<ABI>/libXXX.so** şeklinde paketlenir.

React Native projelerinde, birçok üçüncü parti kütüphane (örneğin depolama, şifreleme, görüntü işleme, ses/video işleme kütüphaneleri) arka planda **.so** dosyaları kullanır. Geliştirici doğrudan **.so** ile uğraşmasa da, emülatör ve cihaz mimarisi ile **.so** dosyalarının mimarisi uyumlu değilse, uygulama *çalışma zamanında* hata verir.

APK İçinde .so Yerleşimi Derlenmiş bir APK içindeki tipik dizin yapısı:

```
lib/  
  armeabi-v7a/  
    libexample.so  
  arm64-v8a/  
    libexample.so  
  x86/  
    libexample.so  
  x86_64/  
    libexample.so
```

Buradaki her klasör, farklı bir **ABI** (*Application Binary Interface*) için derlenmiş aynı kütüphanenin versiyonunu barındırır.

- **armeabi-v7a** → 32-bit ARM
- **arm64-v8a** → 64-bit ARM
- **x86** → 32-bit x86
- **x86_64** → 64-bit x86

Emülatör Mimarisinin .so ile İlişkisi

Android runtime, uygulama başlarken **libXXX.so** dosyalarını, emülatörün/cihazın mimarisi ile eşleşen klasörden yüklemeye çalışır:

- Emülatör **x86_64** ise, önce **lib/x86_64** klasörüne bakar.
- Gerçek bir telefon **arm64-v8a** ise, **lib/arm64-v8a** klasörünü kullanır.

Eğer ilgili ABI için **.so** dosyası yoksa veya **yanlış mimari** bir dosya yüklenmeye çalışılıyorsa:

- uygulama açılışta çökebilir (crash),
- ya da native modül yüklenemez ve React Native tarafında “Native module X cannot be null” gibi hatalar görülebilir.

Tipik Hata Örnekleri (Mimari Uyumsuzluğu)

Mimari uyumsuzluğunda logcat üzerinde örneğin şu tip mesajlar görülebilir:

- `dlopen failed: wrong ELF class: ELFCLASS64`
- `dlopen failed: cannot locate symbol ...`
- `java.lang.UnsatisfiedLinkError: couldn't find "libXXX.so"`

Bu tür hatalar, çoğu zaman:

- Emülatör mimarisi ile APK içindeki `.so` mimarisinin uyuşmaması,
- Gradle tarafında `abiFilters` kullanılarak bazı mimarilerin yanlışlıkla dışarıda bırakılması,
- Üçüncü parti bir kütüphanenin yalnızca sınırlı ABI desteğine sahip olması

gibi nedenlerden kaynaklanır.

React Native Geliştiricisi İçin Pratik Çıkarımlar

1. Proje, native kütüphaneler kullanıyorsa APK/BUNDLE içinde `lib/<ABI>/...` yapısı oluşur.
2. Emülatör mimarisi ile bu `lib` klasörleri arasında **mutlaka** ortak bir ABI bulunmalıdır.
3. Geliştirme makinesi `x86_64` ise, günlük geliştirme emülatörü için **`x86/x86_64 system image`** ve APK içinde **`x86/x86_64 .so`** bulundurmak performans ve stabilite açısından idealdir.
4. Gerçek cihaz dağıtımında, **`arm64-v8a`** ve gerekiyorsa **`armeabi-v7a`** desteği sağlanmalıdır; Google Play tarafında ABI split kullanılarak her cihaza uygun paket gönderilebilir.

Gradle Üzerinden ABI Kontrolü (abiFilters)

Android Gradle yapılandırmasında, hangi ABI'ler için `.so` paketleneyeceğini kontrol eden bir yapı vardır:

```
android {  
    defaultConfig {  
        ndk {  
            abiFilters "armeabi-v7a", "arm64-v8a", "x86", "x86_64"  
        }  
    }  
}
```

- Burada yalnızca `"arm64-v8a"` yazarsanız, x86 emülatörde çalıştırdığınızda ilgili `.so` bulunamayacağı için crash veya link hataları alabilirsiniz.
- Tersine, geliştirme döneminde emülatör için x86/x86_64, üretim dağıtımı için ise arm tabanlı mimariler önceliklendirilebilir.

Özet

- `.so` dosyaları, native kodun mimari bazlı derlenmiş çıktılarıdır.
- Her `lib/<ABI>` klasörü, belirli bir işlemci mimarisi içindir.
- Emülatör/cihaz mimarisi ile APK içindeki `.so` mimarilerinin eşleşmesi zorunludur; aksi hâlde yükleme hataları ve crash'ler görülür.
- React Native geliştiricisi için kritik olan; emülatör ABI'si, cihaz mimarisi ve Gradle `abiFilters` ayarlarının temel mantığını bilmek ve hata durumunda bu ekseninde düşündürmektir.

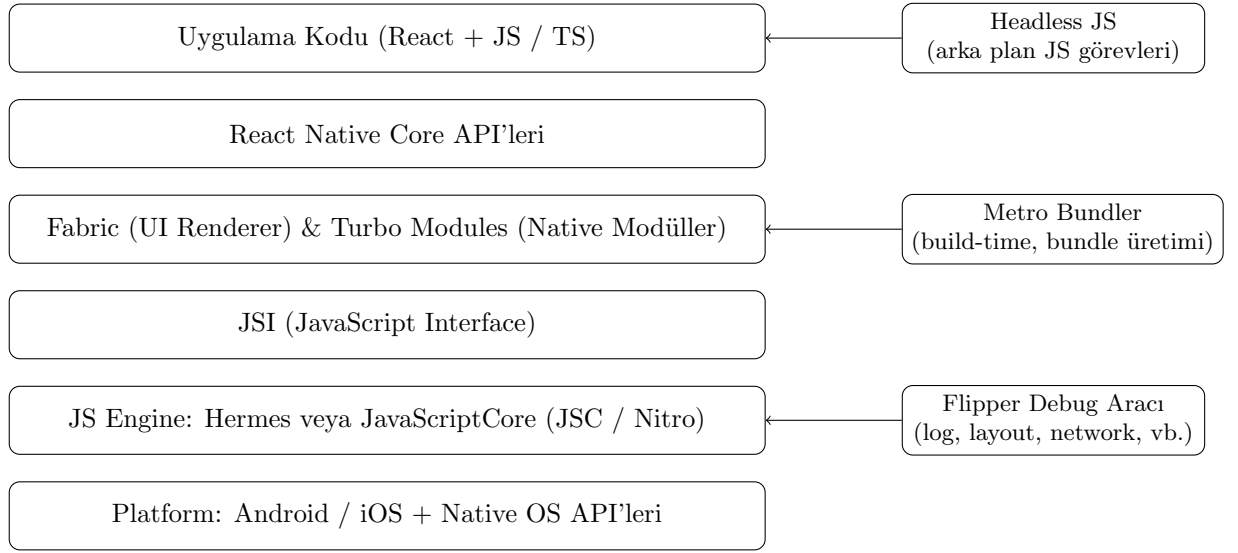


Figure 1: React Native ekosistemindeki kavramların katmanlı mimari şeması

Sonuç

Mobil Geliştiriciler İçin Giriş Rehberi, React Native merkezli bir bakış açısıyla modern mobil uygulama geliştirme sürecini anlaşılır, yapılandırılmış ve bütüncül bir forma dönüştürmeyi amaçlamıştır. Bileşen yapısı, durum yönetimi, navigasyon, yerel modül entegrasyonu, platform farklılıkları ve dağıtım süreçleri gibi başlıklar, tek tek “ipucu” düzeyinde değil, birlikte anlamlı bir resim oluşturacak şekilde ele alınmıştır.

Bu rehberin temel iddiası, iyi bir mobil geliştiricinin yalnızca kütüphane API'lerini ezbere bilmesi değil; aynı zamanda:

- uygulamanın yaşam döngüsünü,
- JS katmanıyla native katman arasındaki ilişkiyi,
- Android ve iOS ekosistemlerinin temel beklentilerini,
- performans, bakım ve ölçeklenebilirlik kaygılarını

kavramsal düzeyde anlayan bir profesyonel olması gerektiğidir. Bu nedenle metin, örnek ve kavramları birbirinden kopuk değil, aynı mimari resmin parçaları olarak sunmuştur.

Pratikte bu dokümandan alınabilecek üç ana çıktı şunlardır:

1. Mobil projeye başlarken hangi kavramları en azından “yeterince” bilmek gerektiğine dair net bir kontrol listesi.
2. React Native projelerinde karşılaşılan tipik sorunları (build, izinler, native entegrasyon, performans vb.) okurken çekinmeyecek seviyede temel platform okuryazarlığı.
3. Uzun vadede derinleşmek istenebilecek alanlar için (native köprüler, mimari desenler, performans optimizasyonu, growth ve ürün odaklı geliştirme vb.) yön gösteren bir başlangıç noktası.

Sonuç olarak, bu rehberin, mobil geliştirme dünyasına girerken “dağınık ve parça parça bilgi” hissini azaltan; yerine, nereye bakması gerektiğini bilen, kendi öğrenme yol haritasını daha bilinçli çizebilen bir geliştirici profiline doğru atılmış ilk adım olacağı varsayılmaktadır.

İsim	Türü	Açıklama
Hermes	JS engine	Meta'nın mobil için optimize ettiği JavaScript motoru. Android ve iOS'ta React Native uygulamalarında daha hızlı açılış, daha düşük bellek kullanımı hedefler; yeni projelerde sıkça varsayılan motor olarak kullanılır.
JavaScriptCore (JSC)	JS engine	Safari / iOS tarafının yerleşik JS motoru. Hermes kullanmazsan iOS'ta RN kodunu genelde JSC çalıştırır. Eski RN projelerinde varsayılan motordur.
Nitro	JIT derleyici adı	JSC'nin içindeki JIT derleyicinin ismi (Safari'de kullanılır). iOS'ta sandbox ve güvenlik kısıtları nedeniyle tam gücüyle her zaman kullanılamaz; bu yüzden RN'de JS performansı Safari'den farklı davranabilir.
Turbo Modules	Native modül mimarisi	Klasik "bridge" modeline göre daha hızlı ve tipi belli yeni native modül sistemi. JSI üzerinden doğrudan erişim sağlar; daha az kopyalama ve daha iyi performans sunar. Üçüncü parti RN kütüphaneleri yavaş yavaş bu mimariye geçmektedir.
JSI (JavaScript Interface)	Köprü katmanı	JavaScript motoru (Hermes, JSC vb.) ile C++ dünyasını bağlayan ince bir soyutlama. Engine'den bağımsız tek bir arayüz üzerinden native kodla konuşmayı sağlar. Turbo Modules ve Fabric bu katman üzerine inşa edilmiştir.
Fabric	Yeni UI / renderer sistemi	React Native'in yeni render altyapısı. Shadow tree ve layout işlemlerini daha modern bir mimariyle yöneterek özellikle büyük listeler ve karmaşık arayüzlerde daha tutarlı, performanslı ve deterministik davranmayı amaçlar.
Metro	Bundler	React Native'in varsayılan JS bundler'ı. TypeScript/JavaScript dosyalarını tek bir bundle'a çevirir, import'ları çözer, hot reload / fast refresh sağlar. Web tarafındaki webpack veya vite gibi düşünülebilir.
Headless JS	Arka plan JS modeli	UI olmadan, arka planda saf JS fonksiyonları çalıştırmaya izin verir. Örneğin background sync, push bildirim işleme gibi "ekran kapalıyken" kısa süreli işleri yürütmek için kullanılır.
Flipper	Debug aracı	RN ve diğer mobil uygulamaları debug etmek için masaüstü aracı. Network log'larını, layout yapısını, React devtools'u, AsyncStorage içeriğini görmeyi sağlar; Hermes, Fabric vb. için eklentiler (plugin) sunar.

Table 1: React Native ekosistemindeki temel kavramların özeti